

정보처리기술사 140회 대비

천기누설 

핵심토픽 200제 

CA/OS

(컴퓨터구조/운영체제)



중급반

CA/OS

NO.1	NO.2	토픽명(대)	토픽명(소)	문제	출제	공통	번호
1	1	컴퓨터기본 구조	컴퓨터기본 구조	1. 컴퓨터시스템의 다섯 가지 구성 요소를 제시하고, 폰 노이만 컴퓨터(Von Neumann Computer)의 기초가 되는 Stored-program computer의 개념에 대하여 설명하시오.	101	응용	3
2	1	주소 지정 방식	주소 지정 방식	CPU의 주소지정방식(Addressing Mode)에 대해 설명하시오.	합숙 _2019.04	공통	Day-3
3	1	가상 메모리 관리	가상 메모리 관리기법	가상메모리에 대하여 다음을 설명하시오. 가. 가상메모리 관리기법 나. 가상메모리 단편화와 해결방안	모의 _2019.11	응용	2
	2			총 3 개의 페이지로 구성된 페이지 기반 가상 메모리 시스템이 있다. 제시되는 페이지 교체 알고리즘의 페이지 교체과정을 적중률(Hit Ratio)과 함께 설명하시오. - 페이지 교체 알고리즘 : FIFO, LRU, LFU - 페이지 번호 참조 순서 : 4, 3, 2, 5, 2, 1, 3, 2, 6, 3, 4 (단, 최초 페이지 프레임은 모두 비어있다고 가정한다.)	모의 _2021.01	응용	3
	3		변환과정, TLB	페이지 방식을 이용하는 가상 메모리에서 가상주소(Virtual address)를 주기억장치 주소인 물리주소(Physical Address)로 변환하는 과정을 페이지 테이블(Page Table) 및 TLB(Translation Lookaside Buffer)를 이용하여 설명하시오.	110	응용	3
4	1	캐시 메모리	캐시 동작, 주소 사상	중앙처리장치(CPU)에서 명령어 블록을 인출하기 위한 캐시기억장치 (Cache Memory)의 동작 순서를 설명하고, 캐시기억장치의 주소 사상 (Address Mapping) 유형 별 동작 방식에 대하여 설명하시오.	모의 _2021.01	응용	3
	2		캐시 일관성	캐시 메모리의 쓰기정책(Write Policy), 캐시 일관성(Cache Coherence)을 유지하기 위한 기법에 대해 설명하시오.	모의 _2021.01	공통	4
5	1	커널, 운영체제	커널, 운영체제	운영체제 다중 프로그래밍 환경에서 이중 모드(Dual mode) 구조에 대하여 다음을 설명하시오. 가. 이중모드 개념 나. 이중모드의 구조 다. 이중모드의 종류	125	응용	4
6	1	스케줄링 알고리즘	CPU 스케줄링 알고리즘	CPU 스케줄링의 선점/비선점 방식을 설명하고, 각 방식의 알고리즘에 대하여 설명하시오	모의 _2018.10	응용	4
	2		RM/EDF	실시간 스케줄링 알고리즘인 RM(Rate Monotonic)과 EDF(Earliest Deadline First)에 대해 설명하고, 다음 3 개의 태스크(Task)를 스케줄링 하시오.	합숙 _2020.01	응용	Day-5
7	1	Deadlock	Deadlock	교착상태의 발생 조건과 해결방안을 설명하고, 회피 기법인 Banker's algorithm과 Wait-Die, Wound-Wait를 설명하시오.	합숙 _2018.08	응용	Day-3
8	1	IPC	IPC	IPC(Inter Process Communication) 통신방식	합숙 _2020.07	공통	Day-3
9	1	동기화	동기화 기법	운영체제의 프로세스 동기화 기법 중 유텍스(Mutex), 세마포어(Semaphore), 모니터(Monitor)에 대하여 설명하시오.	139	관리	4

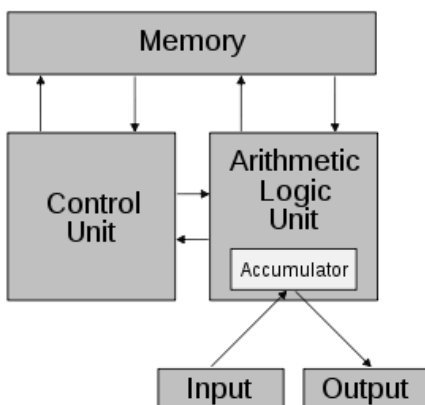
10	1	프로세스	프로세스, 스레드	운영체제의 프로세스에 대하여 다음을 설명하시오. 가. 프로세스 상태 다이어그램 나. 프로세스 제어 블록(PCB) 다. 프로세스 생성 및 종료	모의 _2019.07	관리	3
11	1	인터럽트	인터럽트	인터럽트(Interrupt)에 대하여 다음사항을 설명하시오. 가. 인터럽트 발생 요인(하드웨어 및 소프트웨어) 나. 마스크블(Maskable) 인터럽트와 언마스크블(Non-maskable) 인터럽트 다. 벡터(Vectored) 인터럽트의 동작원리 라. 우선순위(Priority)를 보장하는 다중(Multiple) 인터럽트가 발생한 경우 이를 처리하기 위한 ISR(Interrupt Service Routine)을 프로세서 및 소프트웨어에서 제공하여야 하는 기능요소들과 상호 연관시켜 설명하시오.	120	컴시 응	2
12	1	문맥 교환	문맥 교환	운영체제에서 문맥(context)의 내용, 문맥교환(context switch)의 과정 및 문맥교환이 발생하는 조건을 설명하시오.	110	응용	4
13	1	파이프라인	파이프라인	CPU 파이프라인과 관련하여 아래 사항에 대하여 설명하시오. 가. CPU 명령어 처리의 파이프라인 방식과 비파이프라인 방식의 CPU 처리 시간 비교 (파이프라인 단계 수를 m, 각 파이프라인 단계에서의 처리시간을 k, 실행할 명령어들의 수를 n이라 함) 나. 명령어 파이프라이닝 해저드 3가지 유형과 해결방안	모의 _2020.06	응용	4
14	1	메모리 인터리빙	메모리 인터리빙	메모리 인터리빙의 유형과 액세스 활용방식에 대해 설명하시오	모의 _2018.06	관리	3
15	1	아키텍처	아키텍처	폰 노이만 아키텍처와 하버드 아키텍처에 대해 설명하고 폰 노이만 아키텍처의 문제점 및 해결방안에 대해 설명하시오.	모의 _2018.10	관리	3
16	1	GPU, CPU	GPU, CPU 차이점	인공지능,빅데이터 등 4차 산업혁명으로 대변되는 새로운 기술의 발전이 대용량의 데이터를 효율적으로 처리하는 GPU의 성장을 견인하고 있고 최근 국내에서 금융산업에서도 GPU기반의 아키텍처 도입되고 있다. 다음 아래의 질문에 대하여 설명하시오. 가. GPU와 GPGPU 개념 나. GPU와 CPU의 차이점	모의 _2018.05	응용	3
17	1	DMA	DMA	DMA(Direct Memory Access) 입출력 제어 방식 중 아래 사항에 대하여 설명하시오. 가. DMA 제어기 구조와 버스 연결 방식 나. DMA 동작 모드와 처리 절차	모의 _2020.06	응용	3
18	1	저장장치	RAID	RAID(Redundant Array of Independent/Inexpensive Disks) Level 중에 Raid 0, Raid 5, Raid 6, Raid 1+0, Raid 5+0에 대해 최소 Disk 필요 수량, 공간 효율, 고장 허용 범위를 포함해 설명하시오.	모의 _2017.11	응용	3
	2		스토리지유형	블록스토리지,파일스토리지,오브젝트스토리지의 데이터 접근방식	132	관리	1
19	1	FTS	FTS	FTS(Fault Tolerant System)에 대해 설명하시오	모의 _2019.12	응용	1
20	1	주기억장치	주기억장치	DRAM(Dynamic RAM)과 SRAM(Static RAM)	125	응용	1

21	1	우선순위역전	우선순위역전	Real Time Scheduling이 갖는 문제 중 우선순위 역전(Priority Inversion)이 있다. Task 1, Task 2, Task 3 순으로 우선순위가 낮다고 할 때 우선순위 역전을 사례기반으로 설명하고, 우선순위 역전을 해결하기 위한 2가지 기법에 대하여 설명하시오.(단 P, V연산을 사용한다)	126	관리	4
22	1	클라우드 컴퓨팅	가상화 기술	최근 비즈니스환경은 고객을 중심으로 빠르게 변화하고 있다. DevOps 환경에 대해 다음을 설명하시오. 가. Hypervisor 의 유형 나. Hypervisor 와 Docker 비교 다. Docker 컨테이너 배포절차	모의 _2017.01	관리	3
	2		오토 스케일	최근 부하 분산을 위하여 클라우드 서비스에서 Auto Scale 기능을 많이 활용한다. 다음에 대하여 설명하시오. 가. Auto Scale Up 과 Auto Scale Out 아키텍처 비교 나. 스케줄 기반의 Auto Scale 과 부하 기반의 Auto Scale 기능 비교	121	관리	2
	3		컴퓨팅 기술	엣지(Edge) 컴퓨팅과 클라우드 컴퓨팅의 차이점	139	관리	1

1	Stored-program computer 의 개념
제목	컴퓨터시스템의 다섯 가지 구성 요소를 제시하고, 폰 노이만 컴퓨터(Von Neumann Computer)의 기초가 되는 Stored-program computer 의 개념에 대하여 설명하시오.
도메인	컴퓨터구조/운영체제
정의	프로그램 인스트럭션을 전기적인 장치인 메모리에 저장하는 컴퓨터
키워드	5 가지 구성 요소(제어장치, 연산장치, 메모리, 입력장치, 출력장치), Reprogrammable, Fetch, IR, PC
출제의도분석	컴퓨터 구조의 기초가 되는 구성 요소 및 지식을 숙지하고 있는지를 확인.
답안작성전략	- 컴퓨터시스템의 다섯 가지 구성 요소를 명확히 제시하고 각 구성 요소의 역할을 자세하게 설명함. 이 때 각 구성 요소의 주요 변천사를 보여주어 기술사로서 전산학에 조예가 깊음을 어필함. - Stored-program computer 의 명확한 정의 및 개념 제시. 특히, 나오게 된 배경과 장점에 초점을 맞추어서 설명함.
참고문헌	위키피디아 http://cs.sungshin.ac.kr/~jwoo/ocu/ca/20101.html#1 http://www.sci.brooklyn.cuny.edu/~philaris/core51lec/leccpu.pdf http://www.dauniv.ac.in/downloads/CArch_PPTs/CompArchCh02L01vonNeumann.pdf
모범목차	1. 컴퓨터시스템의 다섯 가지 구성 요소 가) 컴퓨터시스템 다섯 가지 구성 요소의 개요 나) 구성 요소 별 주요 변천사 2. Stored-program computer 개념 가) Stored-program computer 의 정의 나) Stored-program computer 의 등장 배경 다) Stored-program computer 의 원리 및 특징
문제풀이	홍석환 PE (제 99 회 컴퓨터시스템응용/ madusin@naver.com)

■ 컴퓨터시스템 다섯 가지 구성 요소의 개요

- 컴퓨터 시스템은 아래와 같이 (1) Memory (2) Control Unit (3) ALU (4) Input (5) Output 으로 구성됨.



(출처: <http://ageofcomputer.blogspot.kr/2009/05/stored-program-architecture.html>)

구성요소	설명
Control Unit	명령어를 해독하여 ALU, Memory, Input, Output 장치에게 해야 할 일을 지시하는 장치
ALU	메모리로부터 읽어온 Instruction, data 정보를 바탕으로 읽어온 정보를 처리(예를 들어 사칙 연산)하는 장치
Memory	프로그램과 데이터를 저장하기 위한 장치.
Input	컴퓨터 외부(예: 사람)에서 입력을 받아 전달하는 장치 (키보드, 마우스 등)
Output	컴퓨터에서 연산한 결과를 외부에 출력하는 장치 (모니터, 프린터, 파일 출력 등)

■ 구성 요소 별 주요 변천사

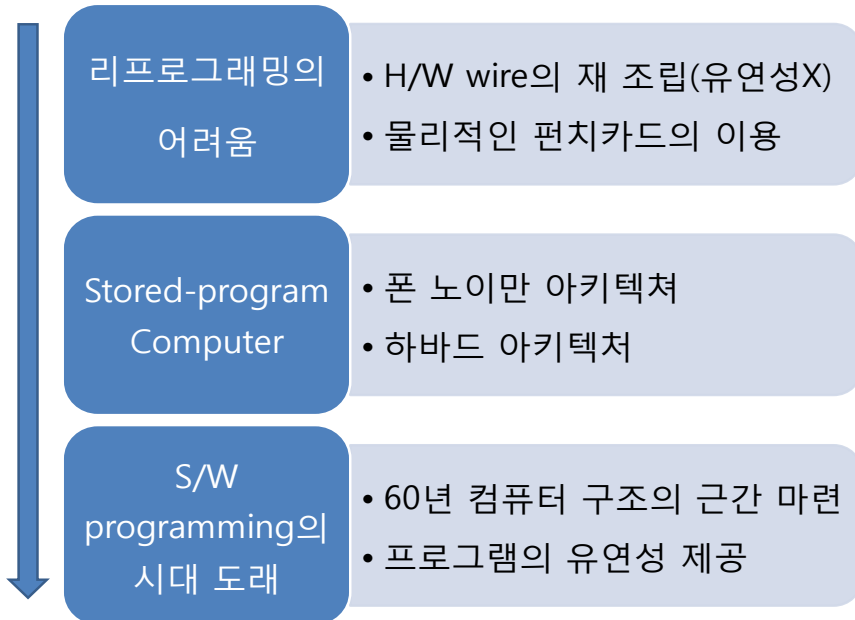
- 폰 노이만 아키텍처가 나온 이후 60 년이 지나는 동안 5 가지 구성 요소는 성능 향상과 자원의 효율적 사용을 목표로 끊임 없이 발전해 오고 있음.

구성요소	설명
CPU(Control Unit + ALU)	- 이용률 향상을 위하여 '파이프라인' 개념이 등장(이후 슈퍼스칼라, 슈퍼파이프라인, VLIW 등의 시도를 함) - 파이프라인 성능 향상을 위해 CISC → RISC(Instruction 별 연산 시간을 맞춤) - 멀티스레딩 기술을 이용하여 CPU 이용률을 향상시킴(Inter-lived MT, Block MT, 하이퍼스레딩) - 연산 속도 향상을 위해 초기에는 core 의 Clock 수 향상 시도 → Multi-core 로 진화(발열 및 Clock 수 증가의 한계 해결)
Memory	- Locality(지역성)을 적극적으로 활용하기 위해 캐시를 도입함. - 메모리 인터리빙을 사용하여 메모리 접근을 병렬화함. - 더블 펌핑을 이용하여 DDR 에서 2 word/cycle 를 달성함.
Input	- 키보드와 마우스 이후 오랜 기간 큰 발전이 없었음. - 최근 음성, 모션 등을 입력으로 받는 장치로 진화 중.
Output	- 스펙링을 이용하여 프린트 - 흑백 → 컬러 → 3D 의 출력을 위한 렌더링 기술과 GPU 로 진화 중.

■ Stored-program computer 의 정의

- 프로그램 연산(program instructions)이 전자적인 장치인 메모리에 존재하는 컴퓨터.

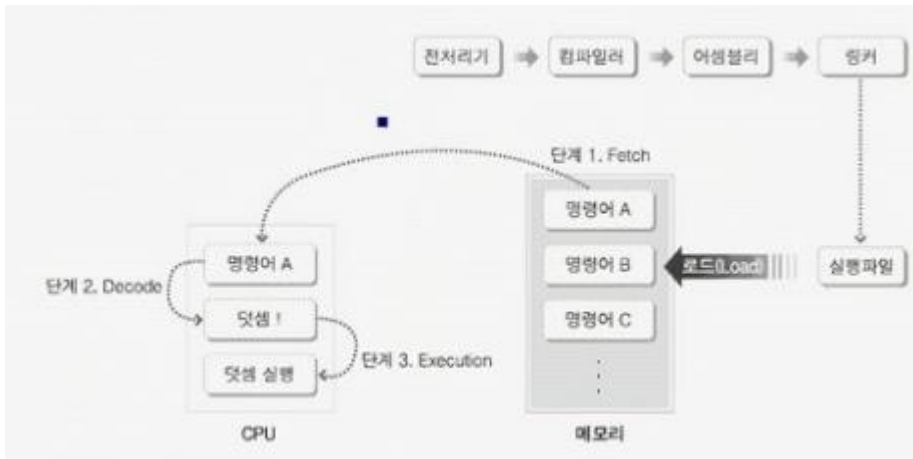
■ Stored-program computer 의 등장배경



- 1930년대까지는 리프로그래밍을 위해서는 H/W 적인 wire 를 재조립하거나 펀치카드를 주로 이용하였음. Wire 재조립은 프로그래밍이 어렵고 버그가 많이 발생하는 불편함이 있었고 펀치카드는 물리적인 손상이나 감긴 정도에 따른 속도차이 등의 물리적인 불편함이 있었음.
- 1936년 universal Turing machine 에 영감을 얻은 폰 노이만이 리프로그래밍(re-programmable)이 쉬운 아키텍처를 제안하면서 등장함.
- Stored-program computer 의 등장에 따라 진정한 의미의 S/W 를 이용한 프로그래밍이 용이해졌고 유연하고 편리하게 프로그래밍을 할 수 있게 됨에 따라 IT 의 급성장의 근간을 마련함.

■ Stored-program computer 의 원리

- Stored-program computer 는 아래 그림과 같이 (1) 실행파일을 준비하고 (2) 실행한 뒤 (3) Fetch (4) Decode (5) Execute 를 수행.



(출처: <http://artisticbit.tistory.com/>)

- 동작 원리 상세 설명

절차	설명
(1) 사전 준비(실행 파일 준비)	- 프로그래머가 사전에 프로그램을 정해진 언어(C, C++, JAVA 등)로 작성. - 컴파일러가 기계가 알아들을 수 있는 바이너리 파일로 컴파일하여 저장장치(디스크 등)에 저장하여 준비해 놓음
(2) 실행	- OS 의 실행 파일 로더를 이용해서 실행파일(바이너리 파일)을 메모리에 읽음(로드) 이 때, 명령어와 데이터를 모두 메인 메모리에 올려 놓음. 향후 CPU 는 메인 메모리에 있는 명령어와 데이터를 이용해서 작동함.
(3) Fetch	- CPU 내의 CU(Control Unit)가 PC 값을 이용해서 메인 메모리의 명령어를 읽어옴(Fetch)
(4) Decode	- 읽어온 명령어를 해독한 뒤 PU(Process Unit, ALU)에게 명령을 지시
(5) Execution	- ALU 는 명령어에 맞게 연산을 수행함. 이 때, ACC, FPR 등의 레지스터와 GPRs(General Purpose Registers)를 이용하여 연산을 수행함.
(6) Write Back	- 연산 결과를 저장해야 하는 경우라면 레지스트리에 저장된 결과를 메인 메모리에 다시 쓰는(Write)과정을 수행함.

- Stored-program computer 는 명령어를 메인 메모리에서 관리하게 되면서 자연스럽게 이러한 동작 원리를 통해 연산을 수행하게 됨.

■ Stored-program computer 의 특징

(1)연산의 유연성	(2) 프로그램의 독립성	(3)CPU-memory 의 병목
- Instruction 이 전기적인 신호에 의해 변경이 가능한 메모리에 있으므로 읽기 쓰기가 용이하고 언제든지 프로그램의 수정이 가능함. - H/W 와 프로그램의 강력한 결합을 풀어 H/W 독립적으로 프로그램을 설계, 구현, 서비스할 수 있도록 독립성을 제공함. - S/W 적인 유연성을 제공하면서 필연적으로 CPU 와 메모리 사이에 버스가 필요해졌고 버스의 병목현상 및 성능 문제가 발생하게 됨. 향후 60 년간 이 효율을 높이기 위한 연구가 이어짐.		

"끝"

9	CPU의 주소지정방식
문제	CPU의 주소지정방식(Addressing Mode)에 대해 설명하시오.
도메인	컴퓨터구조/운영체제(OS/Operating System)
출제배경/의도	연산에 사용될 데이터가 기억장치의 어디에 위치하는지를 지정하는 주소지정방식에 대한 이해 점검치 메모리 접근회수에 따른 단계별 주소지정의 용어에 대한 대응 점검
키워드	- 주소지정방식 단계 : 0 단계, 1 단계, 2 단계, - 유형 : 즉시주소, 묵시주소, 직접주소, 레지스터 직접주소, 레지스터 간접주소, 변위 주소,
목차예시	1. 주소지정방식의 개념 2. 주소지정방식의 유형 가. 주소 지정방식의 단계별 분류 나. 주소 지정방식의 유형
채점 점수 가이드	① 개념, 이해 부족 (1~3점) ② 기본 이해 수준 (3~5점) ③ 정확한 기술 제시 (5~6점) ④ 추가요소 설명(+α)
난이도	★ ★ ☆ ☆ ☆
학습 가이드	주소지정 단계에 대한 개념도 설명을 정확히 제시 각 단계별 주소지정 방식 도식화 기술
참고문헌	컴퓨터 구조와 원리 2.0(한빛 미디어) 컴퓨터 아키텍처(한빛 미디어)
출제자	황윤준 기술사(제 116 회 컴퓨터시스템응용기술사 / yoonjune@naver.com)

1. 주소지정방식의 개념

가. 주소 지정방식의 개념

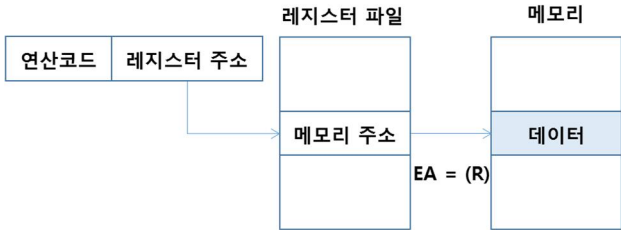
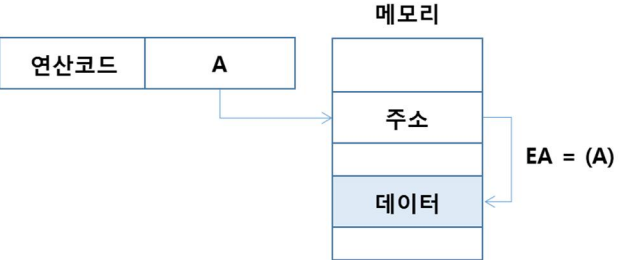
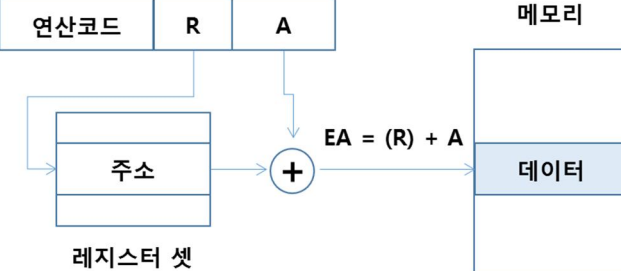
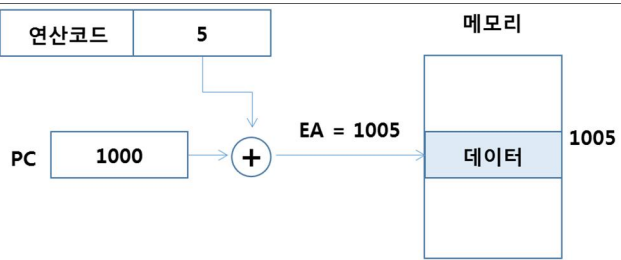
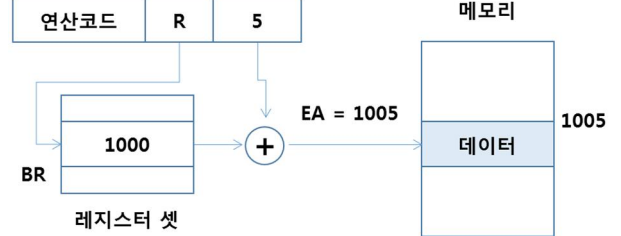
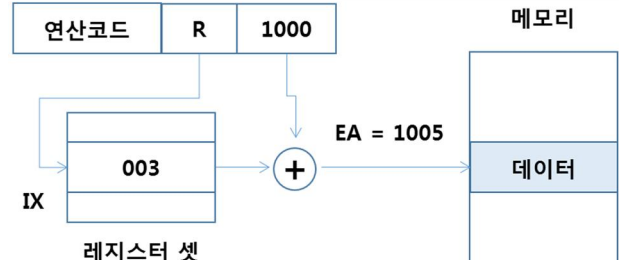
- 프로그램 실행 시 오퍼랜드를 지정하는 방식으로, 연산에 사용될 명령어의 데이터를 기억장치에 지정하는 방식
- 명령어의 일부인 피연산자 필드를 사용하여 데이터가 실제 위치한 유효주소를 결정하는 형식

나. 주소 지정방식의 단계

단계	설명	종류
0 단계 주소지정	- 데이터가 있는 위치를 파악하기 위해 특별한 과정이 필요 없는 방식	- 묵시 주소지정 - 즉시 주소지정(즉치)
1 단계 주소지정	- 데이터의 유효주소를 확정하기 위해 주소계산 및 읽기 연산 과정을 한번 수행하는 방식	- 직접 주소지정 - 레지스터 직접주소 지정
2 단계 주소지정	- 데이터의 유효주소를 확정하기 위해 주소 계산이나 읽기 연산과정을 2번 수행	- 레지스터 간접주소지정 - 간접주소/변위주소 지정 (색인주소, 베이스주소) - PC상대 주소지정

2. 주소지정방식의 유형

유형	개념도	설명
묵시 주소지정	명령어 누산기or정해진 장소 데이터	- 명령어 실행에 필요한 데이터의 위치를 별도 지정하지 않고, 묵시적으로 위치가 정해져 있는 경우 - 명령어의 길이가 짧고, 명령어의 종류 제한 - ex) SHL, PUSH, POP
즉시 주소지정(즉치)	연산코드 Operand(데이터)	- 연산에 사용될 실제 데이터를 오퍼랜드 필드에 직접 집어 넣는 방식
직접 주소지정	연산코드 A EA = A 메모리 데이터	- 간단한 형태의 주소지정방식 - 장점 : 한번의 기억장치 참조가 필요 - 단점 : 융통성이 부족, 제한된 주소 공간 - EA = A
레지스터 직접주소 지정	연산코드 레지스터 주소 EA = R 레지스터 파일 데이터	- 연산에 사용될 데이터를 레지스터에 저장하는 방식 - 데이터 저장 공간이 CPU 내부 레지스터들로 제한 - EA = R

레지스터 간 접주소지정	 레지스터 파일 메모리 연산코드 레지스터 주소 메모리 주소 데이터 EA = (R)	- 오퍼랜드 필드에 레지스터 번호를 가리키고, 그 레지스터에 저장된 내용이 유효주소 - $EA = (R)$
메모리 간접 주소 지정	 메모리 연산코드 A 주소 데이터 EA = (A)	- 오퍼랜드 필드에 실제 데이터의 주소가 위치한 메모리 상의 주소를 지정 - 두번의 메모리 조작필요 - 필요시 여러단계 가능 - $EA = (A)$
변위주소 지 정	 메모리 연산코드 R A 주소 데이터 레지스터 셋 EA = (R) + A	- 오퍼랜드 필드에 레지스터 번호 피르와 변위값 필드가 존재하고 이 두 필드의 조합으로 유효 주소를 생성 - $EA = (R) + A$
PC상대 주소 지정	 메모리 연산코드 5 PC 1000 데이터 1005 EA = 1005	- PC를 사용하는 변위주소 지정방식 - 주로 분기 명령어에서 사용 - PC(프로그램카운터)는 묵시적 지정 가능 - $EA = (PC) + A$
베이스레지 스터 주소지 정	 메모리 연산코드 R 5 BR 1000 데이터 1005 EA = 1005 레지스터 셋	- 베이스 레지스터에 기준이 되는 주소를 저장 - 기억장치 내에서 프로그램의 위치를 지정하는데 주로 사용 - $EA = (BR) + A$
인덱스 주소 지정	 메모리 연산코드 R 1000 IX 003 데이터 EA = 1005 레지스터 셋	- 인덱스 값을 저장하는 특수 레지스터(IX)를 이용, 명령어 실행시 자동으로 값이 증가 - 주로 배열 조작에서 사용 - $EA = A + (IX)$

※. 명령어 형식에 의한 컴퓨터 구조

1) 누산기의 구조

레지스터로 누산기(AC)를 사용한다. 반드시 AC와 연산을 하고 AC에 저장을 한다.

2) 범용 레지스터 구조

CPU 내의 2개 이상의 레지스터를 사용한다. 2주소지정방식, 3주소지정방식을 사용한다.

3) 스택의 구조

스택의 구조는 LIFO 방식을 사용한다. 0주소지정방식이고 PUSH, POP 명령어가 있다. 구조는 다음과 같다.

PUSH X

$SP \leftarrow SP - 1$

$M[SP] \leftarrow MBR$

POP X

$MBR \leftarrow M[SP]$

$SP \leftarrow SP + 1$

[참고자료]

- 인덱스 주소지정방식 : 인덱스 값을 저장하는 특수 레지스터
- 베이스 레지스터 지정 방식 : 베이스 레지스터의 내용 + 변위
- 상대 주소 지정방식 : 현재 명령어의 오퍼랜드 내용 + 프로그램 카운터의 내용
- 자동주소 지정 : 자동감소 레지스터의 출력이 유효주소
- 자동증가 주소지정 : 자동증가 레지스터의 출력이 유효주소
- 레지스터 간접 주소지정 : 레지스터가 실제 오퍼랜드가 저장된 기억장치의 주소값 저장

- 연산자

r : 명령어 내의 피연산자 필드가 명시하는 레지스터 주소

a : 명령어 내의 피연산자 필드가 명시하는 메모리 주소

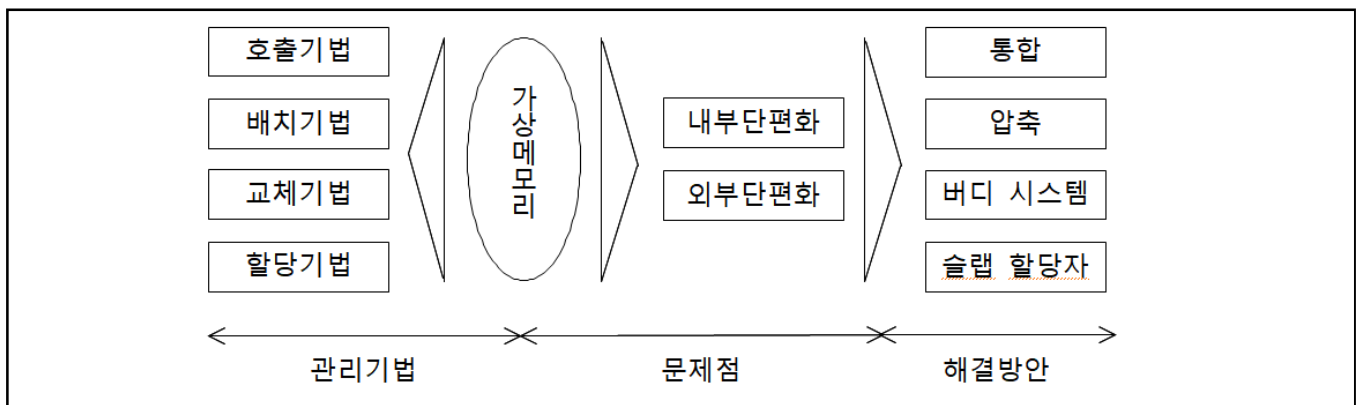
ea : 참조하는 데이터를 포함하는 장소의 실제 주소(유효 주소)

Reg(x) : 레지스터 x의 내용

M(x) : 메모리 x번지의 내용

7	가상메모리 (Virtual Memory)		
문제	가상메모리에 대하여 다음을 설명하시오. 가. 가상메모리 관리기법 나. 가상메모리 단편화와 해결방안		
도메인	CA/OS	난이도	★ ★ ☆ ☆ ☆ (별 5 개 기준)
출제배경	기출 빈도가 높은 가상메모리의 전반적인 지식 및 이해도 확인		
핵심 키워드	호출기법, 배치기법, 할당기법, 교체기법, 내부단편화, 외부단편화, 통합(coalescing), 압축(compaction), 메모리 풀(memory pool), 버디 메모리 시스템 (Buddy Memory system), 슬랩 할당자(Slab Allocator)		
참고문헌	컴퓨터 구조 및 설계 (한티미디어) KPC 기술사 월간모의고사 풀이집		
출제자	김준삼 기술사(제 118 회 정보관리기술사 / apolon32@naver.com)		

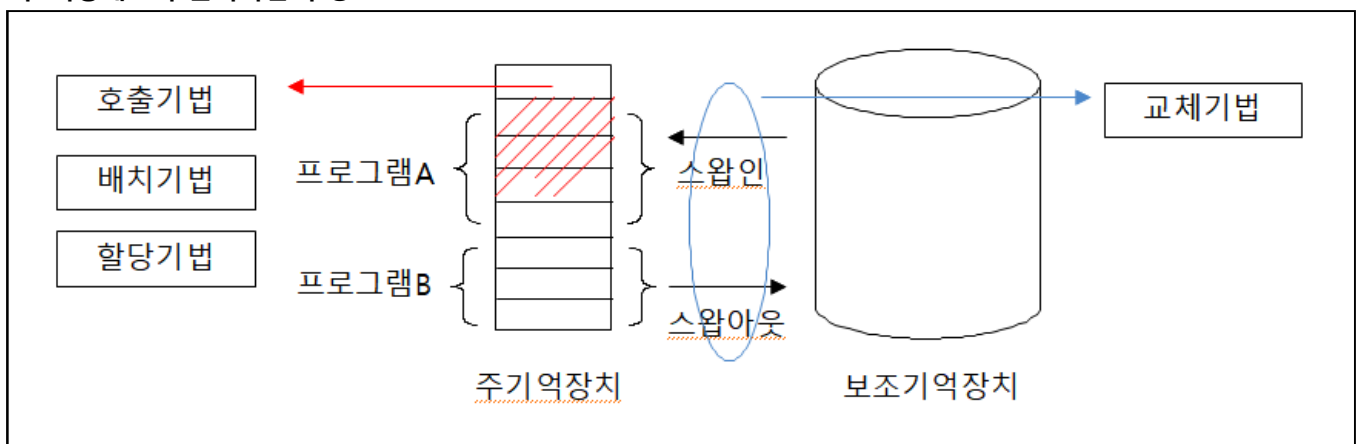
1. 메모리 한계 확장, 가상 메모리의 개요



- (개념) 물리적 주기억장치의 한계를 극복하기 위해 프로그램의 일부를 보조기억장치에 저장하고 필요 시에 호출/배치/할당/교체하는 메모리 관리 기술.

2. 가상메모리 관리기법

가. 가상메모리 관리기법 구성도



- 가상메모리 효율적 관리를 위해 호출/배치/교체/할당 기법을 사용하여 관리.

나. 가상메모리 관리기법의 상세 설명

관리기법	주요기법	설명
호출기법 (Fetch)	요구반입(demand paging) 선반입 (pre paging)	페이지의 주기억장치로 적재 시점을 결정하는 기법
배치기법 (Placement)	Best, First, Worst Fit	블록의 주기억장치의 적재 위치를 결정하는 기법
교체기법 (Replacement)	FIFO, LFU, LRU, NUR	주기억장치의 가용공간이 없는 경우 교체할 페이지를 결정하는 기법
할당기법 (Allocation)	페이징, 세그멘테이션, 페이지드 세그멘테이션	주기억장치의 할당 크기를 결정하는 기법

- 페이징 기법 사용 시에는 내부 단편화 발생, 세그멘테이션 기법을 사용 시에는 외부 단편화 발생.

3. 가상메모리 단편화

가. 가상메모리 단편화 개요

항목	설명
개념도	작업1(25K), 작업2(20K), 작업3(18K), 작업4(32K) 초기상태 작업1,2,3,4 할당된 상태 * 외부단편화 * 내부단편화
정의	주기억장치 내 저장공간이 여러 작은공간으로 나누어져 사용 가능한 공간이 존재하지만 사용이 불가능한 상태

- 페이징 기법을 사용하는 경우 내부단편화 발생, 세그멘테이션 기법을 사용하는 경우 외부단편화가 발생.

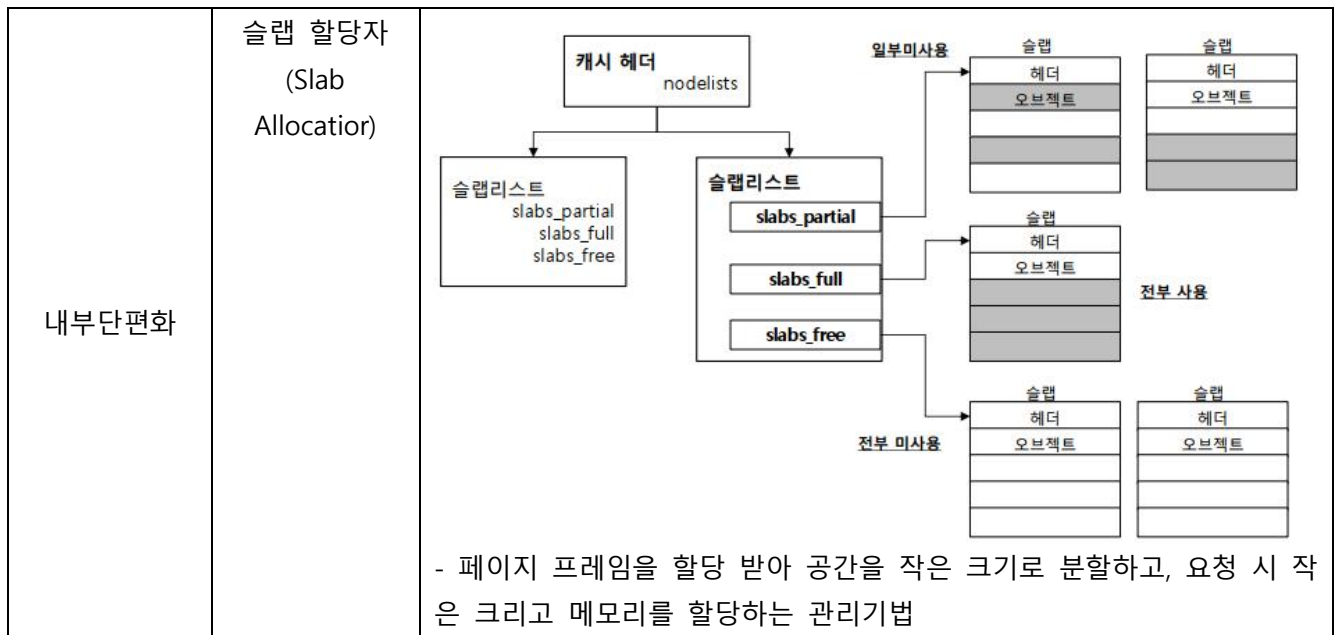
나. 가상메모리 단편화 종류

종류	항목	설명
내부단편화	개념	가상메모리 할당 시 프로세스가 필요한 크기보다 큰 메모리가 할당되어 메모리 공간이 낭비되는 상태
	발생원인	분할된 영역 > 프로그램 크기
외부단편화	개념	할당 가능한 가상메모리의 위치가 불연속적이기 때문에 사용 가능한 공간이 있는 경우에도 낭비되는 상태
	발생원인	분할된 영역 < 프로그램크기, 분할된 영역의 불연속성

- 메모리 효율성/가용성 향상을 위해 메모리 단편화를 최소화할 수 있는 다양한 해결방안을 적용.

4. 가상메모리 단편화 해결방안

항목	해결방안	설명
내/외부 단편화 공통	통합 (coalescing)	- 주기억장치 내의 인접한 단편화 된 공간을 통합하는 기법
	압축 (compaction)	- 주기억장치 내의 빈 분할 영역을 한 곳에 통합하는 기법
	메모리 풀 (memory pool)	- 주기억장치내 다양한 크기의 메모리 풀을 미리 생성한 후 사용하고 반납하는 기법
외부단편화	버디 메모리 시스템 (Buddy Memory system)	- 요청한 프로그램 크기에 알맞은 크기를 할당하기 위해 2의 거듭제곱 크기로 분할하여 할당하는 기법



- Linux 운영체제에서는 임베디드 시스템을 위한 Slob, 대용량 서버 환경을 위한 Slub 할당자도 사용 가능. 끝.

문 제	8. 총 3 개의 페이지로 구성된 페이지 기반 가상 메모리 시스템이 있다. 제시되는 페이지 교체 알고리즘의 페이지 교체과정을 적중률(Hit Ratio)과 함께 설명하시오.		
	- 페이지 교체 알고리즘 : FIFO, LRU, LFU - 페이지 번호 참조 순서 : 4, 3, 2, 5, 2, 1, 3, 2, 6, 3, 4 (단, 최초 페이지 프레임은 모두 비어있다고 가정한다.)		
출 제 영 역	CA/OS	난 이 도	★★☆☆☆ (별 5 개 기준)
출 제 배 경	페이지 교체 알고리즘 동작 이해, 계산과정에 대한 정확한 기술		
출 제 빈 도	페이지 교체 알고리즘과 계산과정 기술		
참 고 자 료	- 쉽게 배우는 운영체제, 컴퓨터 구조와 원리 2.0		
Key word	FIFO/LRU/LFU, Hit Ratio,		
풀이 기술사님	전지웅 (116 회 컴퓨터시스템응용기술사 / ahddl2004@naver.com)		

1. 페이지 교체 알고리즘

FIFO (First In First Out)	LRU (Least Recently Used)	LFU (Least Frequently Used)
•선입선출 •구현 용이, 비효율	•사용시간 •지역성활용, 오버헤드 발생가능	•사용횟수 •지역성활용, 오버헤드 발생가능

- 페이지 기반 가상메모리 시스템에서 원하는 데이터를 참조할 수 없을 경우 어느 페이지의 데이터를 교체할 것인지 결정하기 위하여 교체 알고리즘 활용.

2. FIFO 페이지 교체 과정

구분	설명											
개념	페이지에 적재된 순서대로 가장 먼저 적재된 블록이 삭제되고 새로운 블록이 적재되는 방식											
특징	구현이 용이하지만 단순히 가장 오래된 블록을 선택하여 교체하는 방식으로 효율성을 보장하지 못함.											
페이지 교체 과정	페이지 #	참조순서 (FIFO)										
		4	3	2	5	2	1	3	2	6	3	4
	P1	4	4	4	5	5	5	5	2	2	2	2
	P2		3	3	3	3	1	1	1	6	6	6
	P3			2	2	2	2	3	3	3	3	4
	Hit 여부 (페이지 부재)	x	x	x	x	O	x	x	x	x	O	x
Hit Ratio		Hit Ratio = 2/11										

페이지 적중률 (Hit Ratio)	페이지 부재는 9 회 , 페이지 적중은 2 회 페이지 적중률 = 2/11
---------------------------	---

- FIFO 는 가장 먼저 적재된 블록을 가장 먼저 교체하는 알고리즘임.

3. LRU(Least Recently Used) 페이지 교체 과정

구분	설명																																																																																			
개념	적재되어 있는 페이지 가운데 가장 오랫동안 참조되지 않은 페이지를 교체하는 방식.																																																																																			
특징	- 시간적으로 오랫동안 사용되지 않는 블록을 교체 - 시간기록으로 인한 오버헤드 발생 가능																																																																																			
페이지 교체 과정	<table><tr><th rowspan="2">페이지 #</th><th colspan="11">참조순서 (LRU)</th></tr><tr><th>4</th><th>3</th><th>2</th><th>5</th><th>2</th><th>1</th><th>3</th><th>2</th><th>6</th><th>3</th><th>4</th></tr><tr><td>P1</td><td>4</td><td>4</td><td>4</td><td>5</td><td>5</td><td>5</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td></tr><tr><td>P2</td><td></td><td>3</td><td>3</td><td>3</td><td>3</td><td>1</td><td>1</td><td>1</td><td>6</td><td>6</td><td>6</td></tr><tr><td>P3</td><td></td><td></td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>4</td></tr><tr><td>Hit 여부 (페이지 부재)</td><td>x</td><td>x</td><td>x</td><td>x</td><td>0</td><td>x</td><td>x</td><td>0</td><td>x</td><td>0</td><td>x</td></tr><tr><td>Hit Ratio</td><td colspan="11">Hit Ratio = 3/11</td></tr></table>	페이지 #	참조순서 (LRU)											4	3	2	5	2	1	3	2	6	3	4	P1	4	4	4	5	5	5	3	3	3	3	3	P2		3	3	3	3	1	1	1	6	6	6	P3			2	2	2	2	2	2	2	2	4	Hit 여부 (페이지 부재)	x	x	x	x	0	x	x	0	x	0	x	Hit Ratio	Hit Ratio = 3/11										
	페이지 #		참조순서 (LRU)																																																																																	
		4	3	2	5	2	1	3	2	6	3	4																																																																								
	P1	4	4	4	5	5	5	3	3	3	3	3																																																																								
	P2		3	3	3	3	1	1	1	6	6	6																																																																								
	P3			2	2	2	2	2	2	2	2	4																																																																								
	Hit 여부 (페이지 부재)	x	x	x	x	0	x	x	0	x	0	x																																																																								
Hit Ratio	Hit Ratio = 3/11																																																																																			
페이지 적중률 (Hit Ratio)	페이지 부재는 8 회 , 페이지 적중은 3 회 페이지 적중률 = 3/11																																																																																			

- LRU 는 가장 오랫동안 참조되지 않은 페이지를 교체하는 알고리즘임.

4. LFU(Least Frequently Used) 페이지 교체 과정

구분	설명																																																																																			
개념	적재되어 있는 페이지 가운데 가장 참조 횟수가 적은 페이지를 교체하는 방식.																																																																																			
특징	- 사용된 횟수가 가장 작은 페이지를 교체 - 참조 횟수 기록으로 인한 오버헤드 발생 가능 - 가장 작은 참조 횟수의 페이지가 여러 개일 경우 LRU 에 따라 교체																																																																																			
페이지 교체 과정	<table><tr><th rowspan="2">페이지 #</th><th colspan="11">참조순서 (LFU)</th></tr><tr><th>4</th><th>3</th><th>2</th><th>5</th><th>2</th><th>1</th><th>3</th><th>2</th><th>6</th><th>3</th><th>4</th></tr><tr><td>P1</td><td>4</td><td>4</td><td>4</td><td>5</td><td>5</td><td>5</td><td>3</td><td>3</td><td>3</td><td>3</td><td>3</td></tr><tr><td>P2</td><td></td><td>3</td><td>3</td><td>3</td><td>3</td><td>1</td><td>1</td><td>1</td><td>6</td><td>6</td><td>4</td></tr><tr><td>P3</td><td></td><td></td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td></tr><tr><td>Hit 여부 (페이지 부재)</td><td>x</td><td>x</td><td>x</td><td>x</td><td>0</td><td>x</td><td>x</td><td>0</td><td>x</td><td>0</td><td>x</td></tr><tr><td>Hit Ratio</td><td colspan="11">Hit Ratio = 3/11</td></tr></table>	페이지 #	참조순서 (LFU)											4	3	2	5	2	1	3	2	6	3	4	P1	4	4	4	5	5	5	3	3	3	3	3	P2		3	3	3	3	1	1	1	6	6	4	P3			2	2	2	2	2	2	2	2	2	Hit 여부 (페이지 부재)	x	x	x	x	0	x	x	0	x	0	x	Hit Ratio	Hit Ratio = 3/11										
	페이지 #		참조순서 (LFU)																																																																																	
		4	3	2	5	2	1	3	2	6	3	4																																																																								
	P1	4	4	4	5	5	5	3	3	3	3	3																																																																								
	P2		3	3	3	3	1	1	1	6	6	4																																																																								
	P3			2	2	2	2	2	2	2	2	2																																																																								
	Hit 여부 (페이지 부재)	x	x	x	x	0	x	x	0	x	0	x																																																																								
Hit Ratio	Hit Ratio = 3/11																																																																																			

페이지
적중률
(Hit Ratio)

페이지 부재는 8 회 , 페이지 적중은 3 회
페이지 적중률 = 3/11

- LFU 는 가장 참조 횟수가 적은 페이지를 교체하는 알고리즘임.

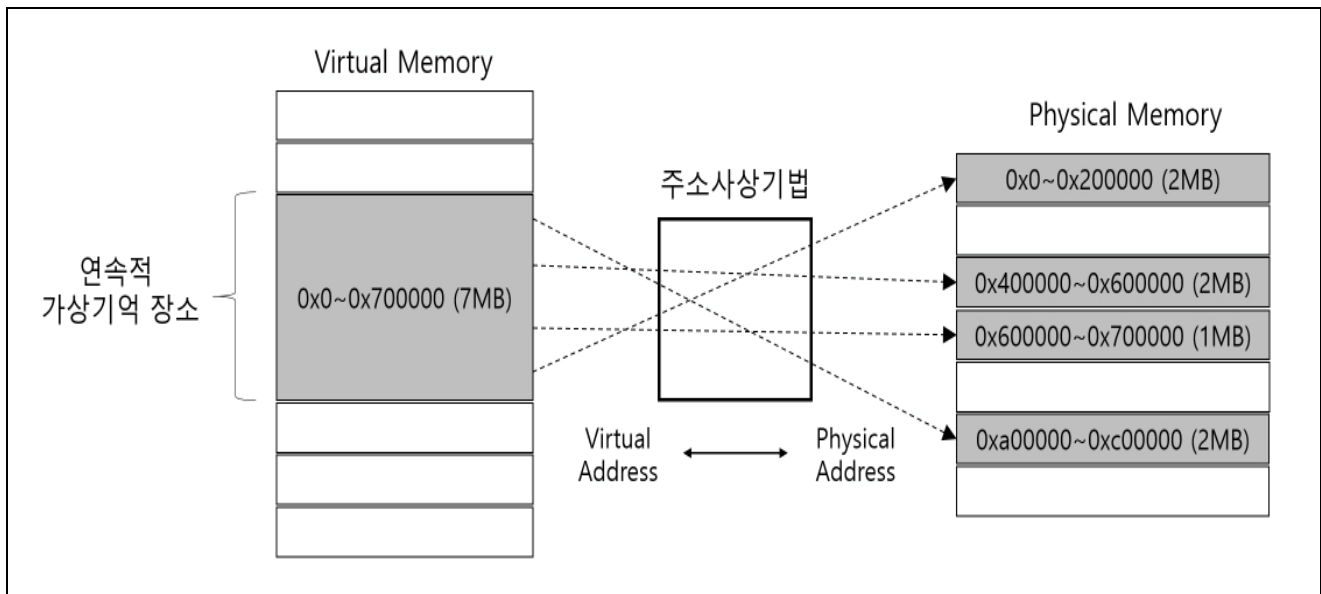
“끝”



종 목	문 제
컴퓨터시스템응용기술사	3교시 6번
페이지 방식을 이용하는 가상메모리에서 가상주소(Virtual address)를 주기억장치 주소인 물리주소(Physical Address)로 변환하는 과정을 페이지 테이블(Page Table) 및 TLB(Translation Lookaside Buffer)를 이용하여 설명하시오.	
도메인/토픽	컴퓨터구조 > 가상메모리
키워드	PMT(Page Map Table), 고속캐시(Associative memory), TLB Hit/Miss, Page fault, Locality
풀이 가이드	- 가상주소와 물리주소간의 변환, 가상메모리 관리 - 가상메모리 관리의 정의 - 주소사상기법의 종류 - 페이지 테이블을 이용한 직접사상 과정 및 원리 - 페이지 테이블을 이용한 직접사상 과정 - 직접사상 방식의 원리 - TLB를 이용한 간접사상 과정 및 원리 - TLB를 이용한 간접사상 과정 - 간접사상 방식의 원리 - 페이지테이블과 TLB 방식간 비교
필수항목 /연관토픽	가상메모리의 실제 주소 매핑 기법 (직접/연관/혼합사상) MMU (Memory Management Unit), 지역성(Locality)
참고문헌	MMU(Memory Management Unit), 블로그, (egloos.zum.com/recipes/v/5232056)
Advice	Fact를 묻는 문제이기 때문에, 문제에 집중한 목차 구성과 상세한 기술 필요하며, 가상메모리는 출제빈도가 높은 영역이므로 꾸준한 학습을 권장 드립니다.
작성자	108회 컴퓨터시스템응용기술사 / 김무형 (emusai@gmail.com)

1. 가상주소와 물리주소간의 변환, 가상메모리 관리

가. 가상메모리 관리의 정의



- 각 프로그램에 실제 메모리 주소가 아닌 논리 메모리 주소를 전달하여 메모리 확장 효과
- 하드 디스크의 큰 공간을 활용하여 주기억장치보다 더 큰 공간을 제공 가능

나. 주소사상기법의 유형

구분	내용
직접사상 (Direct Mapping)	- 주기억장치에서 PMT(Page Mapping Table)을 유지/관리 - 모든 페이지항목이 PMT에 존재 - 주소변환에 많은 시간이 소요
간접사상 (Associate Mapping)	- 고속캐시(Associative memory)에서 매핑테이블을 보관 - 속도가 빠른 장점이 있으나 고비용
직접/간접사상	- PMT를 주기억장치와 연관기억장치에 분산 저장 - 지역성(Locality)에 따라 재사용 가능성이 높은 최근 참조된 페이지는 AMP (Associate Mapping Table)에 보관하고 나머지는 PMT에 보관

- 페이지 방식을 이용하는 가상메모리에서 가상주소(Virtual address)를 주기억장치 주소인 물리주소(Physical Address)로의 변환을 위해 주소사상기법을 수행
- 주소사상기법은 크게 페이지 테이블을 이용한 직접 사상 방식과 TLB(Translation Lookaside Buffer)를 이용한 간접 사상 방식으로 구분 가능

2. 페이지 테이블을 이용한 직접사상 과정 및 원리

가. 페이지 테이블을 이용한 직접사상 과정

구분	상세내역
동작방식	① 가상주소 (Page#, Offset) ② Page# (n bits) → Page Table → Frame# ③ 물리주소 (Frame# (m bits), Offset) ④ Page Frame 프로그램 페이지징기법 주기억장치
세부절차	① 수행중인 프로세스는 논리적인 주소를 참조 ② 페이지 테이블에 저장되어 있는 페이지 프레임 검색 ③ 검색된 페이지 프레임과 변위 주소 값을 더해서 변환 ④ 변환된 물리적 주소 값을 통해 메인 메모리 접근

- 페이지테이블이 주기억장치에 위치하여 데이터 접근을 위해 주기억장치를 2번 접근해야 함
- 빈번한 데이터 쓰기로 인해 Paging의 성능 이슈 발생 가능

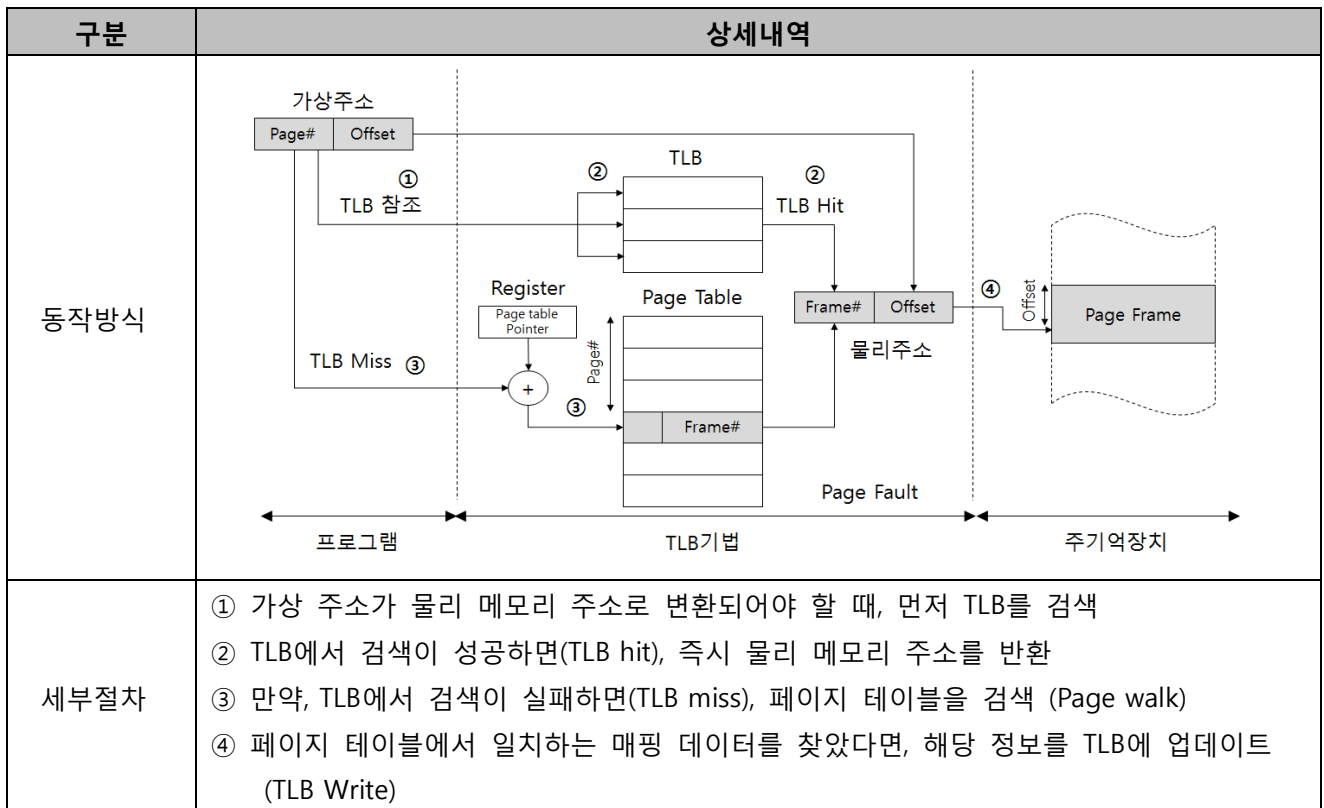
나. 직접사상 방식의 원리

구분	설명	
가상주소 단위	페이지번호 변위 p d 가상주소 $v=(p,d)$	- 페이지 가상 주소: $v = (p, d)$ - p: v가 속해있는 페이지 - d: 페이지 p의 시작위치로부터의 변위
Paging 구성요소	Page	- 일정한 크기의 고정 단위로 나뉘어진 단위
	Main memory	- 실제 물리적인 주소 공간을 지닌 Unit
	Page Frame	- 일정한 크기로 나뉘어진 메인 메모리 단위
	Page Table	- Page들을 리스트로 구성하고 있는 집합
	Page Fault	- Page Table에 Page가 존재하지 않을 경우 발생

- 가상 주소와 메인메모리의 물리 주소간 매핑을 통해 연결

3. TLB를 이용한 간접사상 과정 및 원리

가. TLB를 이용한 간접사상 과정



- 매번 주소 변환 수행하는 대신 변환 결과를 미리 테이블에 저장하여 사용
- 페이지 테이블 항목들을 특수 고속 캐시 사용으로 메모리 참조시간 단축

나. 간접사상 방식의 원리

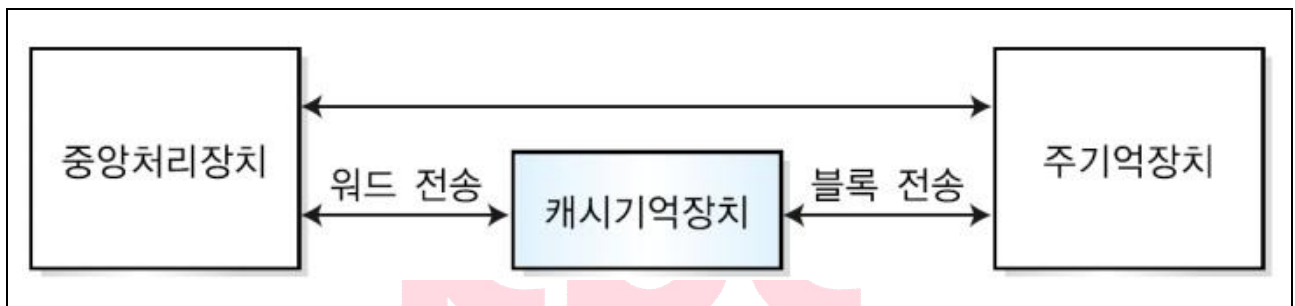
구분	상태 설명	동작절차
TLB Hit	- 가상주소가 TLB에 존재	- 물리주소 통해 Cache 검색 - Cache 에서 Hit 되면 Data 를 CPU 에게 전달
TLB Miss	- 가상주소가 TLB에 없음	- 가상주소로 페이지테이블 검색
TLB Write	- 페이지테이블 항목을 TLB에 갱신	- 페이지테이블에서 찾은 항목을 TLB에 갱신

4. 페이지테이블 방식과 TLB 방식간 비교

항목	페이지테이블	TLB
위치	- 주기억장치 내애 존재	- 고속의 특수 캐시 내에 존재
장단점	- 데이터 접근을 위해 주기억장치 두 번 접근하여 지연 발생	- TLB 모든 항목을 병렬로 고속 탐색 - 고비용으로 적용에 한계점 존재
성능향상방안	- Paging과 Segmentation 결합 - TLB를 통한 고속처리	- 페이지 크기를 증대 - 다중 페이지를 지원

문 제	7. 중앙처리장치(CPU)에서 명령어 블록을 인출하기 위한 캐시기억장치(Cache Memory)의 동작 순서를 설명하고, 캐시기억장치의 주소 사상(Address Mapping) 유형 별 동작 방식에 대하여 설명하시오.		
출 제 영 역	컴퓨터구조	난 이 도	★★★★☆☆
출 제 배 경	시스템 성능에 영향을 미치는 캐시기억장치의 동작 및 주소 사상 동작 방식 숙지 여부 확인		
출 제 빈 도	114 회 응용 4 교시		
참 고 자 료	- 한빛아카데미, 컴퓨터 구조와 원리 2.0 : 비주얼 컴퓨터 아키텍처 - KPC 기술사 IMPACT 실전모의고사 94 회 1 교시		
Key word	직접사상, 연관사상, 집합연관사상, 주기억장치 주소지정 방식, 모듈로 연산, 캐시 교체 알고리즘, 태그 필드, Slot 필드, 단어 필드, 집합 필드		
풀이 기술사님	이시광(119 회 컴퓨터시스템응용기술사 / sk2ckr@naver.com)		

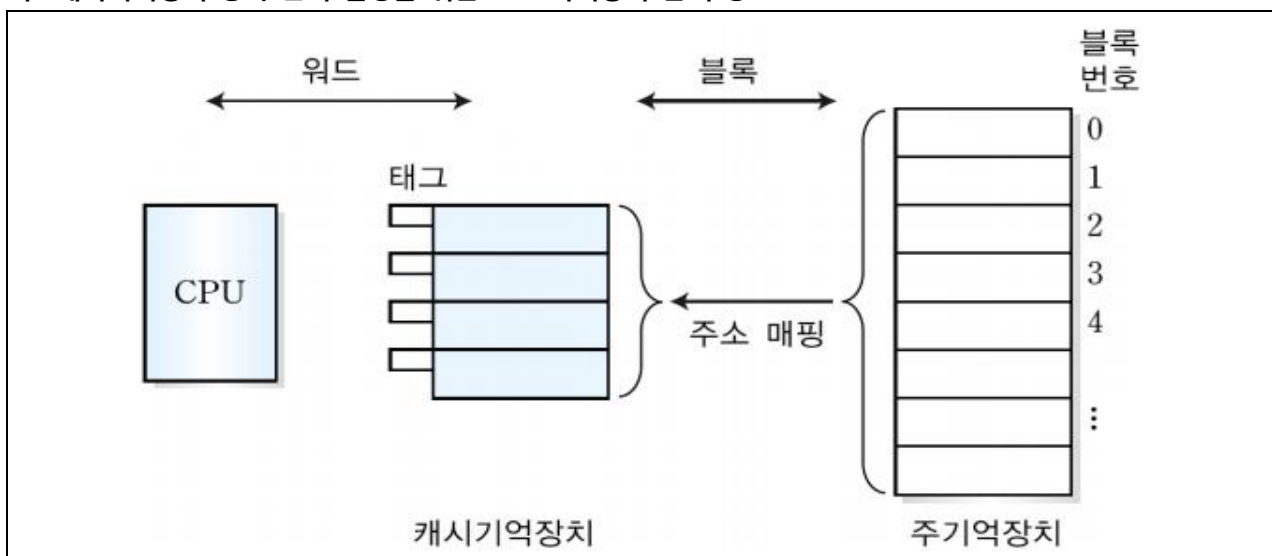
1. 캐시기억장치의 개요



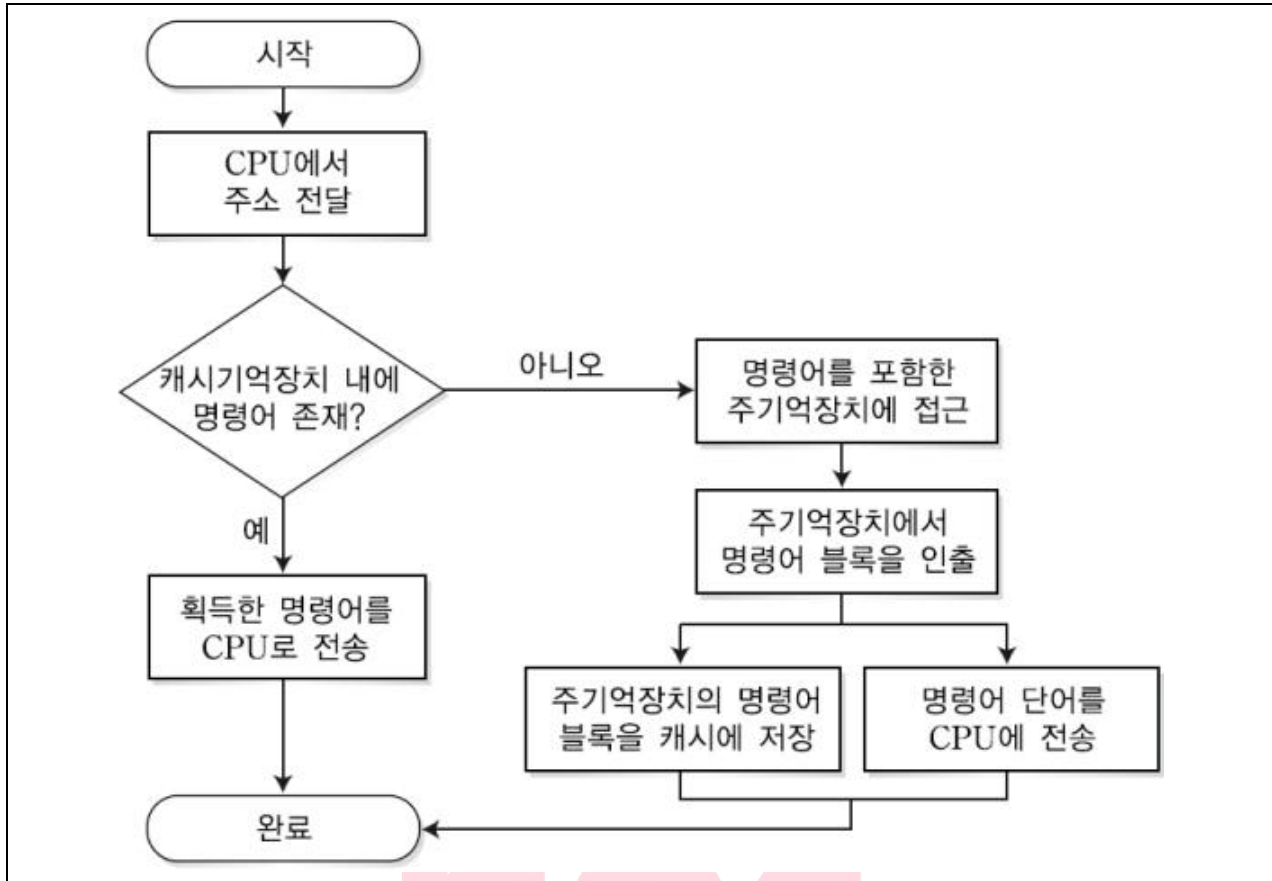
- CPU의 명령어와 데이터 접근 속도 향상을 위해 지역성의 원리 기반 주기억장치에 저장된 일부 명령어와 데이터를 임시로 복사해서 저장하는 기억장치
- 지역성의 원리(Locality) 기반 비용 효율적으로 CPU의 명령어/데이터 접근 속도 향상에 따른 시스템 성능 향상

2. 명령어 블록 인출 위한 캐시기억장치의 동작 순서

가. 캐시기억장치 동작 순서 설명을 위한 CPU-기억장치 간 구성도



나. 명령어 블록 인출 위한 캐시기억장치의 동작 순서 설명



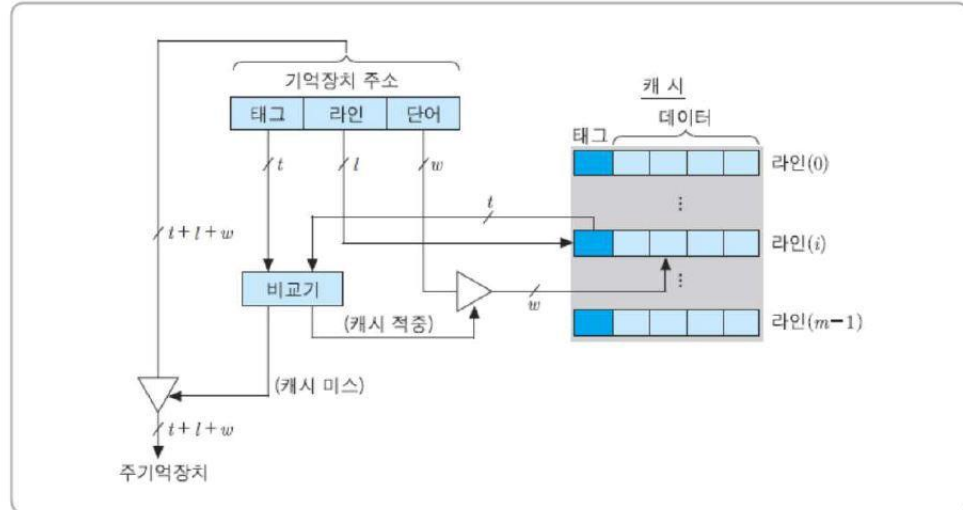
- CPU 에서 명령어 블록 인출을 위해 CPU-캐시기억장치-주기억장치 구조에서 캐시기억장치에 필요한 명령어 블록 존재 여부에 따라 주기억장치에 접근 여부를 결정하여 명령어를 CPU 로 전송

3. 캐시기억장치의 주소 사상 유형 별 동작 방식

가. 직접 사상 (direct mapping) 동작 방식

구분	동작 요소	세부 설명		
		t	s	w
주소 형식	메모리 주소	태그 필드	라인 필드(슬롯)	단어 필드
	주소 필드	- 태그 필드: 태그 번호 - 라인 필드: 캐시 라인 중 하나를 지정 - 단어 필드: 각 블록 내 단어 중 하나를 구분		
동작 과정	캐시 라인 선택	- 캐시로 메모리 주소 전달 시 s 비트의 라인 번호 기반 캐시 라인 선택		
	태그 비교 후 캐시 적중	- 선택 라인의 태그 비트와 주소 비교 - 일치 시 단어 필드 라인 내 단어를 인출하여 전송		
	태그 비교 후 캐시 미스	- 태그 값 불일치 시 캐시 미스로 주기억장치에 접근 - 주기억장치에서 명령어를 인출하고 해당 라인을 교체		

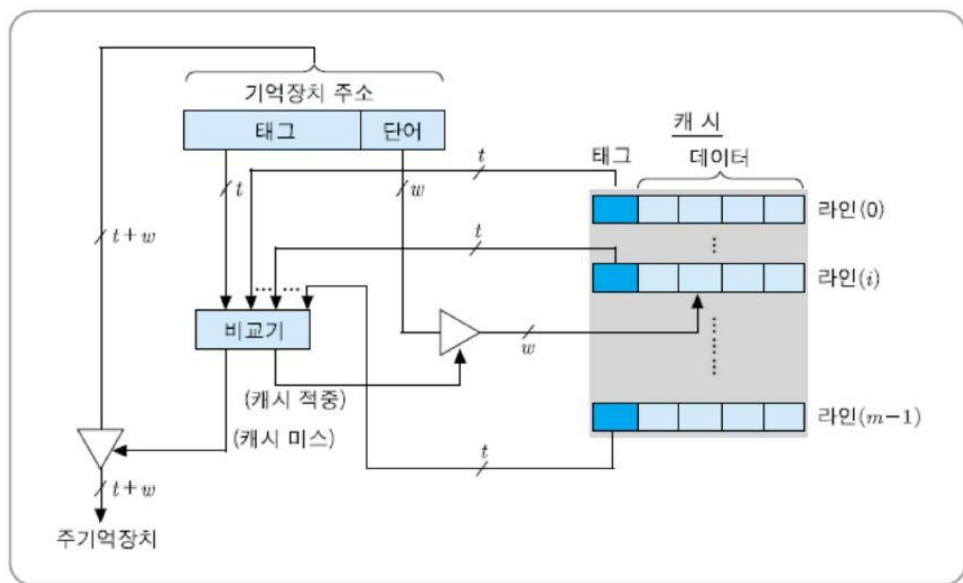
흐름도



나. 연관 사상 (associative mapping) 동작 방식

구분	동작 요소	세부 설명
주소 형식	메모리 주소	t 태그 필드 w 단어 필드
	주소 필드	- 태그 필드: 태그 번호 - 단어 필드: 각 블록 내 단어 중 하나를 구분
동작 과정	태그 비교 후 캐시 적중	- 메모리 블록은 캐시의 아무 블록에나 적재 가능 - 태그 필드의 내용을 캐시 전체와 비교하여 일치 시 전송
	태그 비교 후 캐시 미스	- 태그 값 불일치 시 캐시 미스로 주기억장치에 접근 - 주기억장치에서 명령어를 인출하고 해당 라인을 교체

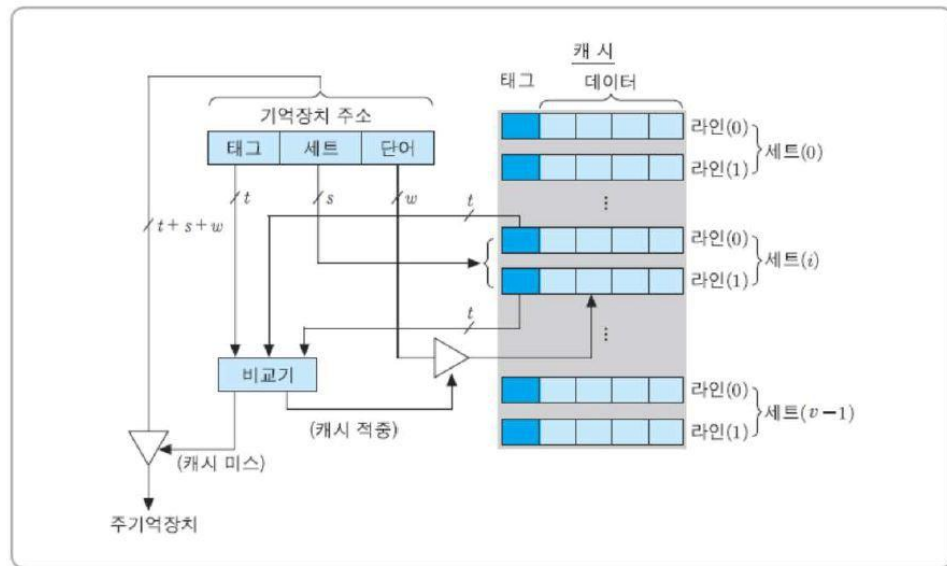
흐름도



다. 집합-연관 사상 (set-associative mapping) 동작 방식

구분	동작 요소	세부 설명
주소 형식	메모리 주소	t d w 태그 필드 집합 필드 단어 필드
	주소 필드	- 태그 필드: 태그 번호 - 집합 필드: 여러 메모리 슬롯을 하나의 집합으로 구분 - 단어 필드: 집합 내 단어 중 하나를 구분
동작 과정	집합 선택	- 메모리 주소 집합 비트를 이용하여 하나의 집합을 선택
	캐시 적중	- 선택한 집합 내 태그 비트와 주소 비교 - 일치 시 단어 필드 라인 내 단어 인출
	캐시 미스	- 태그 값 불일치 시 캐시 미스로 주기억장치에 접근 - 주기억장치에서 명령어를 인출하고 해당 라인을 교체

흐름도



- 캐시기억장치는 지역성의 원리(Locality)를 기반으로 캐시기억장치의 구조 및 동작 방식이 시스템 성능에 큰 영향을 미치므로 캐시기억장치의 구조 및 동작 방식 설계 시 시스템 성능 향상에 대해 고려해야 함.

4. 시스템 성능 향상을 위한 캐시기억장치 구조 및 동작 방식 설계 고려사항

구분	고려사항	사례
캐시메모리 크기	- 용량이 크면 적중률은 높아지지만 접근 시간이 길어지므로 증가한 만큼의 속도 향상을 가져오지 못함	- 1k ~ 128k 단어가 최적
캐시기억 장치의 수	- 캐시를 계층적으로 설치하거나 기능별로 분리된 다수의 캐시기억장치를 사용하는 것이 보편화	- 계층 수 - 통합/분리 형태
블록 크기	- 블록이 커질수록 많은 정보를 읽어올 수 있지만 블록 인출 시간이 길어지게. 블록이 커질수록 더 빈번히 교체	- 4~8 단어가 적당
캐시 인출 방식	- 주기억장치의 블록이 캐시 기억장치의 명령어나 데이터 블록을 인출해 오는 방식. - 주어진 데이터가 캐시에 있는지 어떻게 알아낼 것인가?	- 요구 인출 (Demand Fetch) - 선인출 (Prefetch)

블록 사상 방식	- 캐시는 메모리보다 용량이 작기 때문에 다수의 메모리 블록이 동일한 캐시 블록에 사상되는데, 메모리 블록을 캐시 속에 어떻게 배치할 것인가를 결정하는 전략을 의미 - 주어진 데이터가 캐시 어느 곳에 자리 잡을 것인가?	- 직접사상 - 연관사상 - 집합연관사상
블록 교체 방식	- 새로운 메모리 블록을 캐시 블록에 사상할 경우 기존 캐시 블록 중 하나를 선택해서 메모리로 내보내어 CPU 가 요청하는 새로운 메모리 블록이 사상될 공간을 마련하기 위해 어느 캐시 블록을 비울 것인지 결정하는 기법 - 캐시에 빈 공간을 어떻게 배열할 것 인가?	- Random - Priority - LFU - LRU - NUR
블록 갱신 방식	- 캐시의 블록이 변경되었을 때 그 내용을 주기억장치에 갱신하는 시기와 방법 - 데이터를 쓰는 작업은 어떻게 다룰 것인가?	- Write Through - Write Back

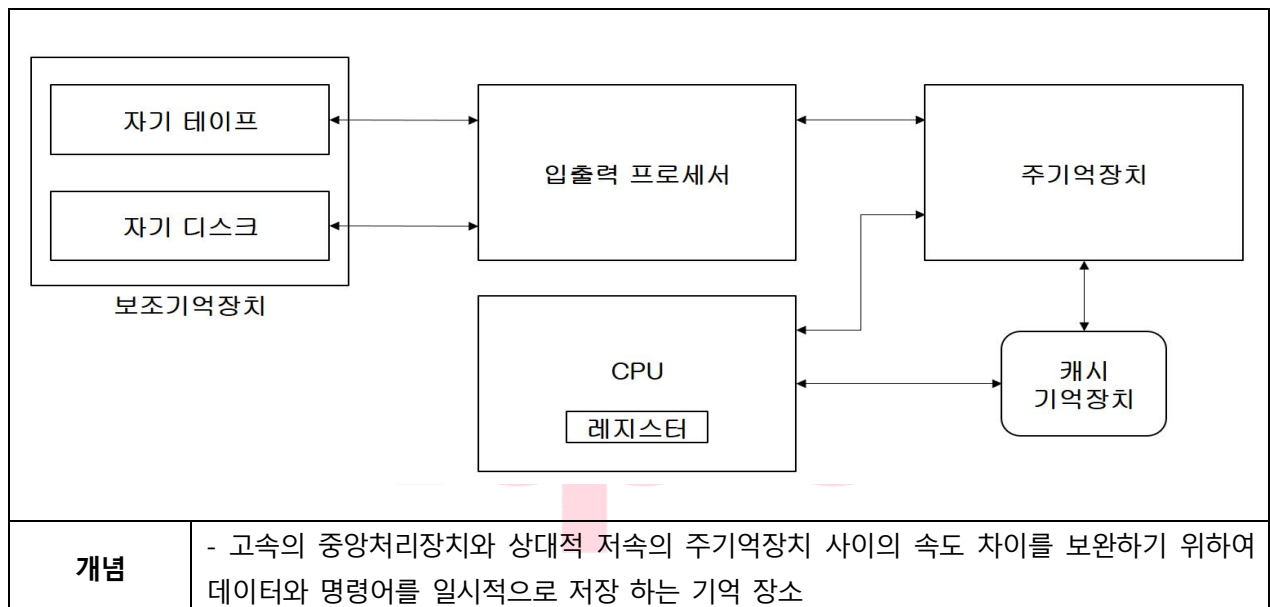
- 각 기법의 특징에 따른 캐시기억장치의 성능인 적중률(Hit Ratio), 캐시기억장치 접근 시간과 비용 관점의 영향도에 따른 적절한 설계 전략 필요

“끝”



문 제	4. 캐시 메모리의 쓰기 정책(Write Policy)과 캐시 일관성(Cache Coherence)을 유지하기 위한 기법에 대해 설명하시오.		
출 제 영 역	CA	난 이 도	★★☆☆☆ (별 5 개 기준)
출 제 배 경	캐시 메모리 일관성 유지 기법에 관련해서 출제된 적이 없고 기본적인 내용이므로 준비 필요		
출 제 빈 도	미출제		
참 고 자 료	- 컴퓨터 구조와 원리 2.0 (한빛아카데미)		
Key word	Write-through, Write-back, 캐시 일관성 유지 기법(SW/HW 기법 등으로 그룹핑해서 준비)		
풀이 기술사님	전정은 (122 회 정보관리기술사 / jjee0423@naver.com)		

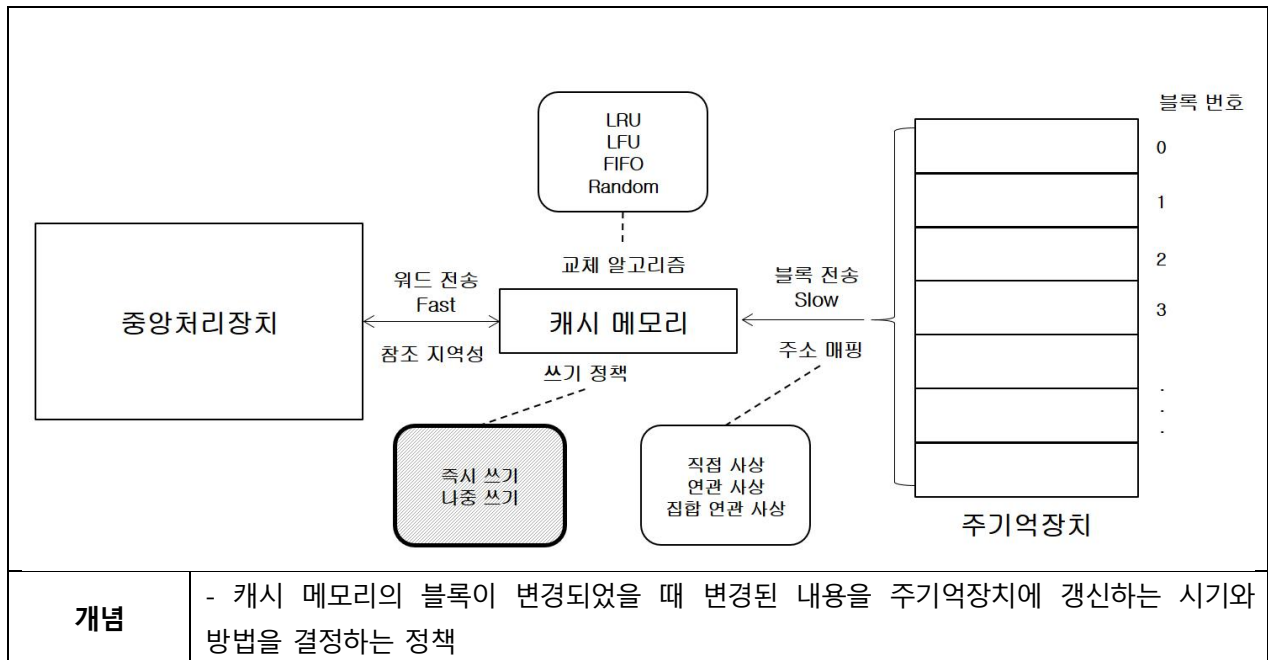
1. CPU의 처리 속도 향상을 위한 캐시 메모리의 개요



- 중앙처리장치가 프로그램을 실행할 때 연산 결과를 캐시 메모리에 기록하는 경우 캐시메모리와 주기억장치에 데이터를 갱신하는 절차가 필요함

2. 캐시 메모리의 쓰기 정책(Write Policy)

가. 캐시 메모리 쓰기 정책의 개념



- 캐시 메모리 쓰기 정책은 즉시 쓰기와 나중 쓰기, 두가지 정책이 존재함

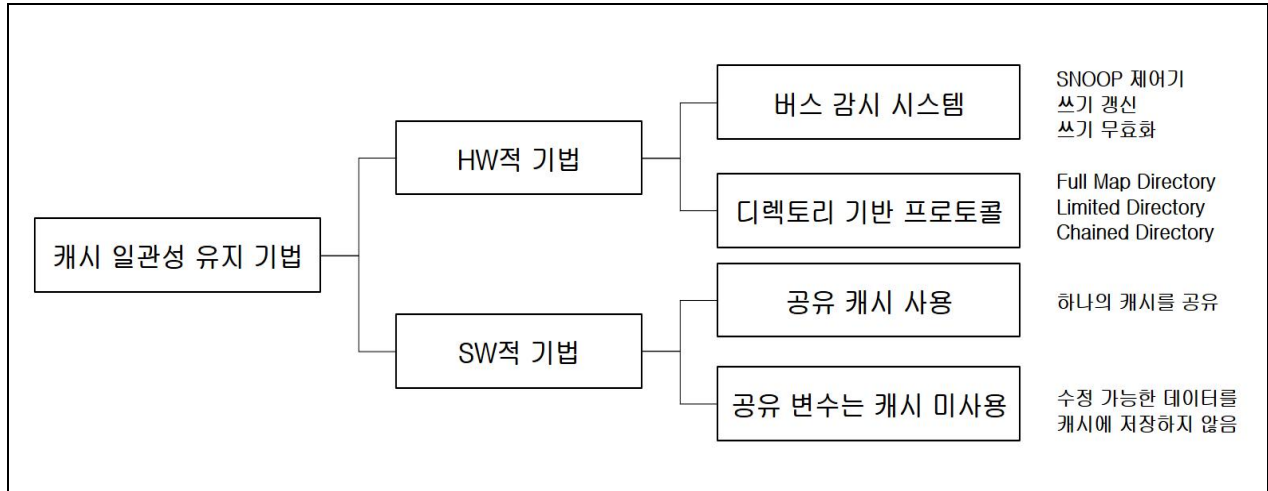
나. 캐시 메모리 쓰기 정책의 종류

구분	즉시 쓰기 (write-through)	나중 쓰기 (write-back)
개념도		
개념	- CPU의 연산 결과를 주기억장치와 캐시에 동시에 쓰는 방식	- CPU의 연산 결과를 캐시에만 쓰고 주기억장치에는 나중에 쓰는 방식
장점	- 단순한 구조의 아키텍처 - 캐시와 주기억장치의 일관성 유지	- 쓰기(write) 시간의 단축 - CPU의 메모리 접근 최소화
단점	- 쓰기 동작으로 인한 버스의 트래픽 증가 - 빈번한 주기억장치 접근으로 쓰기 시간의 증가	- 상대적으로 복잡한 구조의 아키텍처 - <u>캐시 일관성(Cache Coherence) 문제 발생</u>

- 단일 프로세서에서는 나중 쓰기 방식에서 캐시 일관성 문제가 발생 가능하지만 멀티 프로세서에서는 즉시 쓰기 방식과 나중 쓰기 방식 모두 캐시 일관성 문제가 발생 가능함

3. 캐시 일관성(Cache Coherence)을 유지하기 위한 기법

가. 캐시 일관성을 유지하기 위한 기법의 분류



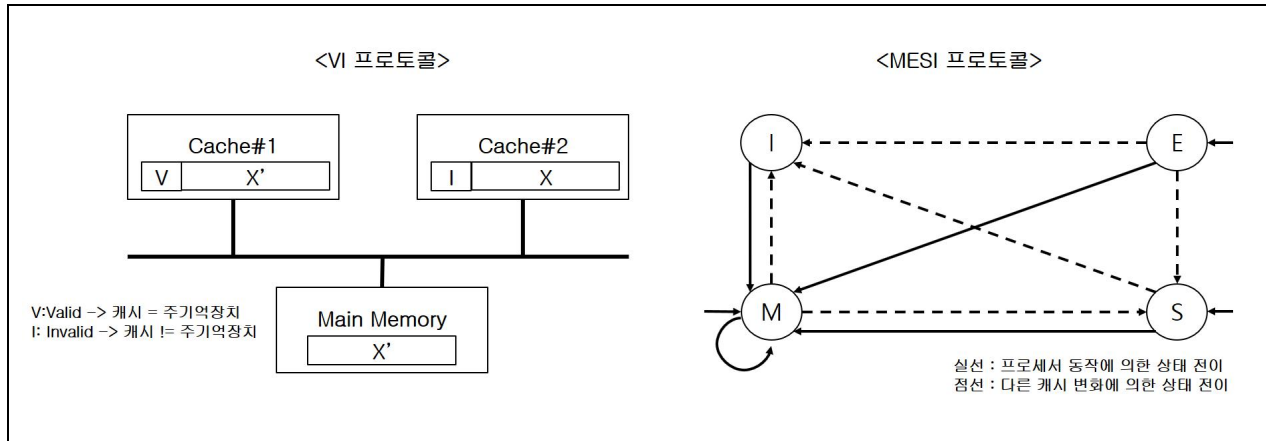
- 캐시 일관성을 유지하기 위한 기법은 크게 HW 적 기법과 SW 적 기법으로 분류할 수 있음

나. 캐시 일관성을 유지하기 위한 기법 상세 설명

구분	캐시 일관성 유지 기법	설명
HW 적 기법	버스 감시 시스템 사용	- 감시기능을 가진 장비를 시스템 버스에 추가 설치하는 방법 - 시스템 버스에 통신량이 증가하는 단점 존재 - SNOOP 제어기, 쓰기 갱신, 쓰기 무효화
	디렉토리 기반 프로토콜	- 캐시의 상태를 디렉토리에 저장하여 일관성을 유지하는 방법 - 수백 개 이상의 프로세서로 구성되는 대규모 시스템에 적용 - Full map directory, Limited Directory, Chained Directory
SW 적 기법	공유 캐시 사용	- 시스템 설계에서 가장 간단한 방법 - 다수의 프로세서가 하나의 캐시 기억장치를 공유하는 방법 - 다중 프로세스가 동시에 캐시에 접근할 때 충돌 발생 가능
	공유 변수는 캐시 미사용	- 수정 가능한 데이터를 캐시 기억장치에 저장하지 않는 방법 - 수정이 발생한 데이터는 캐시가 아닌 주기억장치에 저장 - 사용자가 캐시에 저장 가능한지를 선택하여 선언해야 함

- HW 적 기법 중 버스 감시 시스템을 사용한다면 Write-through 는 VI 프로토콜, Write-back 은 MESI 프로토콜을 사용하여 일관성을 유지할 수 있음

4. 캐시 일관성을 유지하기 위한 VI 프로토콜과 MESI 프로토콜

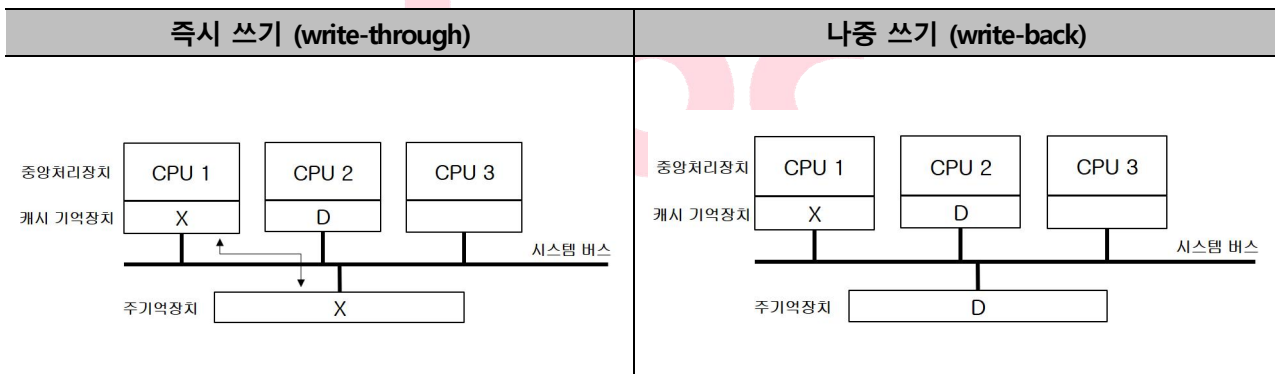


- VI 프로토콜은 메인 메모리와 일치 여부를 나타내는 플래그를 공유하며 MESI 프로토콜은 수정, 베타, 공유, 무효 상태를 나타내는 플래그를 공유하여 캐시 일관성을 유지할 수 있도록 하는 역할을 수행함

“גב”

[기타 자료]

멀티프로세서 시스템에서의 데이터 불일치(캐시 일관성 유지 기법의 필요성)

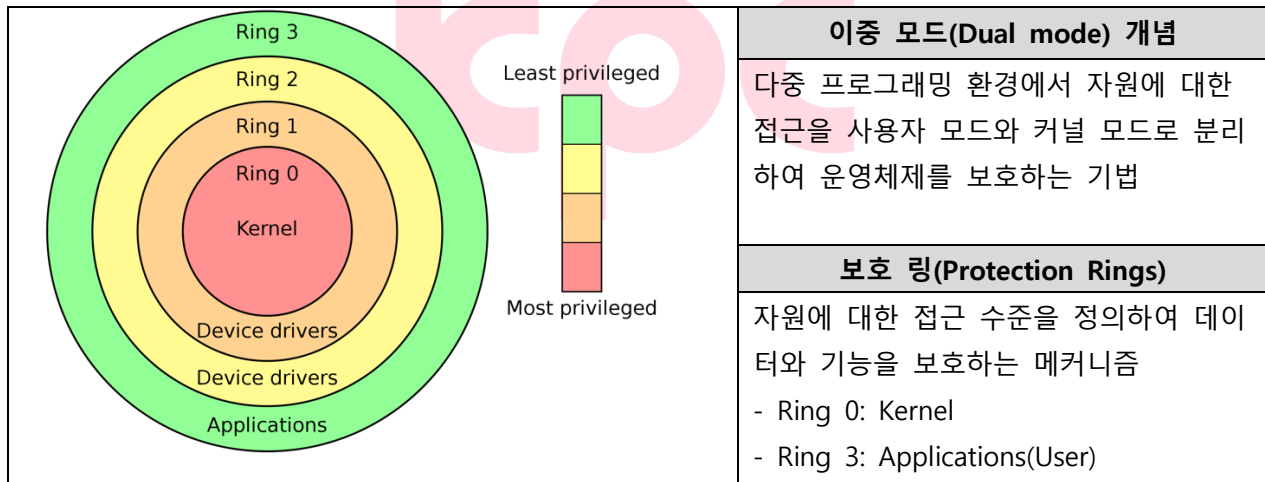


- 단일 프로세서에서는 나중 쓰기 방식에서만 캐시 일관성 문제가 발생하지만, 멀티 프로세서에서는 즉시 쓰기, 나중 쓰기 방식 둘 다 캐시 일관성 문제가 발생하므로 캐시 일관성 유지 기법이 필요함

문 제	4. 운영체제 다중 프로그래밍 환경에서 이중 모드(Dual mode) 구조에 대하여 다음을 설명하시오		
	가. 이중모드 개념		
	나. 이중모드의 구조		
	다. 이중모드의 종류		
출 제 영 역	운영체제	난 이 도	★★★★☆
출 제 배 경	이중모드에 대한 이해		
출 제 빈 도	미출제		
참 고 자 료	- https://en.wikipedia.org/wiki/Protection_ring - Operating System Concepts (10 th Edition)		
Key word	사용자 모드(User mode), 커널 모드(Kernel mode)		
풀 이	이진주(123 회 정보관리기술사)		

1. 이중 모드 개념

가. 이중 모드 개념



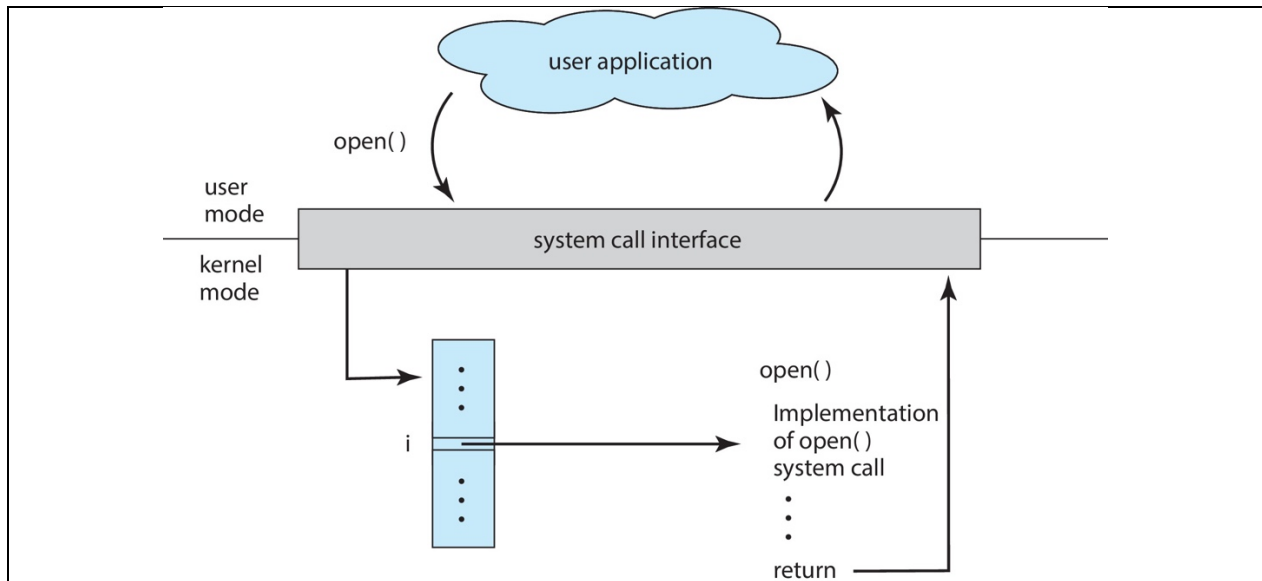
나. 이중 모드의 역할

구 분	역 할	설 명
사용자	보안 제공	- 악의적인 사용자로부터 운영체제 보호
	장애/결함 대응	- Fault Tolerance 개선 및 성능 향상
시스템	장치 보호	- 디바이스 별 드라이버 접근 권한 제어
	메모리 보호	- 메모리 접근 관리 및 제한
	CPU 보호	- 프로세스의 CPU 독점 제어

- 이중 모드 구현은 사용자 관점의 안정성과 시스템 수준의 신뢰성 제공

2. 이중 모드의 구조

가. 이중 모드 구조



- 이중 모드는 사용자 애플리케이션, 시스템 콜 인터페이스, 커널로 구성

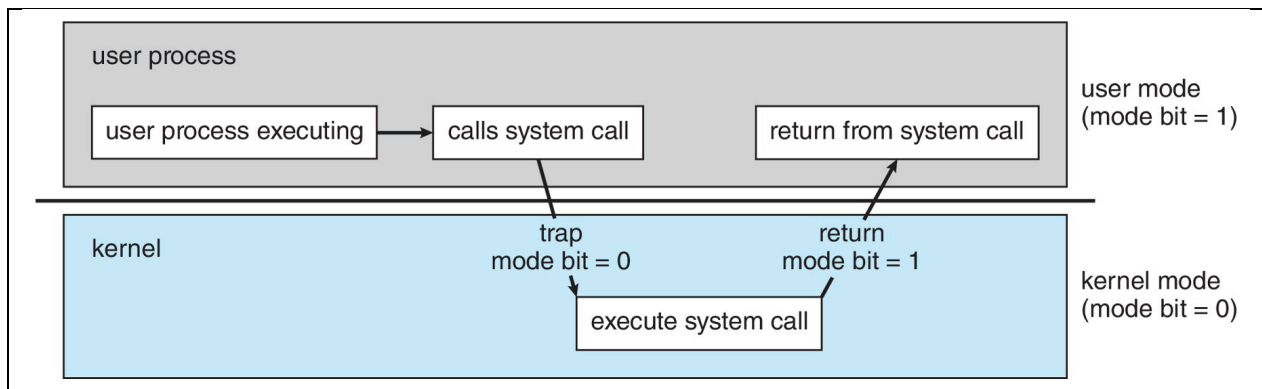
나. 이중 모드의 구현 설명

구분	모드	설명
User Application	User mode	- open(): 사용자가 애플리케이션 실행 - User mode 에서 프로그램 실행
System Call Interface	User mode -> Kernel mode	- implementation of open(): 프로세스 실행으로 시스템 콜 발생 - 운영 체제의 커널에 시스템 콜 수행 요청 - Kernel mode 로 전환
Kernel	Kernel mode	- 커널에서 요청 작업 수행 후 결과 값 반환
User Application	Kernel mode -> User mode	- open() 함수 이후의 프로그램 계속해서 실행

- 프로세스에서 실행하는 코드의 유형에 따라 User mode 와 Kernel mode 전환(switch)하며 작업 수행

3. 이중 모드의 종류

가. 이중 모드 종류



- 이중 모드는 사용자 모드와 커널 모드가 있고, 하드웨어가 제공하는 mode bit 로 구분

나. 이중 모드 종류 설명

종 류	설 명	동의어
User mode	- 사용자 애플리케이션 코드 실행 - 시스템 데이터에 제한된 접근만 허용되며 하드웨어 직접 접근 불가 - 시스템 콜 수행 시 사용자 모드에서 커널 모드로 전환	- slave mode - unprivileged mode
Kernel mode	- 커널 모드에서 운영체제는 모든 하드웨어 접근 및 제어 가능 - 시스템의 모든 메모리에 접근할 수 있고 모든 CPU 명령 실행 - 일부 명령어는 커널 모드 특권(privileged) 수준에서만 코드 실행	- master mode - privileged mode - supervisor mode

- mode bit = 1: user mode, mode bit = 0: kernel mode

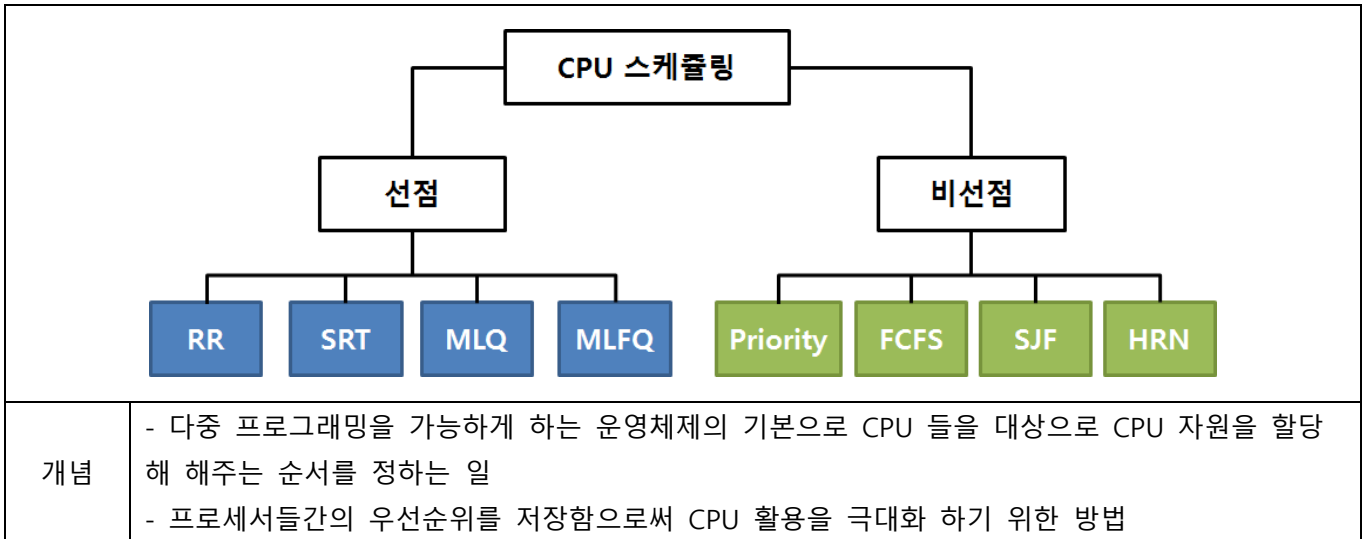
“끝”

기출풀이 의견

이중모드의 개념으로 적절한 분량의 답안 작성이 어려운 경우, 운영체제 다중 프로그래밍 환경에서 이중모드의 필요성이나, 이중모드의 역할을 작성하면 깊이 있는 내용의 답안이 될 수 있습니다.

5	CPU 스케줄링		
문제	CPU 스케줄링의 선점/비선점 방식을 설명하고, 각 방식의 알고리즘에 대하여 설명하시오		
도메인	OS	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제배경	운영체제 기본 고전토픽에 대한 이해 확인		
핵심키워드	선점 스케줄링, 비선점 스케줄링, RR, SRT, MLQ, MLFQ, Priority, FCFS SJF, HRN		
참고문헌	그림으로 배우는 원리와 구조 운영체제(한빛아카데미, 구현회)		
출제자	홍 OO 기술사(제 114 회 컴퓨터시스템응용기술사 / lkalover@naver.com)		

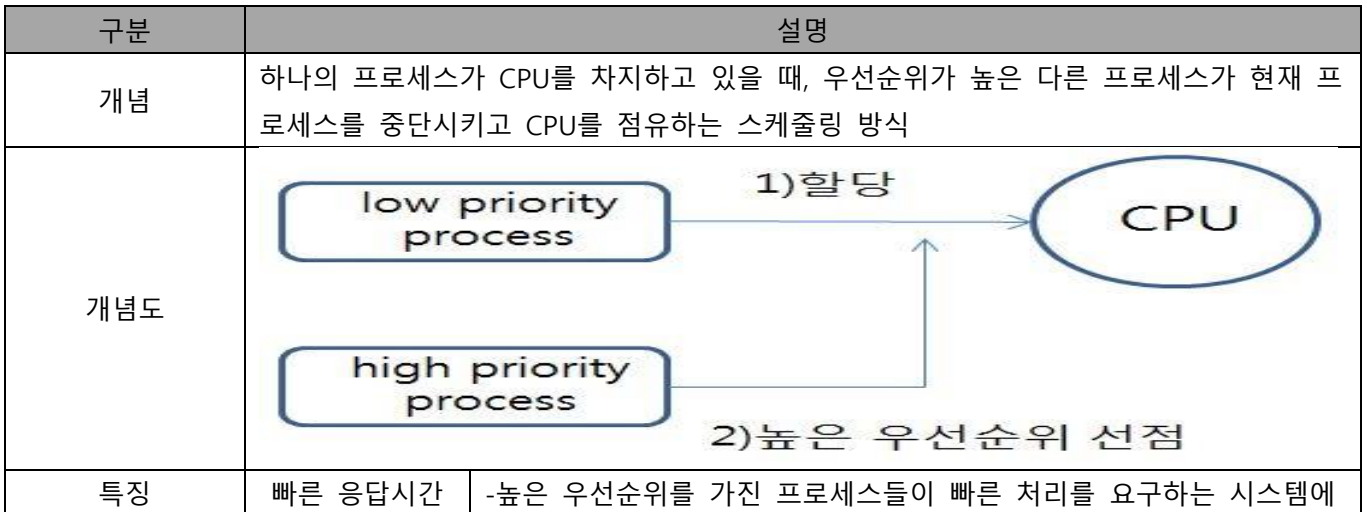
1. CPU 작업 순서 정의 및 활용률 최대화, CPU 스케줄링의 개요



- 운영 체제가 프로세스들에게 CPU 의 자원 배정을 적절히 함으로써 시스템의 성능을 개선할 수 있음
- CPU 스케줄링은 선점 방식과 비선점 방식이 존재함.

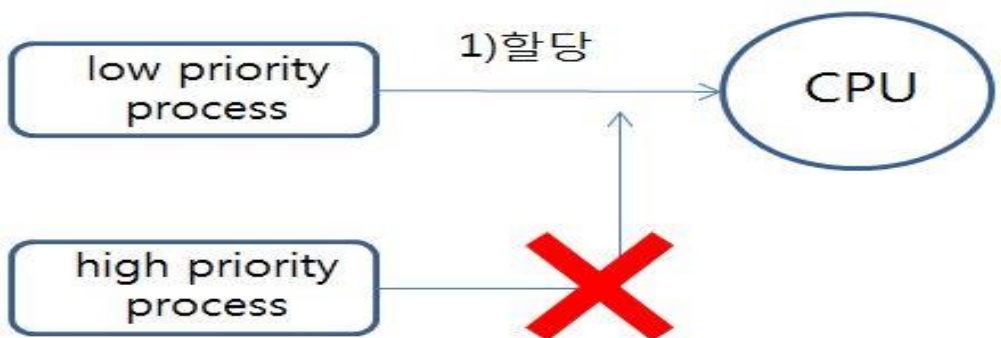
2. CPU 스케줄링의 선점과 비선점 방식 설명

가. CPU 스케줄링의 선점 방식 설명



	제공	유용함 -대화식 시분할 시스템에 적합
	오버헤드 발생 가능성	-선점으로 인한 오버헤드 초래

나. CPU 스케줄링의 비선점 방식 설명

구분	설명	
개념	한 프로세스가 CPU 를 할당 받으면 작업 종료 후 CPU 반환 시까지 다른 프로세스는 CPU 점유가 불가능한 스케줄링 방식	
개념도	 2) 선점 불가, 이전 프로세스 완료까지 대기	
특징	응답시간 예측 가능	-응답시간 예측이 용이함 -처리 편차가 적은 특정 프로세스 환경에 적합
	공정한 처리	-모든 프로세스들을 공정하게 처리함

- 선점 스케줄링은 상대적으로 빠른 응답속도를 보이지만 높은 우선순위의 프로세스가 빈번할 경우 기아현상이 발생할 수 있으며 비선점 스케줄링은 응답시간을 예측할 수 있는 반면 짧은 수행시간을 가진 프로세스가 장시간 대기 가능함

3. 선점, 비선점 방식의 알고리즘 설명

가. 선점 스케줄링 알고리즘

알고리즘	설명	특징
RR (Round Robin)	-프로세스는 같은 크기의 CPU 시간을 할당(Time Quantum), 프로세스가 할당된 시간 내에 처리 완료 못하면 준비큐 리스트의 가장 뒤로 보내지고, CPU는 대기중인 다음 프로세스로 넘어감	-균등한 CPU점유시간
SRTF (Shortest Remaining Time First)	-가장 짧은 시간이 소요되는 프로세스를 먼저 수행, 남은 처리 시간이 더 짧다고 판단되는 프로세스가 준비 큐에 생기면 언제라도 프로세스가 선점됨	-짧은 수행시간 프로세스 우선 수행
다단계 큐 (Multi Level Queue)	-작업들을 여러 종류 그룹으로 분할, 여러 개의 큐를 이용하여 상위단계 작업에 의한 하위단계 작업이 선점 당함, 각 큐는 자신만의 독자적인 스케줄링을 가짐	-독립된 스케줄링 큐
다단계 피드백 큐 (Multi Level Feedback Queue)	입출력 위주와 CPU 위주인 프로세스의 특성에 따라 큐마다 서로 다른 CPU Time Quantum을 부여, 새로운 프로세스는 높은 우선순위, 프로세스의 실행시간이 길어질수록 점점 낮은 우선순	-큐마다 다른 Time Quantum -마지막단계는 RR

	위 큐로 이동 마지막 단계는 Round Robin적용	
--	-------------------------------	--

나. 비선점 스케줄링 알고리즘

알고리즘	설명	특징
FCFS (First Come First Service)	-프로세스가 대기큐에 도착한 순서에 따라 CPU할당함 -FIFO알고리즘이라고도 하며 수행시간이 긴 프로세스가 CPU를 독점하여 Convoy Effect 발생가능함	-도착한 순서대로 처리 -Convoy Effect
SJF (Shortest Job First)	-대기큐에 첫번째 도착 프로세스 처리 => Burst Time 짧은순 처리 => Burst Time 같으면 FIFO처리함	-기아현상 발생 가능성
HRN (Highest Response Ratio Next)	-Response Ratio = (대기 시간+서비스 시간)/서비스 시간 대기중인 프로세스 중 현재 Response Ratio가 가장 높은 것을 선택	-SJF의 기아현상 문제를 해결

- SJF 사용시 발생할 수 있는 기아현상 문제를 HRN 적용으로 해결할 수 있음

4. CPU 스케줄링 성능평가 기준 고려사항

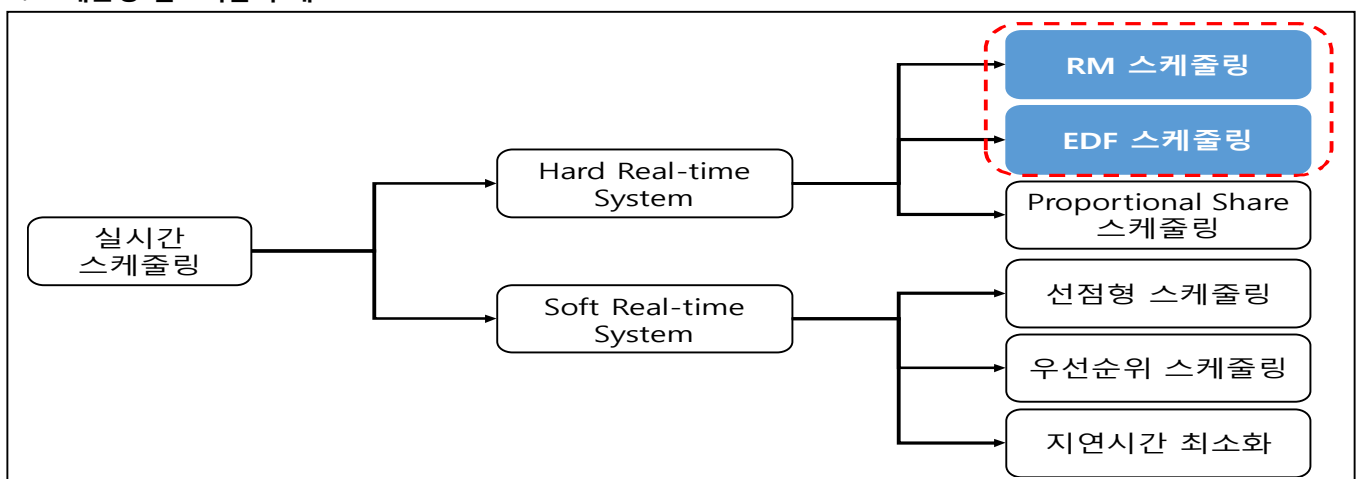
평가기준	고려사항
프로세스 사용율	-프로세서를 실행 상태로 항상 유지하여 유휴상태가 되지 않도록 함
처리율	-주어진 시간에 최대한 많은 작업 처리
반환시간	-전체작업 수행시간을 최소화
대기시간	-대기큐에 기다리는 시간을 최소화
응답시간	-대화식 시스템에서 첫 응답까지의 시간 최소화

- 스케줄링 알고리즘 선정시 각 알고리즘의 특성을 고려하여 선정.

“끝”

8	RM, EDF		
문제	실시간 스케줄링 알고리즘인 RM(Rate Monotonic)과 EDF(Earliest Deadline First)에 대해 설명하고, 다음 3 개의 태스크(Task)를 스케줄링 하시오.		
	프로세스	수행시간	주기
	P1	15	30
	P2	15	40
	P3	5	50
도메인	컴퓨터구조/운영체제		
출제배경/의도	컴시응에서 지속적으로 출제 되었으며, 정확하게 풀이하면 고득점을 맞을 수 있는 문제기 때문에 시험전 연습을 위해 출제. Task 가 3 개인 상황에서의 출제를 고려		
키워드	● 비율단조 스케줄링, (1.0, 0.828, 0.779) ● 경성 실시간, 연성 실시간, 선점, 정적/동적		
목차예시	1. 스케줄링 알고리즘의 개요 2. 실시간 스케줄링 알고리즘인 RM 과 EDF 에 대해 설명 3. 주어진 3 개의 태스크(Task)에 대한 스케줄링		
출제자	김유성 기술사(113 회 정보관리기술사/yousungkim1216@gmail.com)	난이도	★ ★ ★ ☆ ☆
참고문헌	https://ko.wikipedia.org/wiki/최단_마감_우선_스케줄링 kpc 모의고사 2016.11 관리 3 교시		

1. 스케줄링 알고리즘의 개요



- 실시간 스케줄링은 무조건 실시간 처리를 보장하는 것이 아니라 실시간 처리가 불가능한 작업을 거부함을 의미
- 실시간 운영체제(RTOS)에서 수행결과에 대해 시간적 제약조건을 만족하고, 마감시간(Deadline)내에 프로세싱을 보장하기 위한 스케줄링 기법. 이 중에 Hard Real-time system 에서는 주로 RM 과 EDF 알고리즘을 통해 스케줄링

2. 실시간 스케줄링 알고리즘인 RM 과 EDF 에 대해 설명

가. RM(Rate Monotonic)에 대한 설명

구분	상세설명
개념	독립적인 개별 프로세스 주기를 기준으로 주기가 짧을수록 높은 우선 순위를 부여하는 정적 방식의 스케줄링 기법
정책	정적 우선순위
목적	CPU를 더 자주 필요로 하는 태스크에 더 높은 우선순위 부여
우선순위 부여방식	태스크간 선행 관계 없는 조건 만족시, 수행전에 주기가 짧은 태스크가 높은 우선순위 부여
CPU 이용률 한계	- 프로세스 1개 : 100% - 프로세스 2개 : 83% - 프로세스 3개 : 78% - 프로세스 n개 : 69%까지 떨어짐
오버헤드	스케줄링 오버헤드가 낮음
장점	- 프로세스에 대한 마감시간 정보 없이 이용률 한계 제약이 없다면 최적의 스케줄링 - 연성 실시간 시스템 요구조건 만족
단점	- 이용률 한계로 마감시간 만족을 보장할 수 없음 - 주기가 긴 태스크들의 우선순위가 낮아 장시간 대기 - 경성 실시간 시스템 요구조건 만족 못함

- RM 스케줄링은 Soft RTOS 에 적합

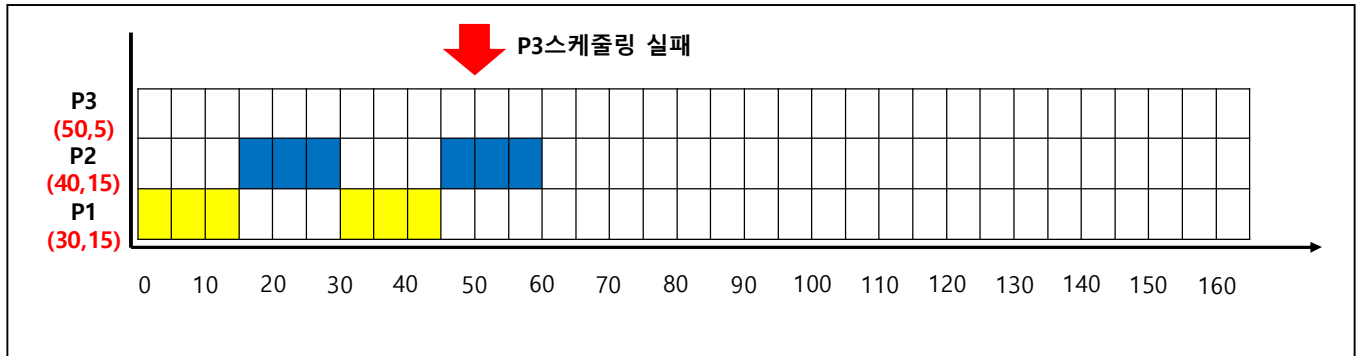
나. EDF(Earliest Deadline First)에 대한 설명

구분	상세설명
개념	임의의 실행 시점에서 가장 가까운 마감시간을 갖는 프로세스에게 높은 우선순위를 할당하는 동적 방식의 스케줄링 기법
정책	동적 우선순위
목적	마감 시간에 따라서 우선순위를 동적으로 부여
우선순위 부여방식	새로 실행하게 된 프로세스의 마감 시간에 맞춰 조정
CPU 이용률 한계	- 이론적으로 100%, 한계 없음 - 문맥교환 비용으로 100%는 불가능
오버헤드	스케줄링 오버헤드가 높음
장점	- 프로세스들이 주기적일 필요 없고, CPU할당 시간도 상수 값으로 정해질 필요 없음 - 이론적으로 총 이용률이 1이하면 스케줄링 가능 - 경성 실시간 시스템 요구조건 만족(마감시간 만족 가능)
단점	- 프로세스가 실행 가능해질 때 자신의 마감시간을 알려주어야 함 - 태스크들의 수행시간, 마감시간, 주기 등 정확히 예측하는 것이 현실적으로 어려움

- EDF 스케줄링은 Hard RTOS 에 적합

3. 주어진 3 개의 태스크(Task)에 대한 스케줄링

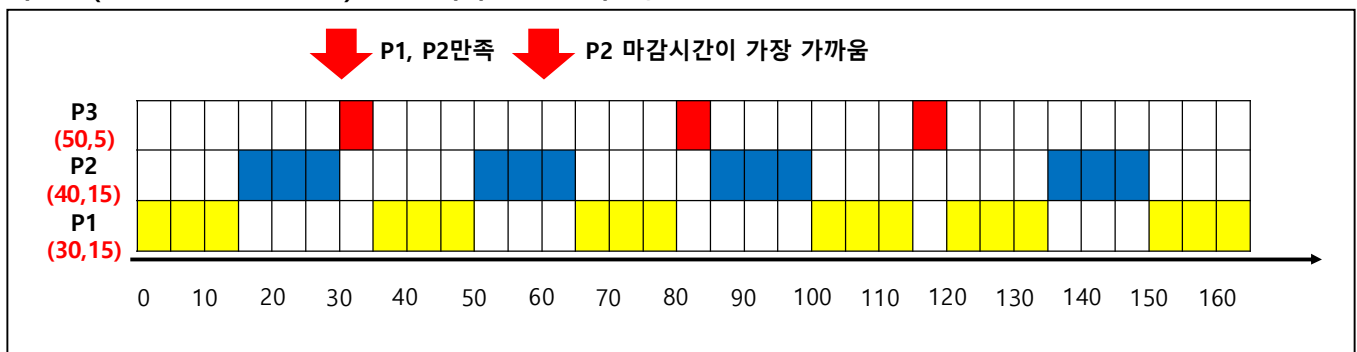
가. RM(Rate Monotonic)으로 3 개의 Task 스케줄링



구분	설명
공식을 통한 스케줄링 가능여부 확인	- CPU 이용률(U) < $N(2^{1/N}-1)$ 조건 성립 시 정상 스케줄링 (N은 프로세스 수) - CPU 이용률(U) = $(P1\text{실행시간}) / (P1\text{주기}) + (P2\text{실행시간}) / (P2\text{주기}) + (P3\text{실행시간}) / (P3\text{주기})$ $15/30 + 15/40 + 5/50 = 0.5 + 0.38 + 0.1 = 0.98$ (98% CPU이용률) - 프로세스가 3개일 때 CPU 이용률 = 0.77 - CPU 이용률이 98%로 77%보다 크기 때문에 마감시간을 충족하지 못하는 스케줄링 일 가능성이 있음
결과	- P3의 경우 주기가 길어 우선순위가 P1, P2보다 떨어짐 - P3의 마감시간인 50이 되어도 P1, P2에 선점되어 P3는 실행하지 못함

- 프로세스 수(N)이 무한대이면, $U = \log_2(2)$ (약 0.693). 즉 CPU 이용률이 69.3% 이하이면 항상 스케줄링 가능

나. EDF(Earliest Deadline First)으로 3 개의 Task 스케줄링



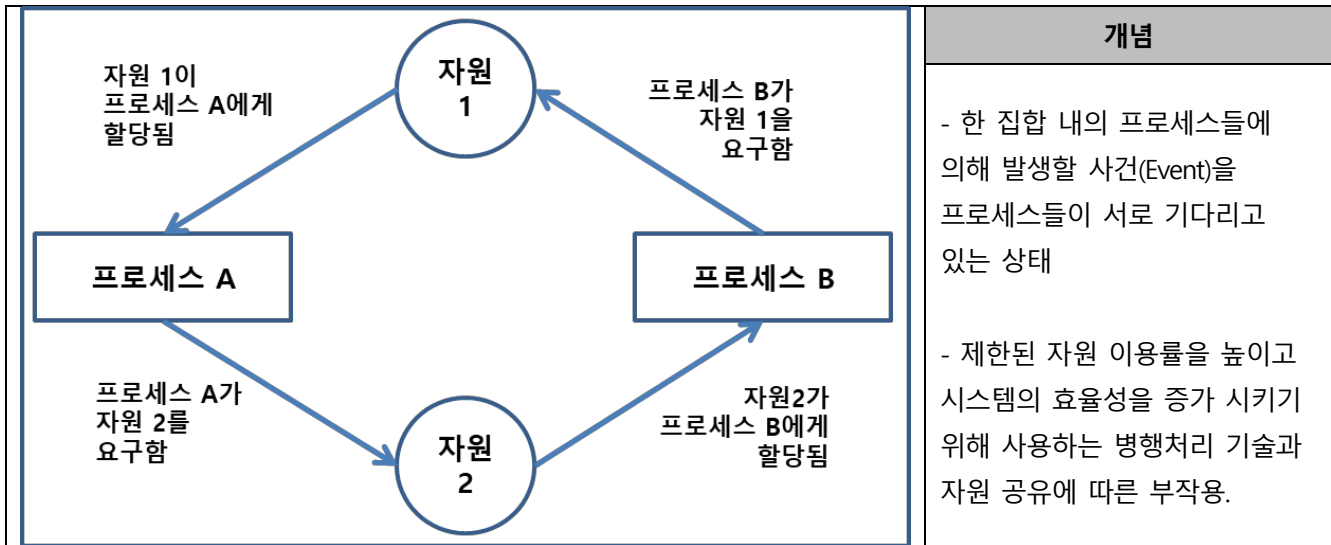
구분	설명
공식을 통한 스케줄링 가능여부 확인	이론적으로 100%, 한계 없음
결과	- 모든 프로세스의 마감시간을 준수하면서 스케줄링 가능 - EDF의 우선순위는 새로 실행 가능하게 된 프로세스의 마감시간에

- EDF 스케줄링은 이론상 CPU 이용률의 최대 100%까지 보장하지만, 프로세스 사이 또는 인터럽트 핸들링 때의 문맥교환 비용으로 100%의 CPU 이용률은 현실적으로 불가능함

“끝”

8	교착상태
문제	교착상태의 발생 조건과 해결방안을 설명하고, 회피 기법인 Banker's algorithm 과 Wait-Die, Wound-Wait 를 설명하시오.
도메인	컴퓨터구조/운영체제
출제배경/의도	CA 기본 토픽, 교착상태의 발생 원인과 회피기법 알고리즘 이해 확인
키워드	● 자원 요구, 할당, 상호배제, 점유와 대기, 비선점, 환형 대기 ● 예방, 회피, 발견, 복구, ● Available, Max, Allocation, Need, 안정상태, wait-die, 비선점, wound-wait, 선점
목차예시	1. 자원의 요구가 뒤엉킨 상태, 교착상태의 개요 2. 교착상태의 발생 조건과 해결 방법 3. Banker's Algorithm 과 Wait-Die, Wound-Wait 설명
채점 점수 가이드	① 개념, 이해 부족 (1~10점) ② 기본 이해 수준 (11~13점) ③ 정확한 기술 제시 (14~16점) ④ 추가요소 설명(+α)
난이도	★ ★ ☆ ☆ ☆
학습 가이드	은행가 알고리즘의 필요한 자료구조와 진행 되는 절차에 대해서 명확하게 작성 wait-die 와 wound-wait 의 발생하는 상황을 도식화로 설명하고 차이 설명
참고문헌	IT CookBook, 운영체제: 그림으로 배우는 원리와 구조 KPC 모의고사 53 회 관리 2 교시
출제자	공수재 기술사(제 111 회 컴퓨터시스템응용기술사 / ksujae22@naver.com)

1. 자원의 요구가 뒤엎킨 상태, 교착상태의 개요



2. 교착상태의 발생 조건과 해결 방법

가. 교착상태의 발생 조건

필수 조건	설명
① 상호배제	- 자원이 최소 하나 이상 비공유, 즉 한 번에 한 프로세스만 해당 자원을 사용할 수 있어야 함. (요청한 자원이 해제될 때까지 대기)
② 점유와 대기	- 최소한 자원 하나를 보유하고 다른 프로세스에 할당된 자원을 얻기 위해 기다리는 프로세스가 있어야 함.
③ 비선점	- 자원은 선점될 수 없음. 즉 자원을 강제로 뺏을 수 없으며 자원을 점유하고 있는 프로세스가 끝나야 해제됨.
④ 순환대기	- 대기 프로세스 집합 $\{P_0, P_1, \dots, P_n\}$ 이 있을 때, P_0 은 P_1 이 보유하고 있는 자원을, P_1 은 P_2 , P_2 는 P_3 , P_{n-1} 은 P_n , P_n 은 P_0 이 보유하고 있는 자원을 각각 얻기 위해 대기

- 위 네가지 조건이 동시에 발생할 때, 즉 필요 충분조건이 성립될 때 발생

나. 교착상태의 해결 방안

해결 방안	설명	특징 및 기술 요소
① 교착상태 예방	- 4가지 교착 상태 조건이 발생 되지 않도록 함.	- 자원 낭비 및 비용 증가 - Starvation 발생 가능
② 교착상태 회피	- 프로세스의 시작 거부 - 자원 할당의 거부 - 자원이 언제 요구되는지에 대한 추가 정보 필요	- Banker's algorithm (은행가 알고리즘) - Wait-Die, Wound-Wait
③ 교착상태 발견	- 시스템의 상태를 감시하는 알고리즘을 통하여 교착상태를 검사하는 알고리즘	- 자원 할당 그래프 - 교착상태 검출(Graph reduction, cycle Detection, Knot detection)
④ 교착상태 회복	- 프로세스를 한 개 이상 중지 - 교착상태의 프로세스들로부터 자원 선점	- 선점자원 우선순위, 복귀, 기아 (Starvation)의 문제 해결 필요

- 교착상태 회피 방법으로 Banker's Algorithm 과 Wound-Wait, Wait-Die 알고리즘 활용

3. Banker's Algorithm 과 Wait-Die, Wound-Wait 설명

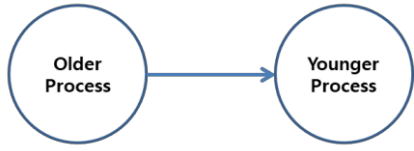
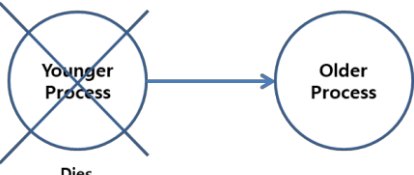
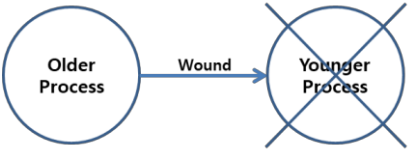
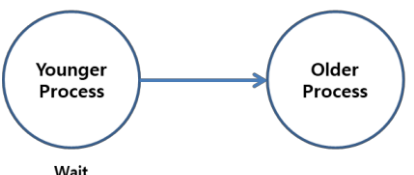
가. Banker's Algorithm

구분	설명	
개념	- 프로세스가 자원을 요구할 때 시스템은 자원을 할당한 후에도 안정 상태로 남아 있게 되는 지를 사전에 검사하고 교착상태의 발생을 회피하는 기법 - 자원 할당 상태가 안정이라면 처리가 이루어지고 프로세스 P_i는 자원을 할당 받음. 불안정 상태이면 P_i는 $Request_i$를 대기하고 이전 자원 할당 상태로 복귀함.	
자료구조	① Available	- 각 형태별로 사용 가능한 자원의 수 - $Available[j]=K$, 자원종류 R_j중 K개 사용가능
	② Max	- 각 프로세스의 최대 자원의 요구 - $Max[i,j]=K$, P_i는 자원종류 R_j중 최대 K개 요청.
	③ Allocation	- 현재 각 프로세스에 할당되어 있는 각 형태의 자원 수 - $Allocation[i,j]=K$, P_i는 현재 자원종류 R_j를 K개 할당받고 있음
	④ Need	- 각 프로세스에 남아 있는 자원 요구 - $Need[i,j]=(Max-Allocation)$, P_i는 자원종류 R_j를 K개 더 요구함
절차	① 현재 사용 가능한 리소스의 양 구함(Available)	
	② 추가 요구량을 구함(Request=Max-Allocation)	
	③ 추가 요구 자원이 현재 여유 자원보다 적은 프로세스를 탐색(수행 가능 프로세스) ($Request(i) < Available(i)$)	
	④ 반복 검사- 수행 가능한 프로세스가 점유한 자원을 여유 자원으로 바꾸는 가정으로 3번부터 반복($Available = Available + Allocation$)	
	⑤ 안정 상태 확인- 안정상태에 있는 경우 자원 할당, 불안정 상태 경우 다른 프로세스들이 자원을 해제 할 때까지 기다림	

- 할당 할 수 있는 자원의 수와 사용자 수가 일정해야 함.

- 항상 불안정 상태를 방지해야 하므로 자원 이용도가 낮음

나. Wait-Die, Wound-Wait 설명

구분	Wait-Die	Wound-Wait
개념	비선점 기법에 기반을 두고 먼저 시작한 프로세스가 요청하는 경우 wait, 새로운 프로세스가 요청하는 경우 die	선점 기법에 기반을 두고 먼저 시작한 프로세스가 요청하는 경우 선점(wound), 새로운 프로세스가 요청하는 경우 wait
개념도	Situation 1  Wait Situation 2  Dies	Situation 1  Wound Situation 2  Wait

고참 프로세스 요청	고참 프로세스가 기다림(wait)	신참 프로세스를 wound하여 선점
신참 프로세스 요청	신참 프로세스는 철회(die)	신참 프로세스는 대기(wait)

- 중지된 프로세스는 기존 타임 스탬프를 가지고 재시작하여, 무한정 복귀 되는 일을 방지(Starvation 해결)
- 구현이나 관리가 쉬우나, 프로세스 재수행 로직으로 성능이 저하 될 수 있음.

[기타자료]**Deadlock과 Starvation 비교**

구분	Deadlock	Starvation
개념	다수의 프로세스가 수행을 못하고 특정 사건을 기다리며 무한 대기	특정 프로세스가 자원을 할당 받기 위해 무한정 대기하는 상태
발생 원인	- 상호배제, 점유와 대기, 비선점, 환형 대기	- 자원의 편중된 분배 정책
해결 방안	- 예방, 회피, 발견, 회복	- Aging 기법

Deadlock과 Livelock 비교

구분	Deadlock	Livelock
개념	다수의 프로세스가 수행을 못하고 특정 사건을 기다리며 무한 대기	상대 프로세스의 자원 요구에 서로 양보하며 무한 대기 상태
실행 흐름	실행 흐름 막힘	실행 흐름은 막히지 않음
자원 소모	자원 소모 없는 waiting 상태	실행 될 때까지 자원 낭비
발생 원인	상호자원 요구	상호자원 양보
발생 조건	상호배제, 점유/대기, 비선점, 환형대기	선점, 다중 프로세싱
해결 방안	예방, 회피, 발견, 회복	Try and back off, Process kill

7	IPC(Inter Process Communication)		
문제	IPC(Inter Process Communication) 통신방식		
도메인	CA/OS		
출제배경/의도	IPC 통신방식에 대한 이해		
키워드	Message Queue, Shared Memory, Semaphore, Pipe, Named Pip, Memory Map, Socket, Signal		
목차예시	목차사례 1. 프로세스간 정보교환, IPC(Inter Process Communication)의 개요 2. IPC 통신방식		
출제자	전 XX 기술사 (제 116 회 컴퓨터시스템응용기술사/ahddl2004@naver.com)	난이도	★ ★ ☆ ☆ ☆
참고문헌	https://velog.io/@pa324/%EC%9A%B4%EC%98%81%EC%B2%B4%EC%A0%9C-IPC-%EA%B8%B0%EB%B2%95-71k2iuz9rh https://doitnow-man.tistory.com/110		

1. 프로세스간 정보교환, IPC(Inter Process Communication)의 개요

- 독립된 프로세스들 사이에 자원, 정보공유 및 교환 등을 위하여 서로 데이터를 주고 받는 행위
- Message Queue, Shared Memory, Semaphore 등의 통신방식 활용

2. IPC 통신방식

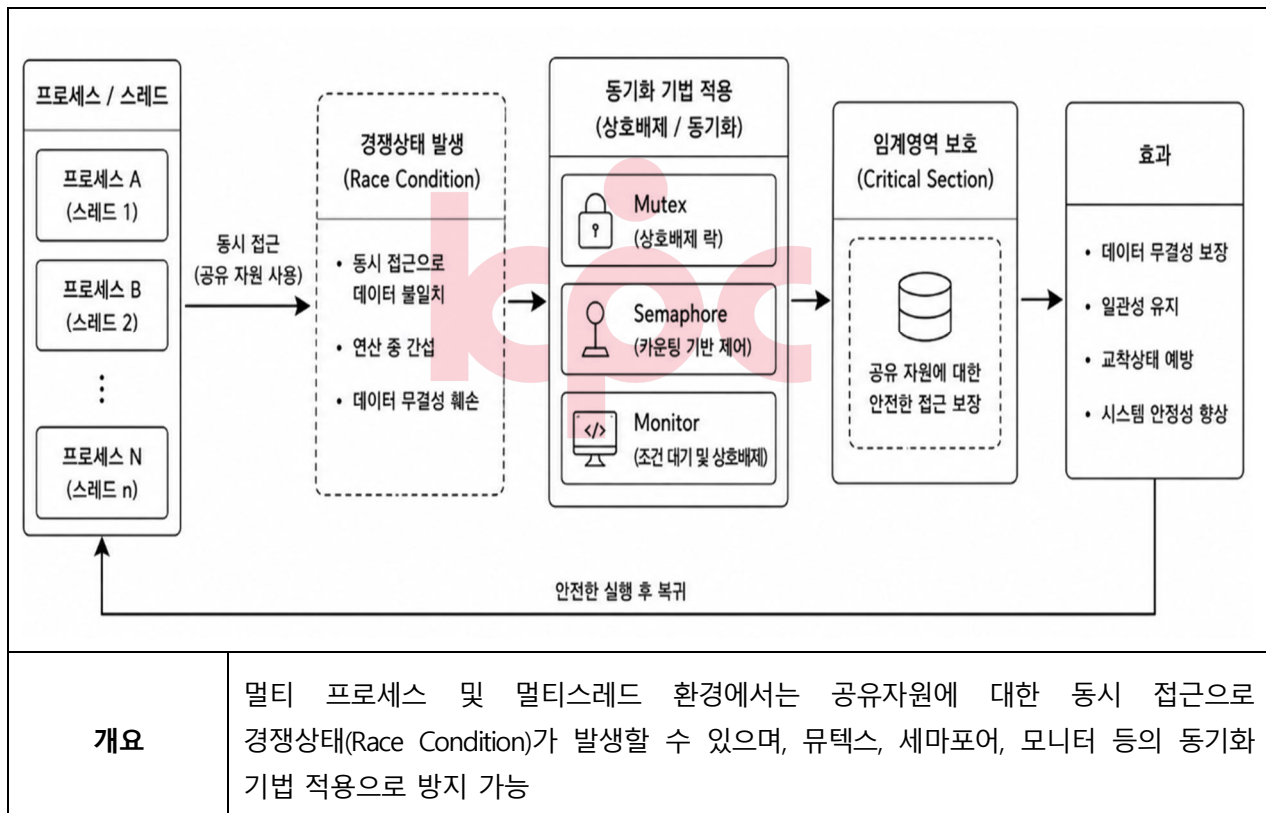
통신방식	개념도	주요 내용
Message Queue		● 커널 내 Message Queue를 이용하여 프로세스간 메시지를 전달하는 통신 방식 ● FIFO, LIFO(Last In First Out) 을 모두 지원하며, 고정된 크기의 구조체를 가지고 있음 ● FIFO, LIFO 모두 지원 ● 양방향 통신도 가능
Shared Memory		● 두 개 이상의 프로세스가 특정한 메모리 영역을 공유하는 것 ● 공유 메모리에 대한 읽기/쓰기를 통하여 데이터 통신 수행 ● 동기화 문제가 발생할 수 있음 ● 충돌 가능성은 있으나, 일반적으로 더 빠르고 커널 도움이 거의 필요 없음

Semaphore		● 두 개 이상의 프로세스가 하나의 공유 자원을 접근할 때 한 순간에 하나의 프로세스만 공유자원에 접근할 수 있도록 보장 ● 2개의 원자적 연산 P와 V가 정의 ● 이진 세마포어, 계수형 세마포어
Pipe		● 익명의 PIPE를 통해서 동일한 PPID를 가진 프로세스들 간에 단방향 통신을 지원 ● FIFO 구조로서 생성된 Pipe에 대한 Read/Write 가능 ● 부모/자식 프로세스간 통신에서 사용 ● 양방향 통신을 위해서는 Read와 Write를 위한 Pipe를 각각 생성
Named Pipe		● 이름을 가진 PIPE를 통해서 프로세스들 간 단방향 통신 지원 ● 상호 관련 없는 프로세스 간에도 PIPE 이름만 알면 통신 가능 ● 양방향 통신을 위해서는 Read와 Write를 위한 Pipe를 각각 생성
Memory Map		● 파일을 프로세스의 메모리에 일정 부분 맵핑해서 사용 ● 디스크에 있는 파일의 블록들과 프로세스의 페이지가 연결. ● 파일로 대용량 데이터를 공유 할 때 사용 ● 프로세스를 실행할 때 실행 파일의 각 세그먼트를 메모리에 사상하기 위해 메모리 맵 파일을 이용
Socket		● 네트워크 소켓통신을 사용한 데이터 공유 방법으로 원격에서 프로세스간 데이터를 공유 할 때 사용
Signal		커널 또는 프로세스에서 다른 프로세스에 어떤 이벤트가 발생되었는지를 알려주는 기법 프로세스 관련 코드에 관련 시그널 핸들러를 등록해서, 해당 시그널 처리 실행 SIGKILL, SIGINT, SIGSTOP 등

"끝"

문 제	2. 운영체제의 프로세스 동기화 기법 중 뮤텍스(Mutex), 세마포어(Semaphore), 모니터(Monitor)에 대하여 설명하시오.		
출 제 영 역	CA/OS	난 이 도	★★☆☆☆ (별 5 개 기준)
출 제 배 경	멀티스레드/멀티프로세스 환경에서 경쟁 조건(Race Condition)을 방지하고 상호배제(Mutual Exclusion)를 달성하기 위한 핵심 동기화 메커니즘의 이해도 검증		
출 제 빈 도	컴시응(138, 132)		
참 고 자 료	컴시응 기출풀이집, KPC 모의고사 해설집		
K e y w o r d	경쟁 조건(Race Condition), 임계 구역(Critical Section), 상호배제(Mutual Exclusion), Lock, P 연산, V 연산, 배타 동기 큐, 조건 동기 큐		
풀이 기술사님	김하랑(139 회 정보관리기술사 / harangg26@gmail.com)		

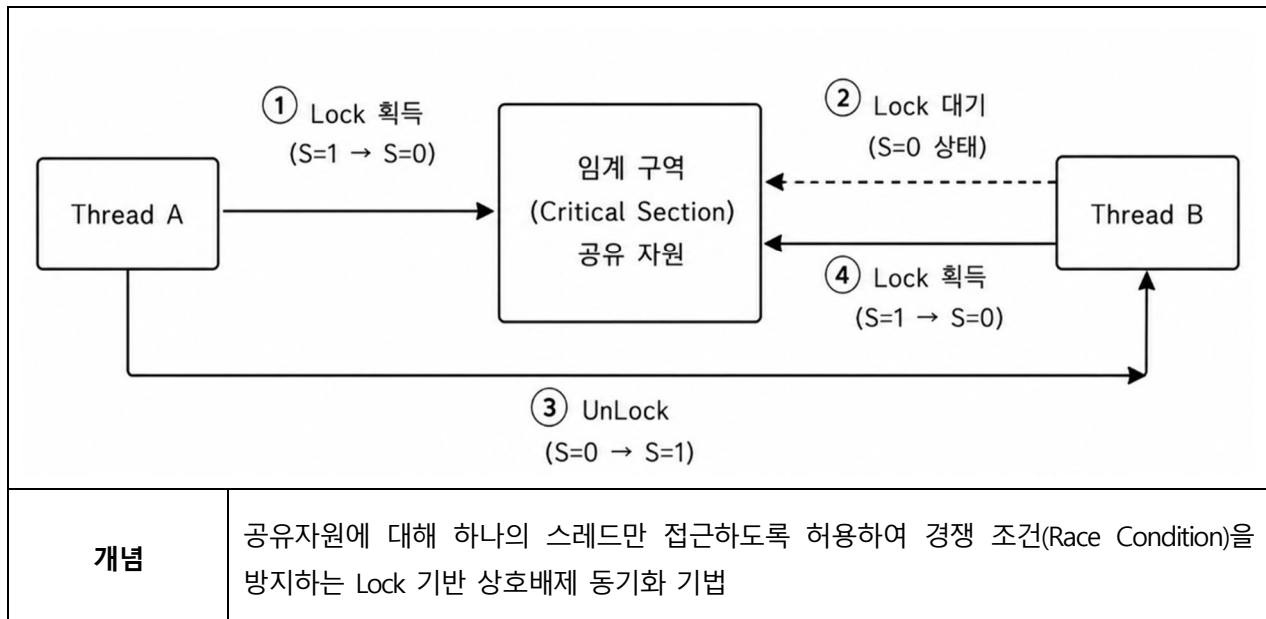
1. 경쟁상태 방지 및 동시성 제어를 위한 프로세스 동기화의 개요



- 공유자원에 대한 안전한 접근을 위해 가장 기본적인 상호배제 기법인 Mutex 가 활용됨

2. 상호배제를 위한 뮤텝스(Mutex) 설명

가. 뮤텝스의 개념



- Thread A가 Lock을 획득하여 임계 구역을 수행하는 동안 Thread B는 진입이 차단, 대기 상태에 머무르게 됨

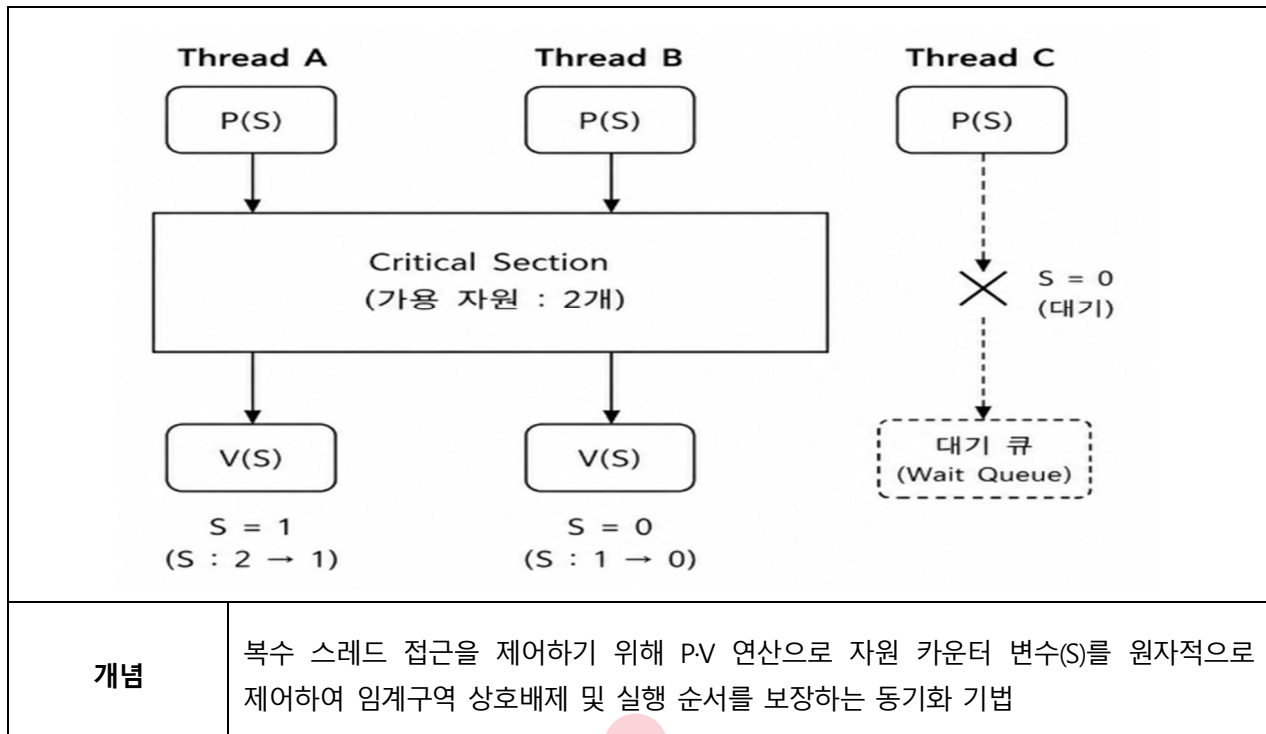
나. 뮤텝스 기술요소

구분	기술요소	설명
구성	소유권 (Ownership)	Lock을 획득한 오직 하나의 프로세스/스레드만이 해당 Lock을 해제할 수 있는 권한을 가짐.
	이진 플래그	자원의 접근 가능 상태를 나타내는 이진 플래그 (0: Locked, 1: Unlocked).
동작	lock() (Acquire)	임계 구역 진입 전 Lock을 요청하는 연산. 자원이 사용 중이면 대기(Busy Waiting 또는 Block).
	unlock() (Release)	임계 구역 수행 완료 후 자원을 반납하고 Lock을 해제하여 다른 스레드에 소유권을 이양하는 연산.

- 뮤텝스는 단일 자원 접근만 제어하므로 다수 자원 및 자원 개수 관리에는 Semaphore가 활용됨

3. 다수 자원관리를 위한 세마포어(Semaphore) 설명

가. 세마포어의 개념



- 가용 자원의 개수가 2 개인 상황에서 Thread A와 B는 자원을 할당받아 진입하지만, 세마포어 카운터(S)가 0 이 된 시점에 요청한 Thread C는 자원을 얻지 못하고 대기 큐에서 블록됨

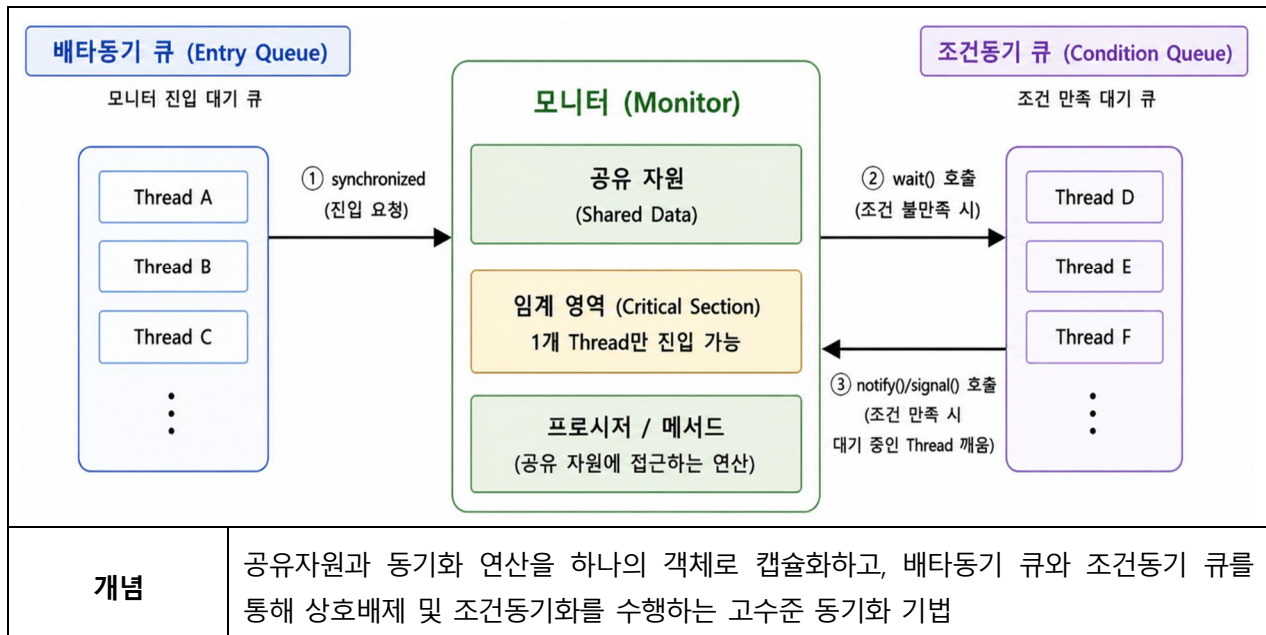
나. 세마포어 기술요소

구분	기술요소	설명
핵심 연산	P() 연산 (Wait)	자원을 요청하는 연산. 세마포어 변수(S) 를 감소시키며, $S < 0$ 이 되면 프로세스를 대기 큐로 이동.
	V() 연산 (Signal)	자원을 반납하는 연산. 세마포어 변수(S) 를 증가시키며, 대기 큐에 프로세스가 있으면 깨움(Wakeup).
	카운터 변수 (S)	가용 자원의 개수를 나타내는 원자적으로 증감하는 정수형 변수
유형	이진 세마포어	0 과 1 의 값만 가지는 세마포어로, 뮤텁스와 유사하나 소유권 개념이 없다는 차이점이 있음.
	카운터 세마포어	가용 자원의 개수가 0 이상인 정수 값을 가지며, 다중 자원을 제어할 때 사용.

- 뮤텁스와 세마포어는 개발자가 직접 Lock 을 관리해야 하므로 오류 가능성이 존재하며, 이를 추상화한 고수준 기법인 모니터 기법이 활용 가능함

4. 고수준 동기화 기법, 모니터 설명

가. 모니터의 개념



- 배타동기 큐를 통해 한 번에 하나의 스레드만 모니터 내부에 진입하도록 보장하고 조건동기 큐, 제어 언어를 통해 정밀한 스레드 간 동기화를 수행하는 고수준 제어

나. 모니터 기술요소

구분	기술요소	설명
동기화 요소	배타동기 큐 (Entry Queue)	모니터 내부 메서드를 실행하기 위해 프로세스들이 차례대로 대기하는 상호배제용 큐.
	조건동기 큐 (Condition Queue)	wait() 연산에 의해 블록된 프로세스들이 특정 조건별로 대기하는 공간.
모니터 내부 요소	공유 자원 (Shared Data)	모니터 객체 내에서 모든 스레드가 공유하는 데이터
	임계 영역 (Critical Section)	공유 자원에 접근 가능한 영역 (1 개 스레드만 진입 가능)
	프로시저 / 메서드	공유 자원에 접근하기 위한 연산(함수)로 모니터 내부에서만 실행 가능
제어 언어 요소 (JAVA 사례)	Synchronized	스레드를 배타동기 큐로 진입
	notify(), notifyAll()	조건동기 큐 스레드를 깨워 임계구역 재진입
	wait()	임계구역에서 조건동기 큐로 스레드 진입, 잠금

- Mutex 가 상호배제, Semaphore 가 자원수 제어, Monitor 가 동기화 추상화를 제공함으로써 안전하고 효율적인 동시성 제어를 지원함

"끝"

기출풀이 의견

1. 멀티스레드 환경에서 경쟁 조건(Race Condition)을 방지하기 위한 핵심 동기화 메커니즘을 묻는 문제입니다. 뮉텍스, 세마포어, 모니터의 개념과 적용 목적을 이해하고, 특히 객체지향 기반의 고수준 동기화 기법인 모니터의 내부 구조와 메커니즘을 개념도로 표현하는 것이 중요합니다.



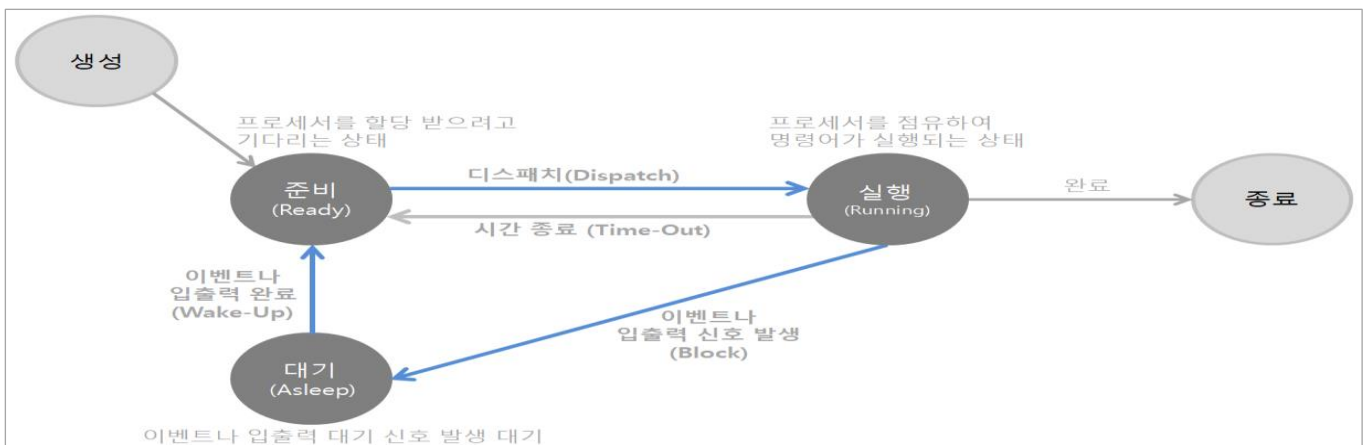
6	프로세스		
문제	운영체제의 프로세스에 대하여 다음을 설명하시오. 가. 프로세스 상태 다이어그램 나. 프로세스 제어 블록(PCB) 나. 프로세스 생성 및 종료		
도메인	운영체제	난이도	★ ★ ☆ ☆ ☆ (별 5 개 기준)
출제배경	기출문제로 출제 예상		
핵심 키워드	준비, 실행, 대기, 프로세스 식별자, 프로그램 카운터, fork, exit		
참고문헌	IT CookBook 운영체제(구현회 저) , Operating System Concepts Second Edition(홍릉과학출판사)		
출제자	조 O 연 기술사(제 117 회 정보관리기술사 / smartitpe@naver.com)		

1. 운영체제의 실행중인 프로그램, 프로세스의 개요

개념	디스크의 프로그램을 메모리에 적재(Load)하여 운영체제의 제어를 받는 실행중인 프로그램
개념도	<pre> graph TD Processor[프로세서] --- Memory[메모리] subgraph Memory Process[프로세스] end Disk[(디스크)] subgraph Disk Program[프로그램 (코드, 정적 데이터)] end Program -.-> 적재 (Loading) Process </pre>

2. 운영체제 프로세스의 상태 다이어그램, 프로세스 제어 블록 설명

가. 프로세스 상태 다이어그램

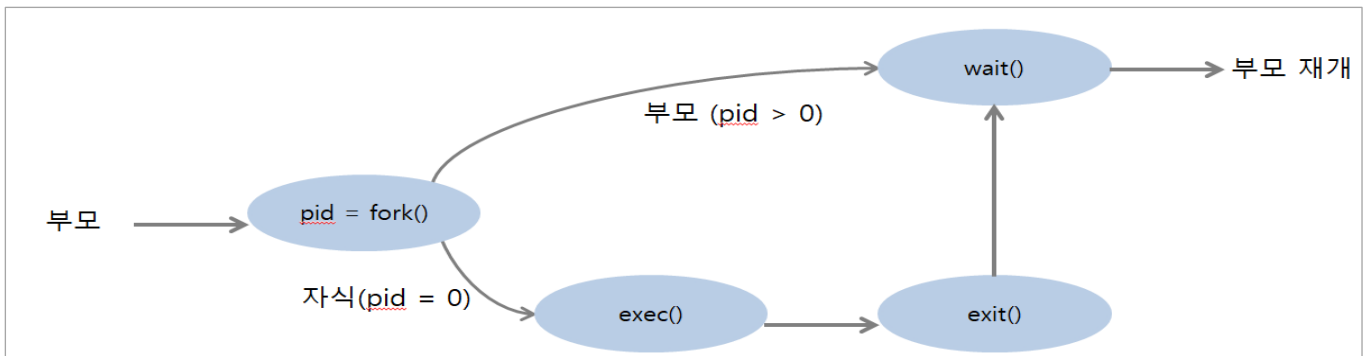


상태	내용
준비(Ready)	- 프로세스가 CPU 를 사용하여 실행할 준비가 된(이용 가능한) 상태 - 프로세스 상태 리스트(process status list) 중 준비 리스트에 들어가며 우선순위에 의해 정렬됨
	- 프로세스가 CPU 를 차지하여 실행 중인 상태, 명령어들이 실행되고 있는 상태
대기(asleep)	-대기(wait) 또는 블록(block) 상태. "I/O 동작의 완료" 등과 같이 이벤트 발생 대기 상태 -프로세스 상태 리스트 중 블록 리스트에 들어감

나. 프로세스 제어 블록 설명

개념	운영체제가 프로그램 실행을 위해 필요한 자료를 담고 있는 자료구조	
구성요소	프로세스 식별자(PID)	각 프로세스에 대한 고유 식별자(숫자, 색인항목)
	프로세스 상태	생성, 준비, 실행, 대기, 중단 등의 프로세스 상태
	프로그램 카운터	프로세스 실행을 위한 다음 명령의 주소 표시
	레지스터 저장 영역	누산기, 인덱스 레지스터, 범용 레지스터, 조건 코드 등에 관한 정보 인터럽트 발생 시 프로그램 카운터와 함께 저장, 재실행 시 원상 복귀
	스케줄링 정보	우선순위, 스케줄링 큐 포인터, 스케줄 매개변수
	계정 정보	프로세서 사용시간, 실제 사용시간, 사용 상한시간, 계정 정보 등
	입출력 상태 정보	입출력 요구 프로세스에 할당된 입출력 장치, 개방된 파일 목록
	메모리 관리 정보	메모리 영역 정의에 필요한 상한/하한 레지스터(경계 레지스터) 또는 페이지 테이블 정보

3. 프로세스 생성 및 종료 설명



절차	설명	
생성	• 프로세스는 운영체제 또는 다른 사용자의 응용 프로그램 요청에 의해 생성	
	생성 과정	• 새로운 프로세스에 프로세스 식별자(PID) 할당
		• 프로세스의 모든 구성 요소를 포함할 수 있는 주소 공간, PCB 공간 할당
		• 프로세스 제어 블록(PCB) 초기화
종료	• 링크(해당 큐에 삽입)	
	• 프로세스가 마지막 문장의 실행을 끝내고, exit() 시스템 호출을 사용하여 운영체제에게 자신의 삭제를 요청하여 종료처리	

종목	컴퓨터 시스템 응용	문제번호	2교시 5번
5. 인터럽트(Interrupt)에 대하여 다음사항을 설명하십시오. 가. 인터럽트 발생 요인(하드웨어 및 소프트웨어) 나. 마스크블(Maskable) 인터럽트와 논마스크블(Non-maskable) 인터럽트 다. 벡터(Vectored) 인터럽트의 동작원리 라. 우선순위(Priority)를 보장하는 다중(Multiple) 인터럽트가 발생할 경우 이를 처리하기 위한 ISR(Interrupt Service Routine)을 프로세서 및 소프트웨어에서 제공하여야 하는 기능 요소들과 상호 연관시켜 설명하십시오.			
출제영역	CA/OS	난이도	상
참고문헌	https://m.blog.naver.com/PostView.nhn?blogId=scw0531&logNo=220650635893&proxyReferer=https%3A%2F%2Fwww.google.com%2F		
주요 키워드	외부 인터럽트, 내부 인터럽트, 프로그램 검사 인터럽트, Supervisor Call 인터럽트, Maskable, Non-Maskable, 인터럽트 신호선, 인터럽트 번호, IVT, ISR, 스택, 허용 플래그		
답안전략/의견	질문한 사항이 많으므로 각 질문에 대하여 한 단락씩 구성하는 것이 좋음. (인터럽트 개요 부분은 시간 관계상 작성하지 않아도 됨)		
작성자	김영란 PE / young_nan@hanmail.net		

I. 인터럽트의 개요

가. 인터럽트의 개념

- 컴퓨터 시스템의 외부, 내부적인 원인으로 CPU가 처리하던 프로그램을 중단하고 컴퓨터 제어를 특수 사건을 처리할 수 있도록 보내는 제어신호.

나. 인터럽트의 종류

구분	내용	특징
발생원인에 따른 분류	하드웨어 인터럽트	- 전원 오류, 기계 착오 등 인터럽트의 원인이 하드웨어에 있음.
	소프트웨어 인터럽트	- Overflow, Divide by Zero 등 인터럽트의 원인이 소프트웨어에 있음.
차단 가능성에 따른 분류	Maskable	- 인터럽트의 요청을 받아들이지 않고 무시할 수 있는 인터럽트
	Non-Maskable	- 인터럽트의 요청을 무시할 수 없는 인터럽트
인터럽트 조사 방식에 따른 분류	Polling (소프트웨어 방식)	- 인터럽트 발생시 우선순위가 가장 높은 인터럽트 자원(Source)부터 인터럽트 요청 플래그를 차례로 검사하여 찾고 이에 해당하는 인터럽트 서비스 루틴을 수행하는 방식
	Vector (하드웨어 방식)	- CPU와 인터럽트를 요청할 수 있는 장치 사이에 장치 번호에 해당하는 버스를 직렬/병렬로 연결하여 요청 장치의 번호를 CPU에 알리는 방식. - Daisy chain, Parrel

II. 인터럽트의 하드웨어 및 소프트웨어적 발생 요인

구분	내용	특징
하드웨어 인터럽트	외부인터럽트	-전원이상, 기계착오, 외부신호, 입출력
	내부인터럽트	- Overflow, UnderFlow, 0으로 나누기, 명령어 오류
소프트웨어 인터럽트	프로그램 검사 인터럽트	-수행중인 프로세스의 잘못된 명령에 의해 발생 불법적인 Instruction 실행 (Overflow, Stack Overflow, Devide By Zero 등) -보호된 Memory에 접근 시도
	Supervisor Call 인터럽트	-Supervisor Call 호출 -복잡한 입/출력. 기억장치 할당

- 종류별 우선순위 : 외부인터럽트 > 내부인터럽트 > SW인터럽트 (전원이상 > 기계착오 > 외부신호 > 입출력 > 내부 > SW인터럽트)

III. Maskable 인터럽트와 Non-Maskable 인터럽트

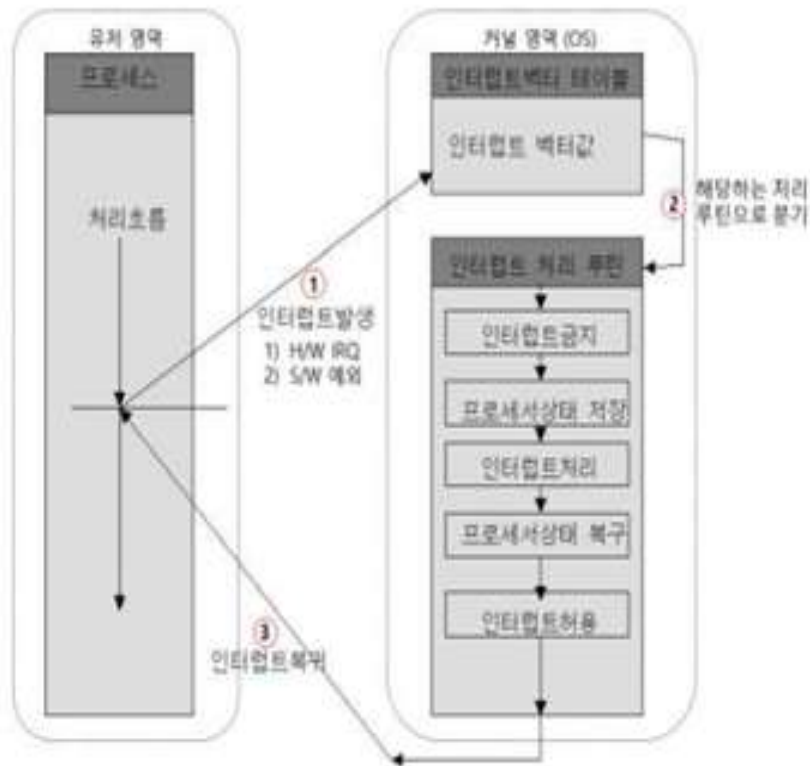
구분	Maskable Interrupt	Non-Maskable Interrupt(NMI)
정의	- 인터럽트의 요청을 받아들이지 않고 무시할 수 있는 인터럽트	- 인터럽트 요청 시 무조건 바로 처리해야 하는 인터럽트
유형	- 하드웨어 인터럽트	- 하드웨어 인터럽트
우선순위	- 우선순위 낮음	- 우선 순위 높음
처리 시점	- 지연 처리 또는 처리하지 않음	- 즉시 처리
처리 방식	- Vectored or non vectored	- All Vectored
예시	- RST 6.5, RST 7.5, RST 5.5 of 8085 microprocessor	- Trap of 8085 microprocessor

IV. 벡터 인터럽트의 동작 원리

가. 벡터 인터럽트의 개념

구분	설명	
정의	- CPU와 인터럽트를 요청할 수 있는 장치 사이에 장치 번호에 해당하는 버스를 직렬/병렬로 연결하여 요청 장치의 번호를 CPU에 알리는 방식.	
유형	- Daisy Chain, Parrel 방식(다중 회선 방식)	
구성 요소	인터럽트 신호선 (INTR, INTA)	- 디바이스로부터 인터럽트 신호를 감지하는 하드웨어 선.
	인터럽트 번호	- 각 인터럽트 신호선마다 고유의 번호가 할당됨
	IVT	- Interrupt Vector Table, 인터럽트 신호 번호에 따라 수행할 ISR를 매핑 시켜놓은 테이블
	ISR	- Interrupt Service Routine, 인터럽트 발생시 수행할 동작을 정의해 놓은 루틴

나. 벡터 인터럽트의 동작 원리



절차	세부 동작 절차	설 명
인터럽트 발생	하드웨어	- 외부 인터럽트, 내부 인터럽트
	소프트웨어	- SVC, 프로그램 검사
IVT	인터럽트 벡터 조회	- 인터럽트 서비스 루틴 검색
	ISR로 분기	- 해당 ISR로 분기
ISR	인터럽트 금지	- Locking
	프로세스 상태 저장	- Context Switching
	인터럽트 처리	- 인터럽트 요청 작업 수행
	프로세스 상태 복구	- Context Switching
	인터럽트 허용	- Unlock, 인터럽트 루틴 종료

V. 우선 순위를 보장하는 다중(Multiple) 인터럽트를 처리하기 위한 ISR

구분	설명	
개념	- 인터럽트의 우선 순위를 정하고, 우선 순위가 낮은 인터럽트가 처리되고 있는 동안 우선순위가 더 높은 인터럽트가 들어오면, 현재의 인터럽트 서비스 루틴의 수행을 중단하고 새로운 인터럽트를 처리하는 방식	
조건	우선순위	- 인터럽트간 우선순위를 정의함
	스택	- ISR 전용 스택이 존재함
	허용 플래그	- CPU내에서 인터럽트 허용/불가 플래그 설정.

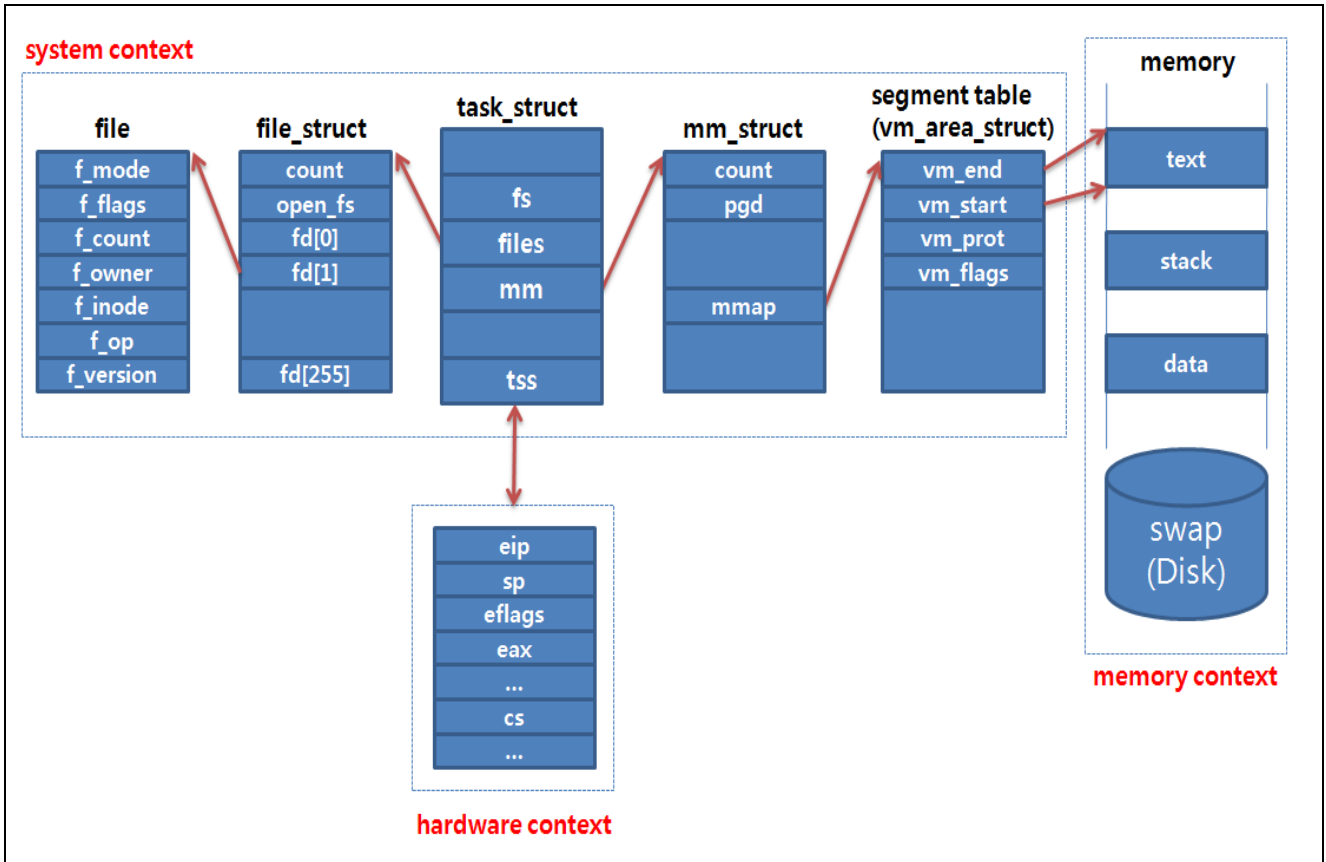
구분	설명
개념도	Main Program 우선순위 Network > Device 1. Device-A의 ISR를 처리하는 동안 Network 인터럽트가 발생함. 2. 네트워크 우선순위가 Device-A의 우선순위보다 높으므로 Device-A ISR 실행을 중단하고 네트워크 ISR로 분기하여 실행 3. 네트워크 ISR실행이 완료되면 다시 Device-A ISR을 실행함

"11"

종 목	문 제
컴퓨터시스템응용기술사	4교시 2번
운영체제에서 문맥(context)의 내용, 문맥교환(context switch)의 과정 및 문맥교환이 발생하는 조건을 설명하시오.	
도메인/토픽	운영체제 > 문맥교환(Context Switching)
키워드	System context, Memory context, Hardware context Kernel mode, User mode, PCB, Context Save, Context Restore Dispatch, Time run out, Block, Time slice, I/O, System call
풀이 가이드	1. 프로세스 자원과 제어 흐름의 집합, 문맥의 내용 가. 문맥의 유형 -- 시스템, 메모리, 하드웨어 문맥 나. 문맥의 내용 -- Linux의 시스템 문맥 기준 2. 문맥 교환의 과정 가. 문맥 교환의 과정 -- 문맥 교환 과정 도식화 나. 문맥 교환의 과정 설명 -- 커널모드 전환 → Context Save → Context Restore → 사용자모드 전환 3. 문맥 교환이 발생하는 조건 가. 문맥 교환이 발생하는 조건 -- 프로세스 상태 전이도로 표현 나. 문맥 교환이 발생하는 조건 설명 -- dispatch, time slice만료, I/O 작업, System call
필수항목 /연관토픽	Process, Thread, Kernel, PCB, Dispatcher
참고문헌	108회 정보관리/컴퓨터시스템응용기술사 기출풀이, 107회 기술사회, 2016. 2. 27 Virtual Memory:Systems, Carnegie Mellon, 2010. 10. 19 Linux Processes, TLEP, 1999. 1 (http://www.tldp.org/LDP/tlk/kernel/processes.html) TASK관리, 블로그, 2012. 6. 21 (http://mintnlatte.tistory.com/400) Process Context, 신라대학교 컴퓨터공학과, 2008 (slideplayer.com/slide/4842723/)
Advice	Fact를 묻는 문제이기 때문에 문제에 집중한 목차 구성에 기반하여 상세한 내용 작성 시, 고득점이 예상됩니다. 프로세스 상태 전이 및 문맥 교환 관련 문제는 지속적으로 출제 중이므로 반드시 학습이 필요합니다.
작성자	108회 컴퓨터시스템응용기술사 / 김인수 (diun@paran.com)

1. 프로세스 자원과 제어 흐름의 집합, 문맥(Context)의 내용

가. 문맥(Context)의 유형



- 문맥(Context)이란, 여러 프로세스가 함께 수행되는 시분할 시스템 환경에서 특정 프로세스의 처리를 위해 할당된 자원이 일시적으로 반환되어 다른 선점 프로세스를 처리하고 재 할당 되었을 경우(이를 문맥교환이라 함), 기존 수행 중이었던 작업 시점부터 재현하기 위한 정보를 지칭
- 문맥교환(Context Switching) 발생 시, 이전 프로세스 상태를 재현할 수 있도록 지원하는 정보

시스템 문맥	(System Context) - 커널에 할당된 자료구조 (task_struct, 파일 디스크립터, 세그먼트, 페이지 테이블)
메모리 문맥	(Memory Context) - Text, Data, Stack, Heap 영역, swap 공간
하드웨어문맥	(Hardware Context) - 문맥 교환 시, 현재 실행 위치에 대한 정보를 유지

- 프로세스의 문맥(Context)을 크게 세가지로 분류하면, [시스템 문맥], [메모리 문맥], [하드웨어 문맥]으로 구분 가능

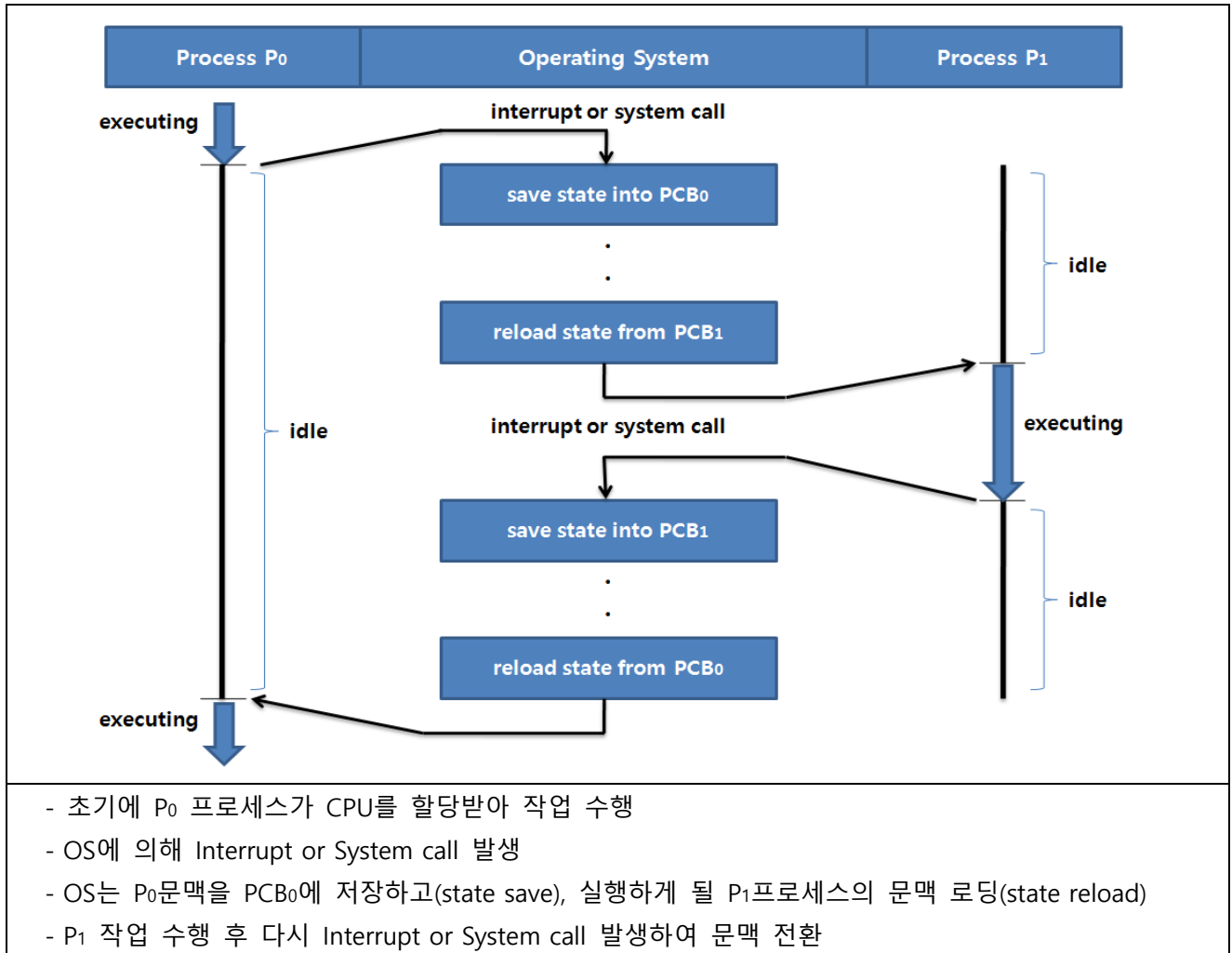
나. 문맥(Context)의 내용 - 시스템 문맥 기준 (/include/linux/sched.h)

구분	설명	
Task identification	- 태스크를 인식하기 위한 변수	
	pid	- 태스크 ID
	tgid	- 태스크가 속해있는 쓰레드 그룹 ID
	사용 접근제어필드	- uid(사용자id), euid(유효 사용자id), suid(저장된 사용자id), fsuid(파일시스템 사용자id)
	그룹 접근제어필드	- gid, egid, sgid, fsgid
State	- 생성에서 소멸까지 상태 관리를 위한 변수	
	- TASK_RUNNING(0), INTERRUPTIBLE(1), UNINTERRUPTIBLE(2), STOPPED(4), TRACED(8), EXIT_ZOMBIE(16), DEAD(32)	
Task relationship	- 태스크는 생성되면서 가족 관계를 가짐	
	real_parent	- 현재 태스크를 생성한 부모 태스크의 task_struct 구조체
	parent	- 현재 부모 태스크의 task_struct 구조체
	children / sibling	- 자식/형제를 리스트로 연결한 뒤, 리스트의 헤드를 저장
Scheduling information	- 태스크에게 비동기적인 사건의 발생을 알리는 메커니즘	
	- signal, sighand, blocked, pending	
Memory information	- 명령어와 데이터를 text, data, stack, heap 공간 등에 저장.	
	- task_struct 공간에 대한 위치, 크기, 접근제어 정보 등을 관리하는 변수 존재	
File information	- mm_struct	
	files	- 태스크가 오픈한 파일 접근 (files_struct 구조체)
Thread structure	fs	- 루트디렉토리의 inode와 현재 디렉토리의 inode 접근(fs_struct)
	- 문맥 교환을 수행할 때 태스크가 현재 어디까지 실행되었는지 기억하는 공간	
Time information	- TSS (Task State Segment)	
	- 태스크의 시간 정보를 위한 변수	
	start_time	- 태스크가 시작된 시간
	cpu_time	- 사용한 cpu시간
Format	run_delay	- 대기 시간
	- 리눅스는 Linux exec 도메인 뿐 아니라 BSD, SVR4 exec 도메인도 지원	
	personality	- 커널에서 컴파일된 프로그램도 재컴파일 없이 수행 위한 변수
Resource limits	binfmt	- 다양한 binary format을 지원 (a.out, elf, java, shell script)
	- 태스크가 사용할 수 있는 자원의 한계. 배열로 구현	
	rlim_max	- 최대 허용 자원 수
Resource limits	rlim_cur	- 현재 설정된 허용 자원 수

- 프로세스 실행이 전환될 때에 문맥 교환(Context Switching) 발생

2. 문맥 교환(Context Switching)의 과정

가. 문맥 교환(Context Switching)의 과정 도식



- 초기에 P₀ 프로세스가 CPU를 할당받아 작업 수행
- OS에 의해 Interrupt or System call 발생
- OS는 P₀문맥을 PCB₀에 저장하고(state save), 실행하게 될 P₁프로세스의 문맥 로딩(state reload)
- P₁ 작업 수행 후 다시 Interrupt or System call 발생하여 문맥 전환
- 문맥교환(Context Switching)은 하나의 프로세스가 CPU를 사용중인 상태에서 다른 프로세스가 CPU를 사용하도록 하기 위해 이전의 프로세스 상태를 보관하고, 새로운 프로세스의 상태를 적재하는 일련의 작업 과정

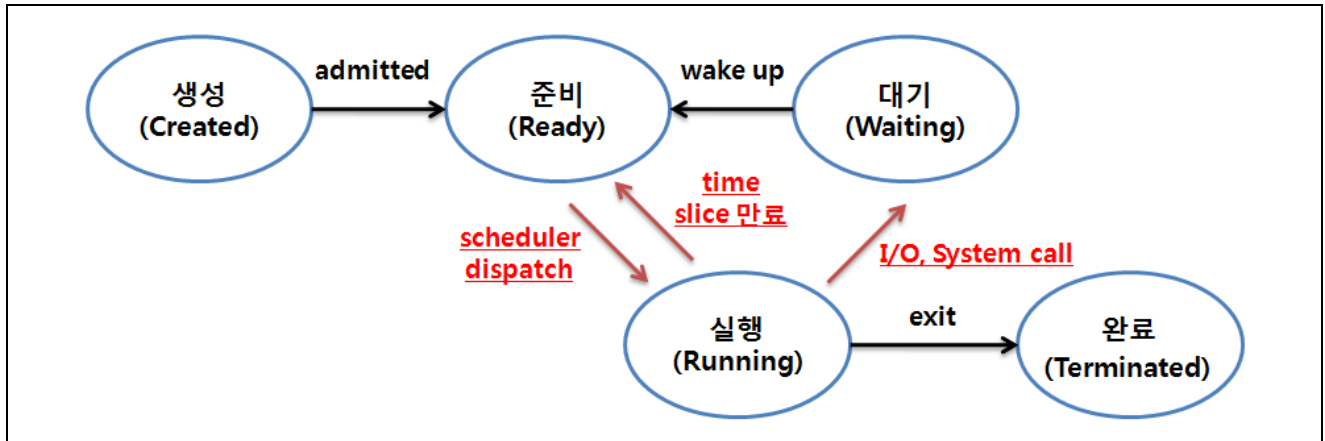
나. 문맥 교환(Context Switching)의 과정 설명

과정	내용
1. 문맥교환 요구	- 인터럽트나 시스템 호출에 의한 문맥 교환 요구
2. 커널 모드 전환	- 사용자 모드 → 커널 모드
3. Context Save	- 기존 프로세스의 현재 상태 정보를 PCB에 저장
4. Context Restore	- 다음에 실행할 프로세스의 상태 정보를 PCB에서 복원
5. 사용자 모드 전환	- 커널 모드 → 사용자 모드

- 대상 프로세스의 정보를 PCB 통해 저장/복원하는 과정 통해 문맥 교환 수행

3. 문맥 교환(Context Switching)이 발생하는 조건

가. 문맥 교환(Context Switching)이 발생하는 조건 - 프로세스 상태 전이도



- Ready → Running, Running → Ready, Running → Waiting 전환 시에 문맥 교환 발생

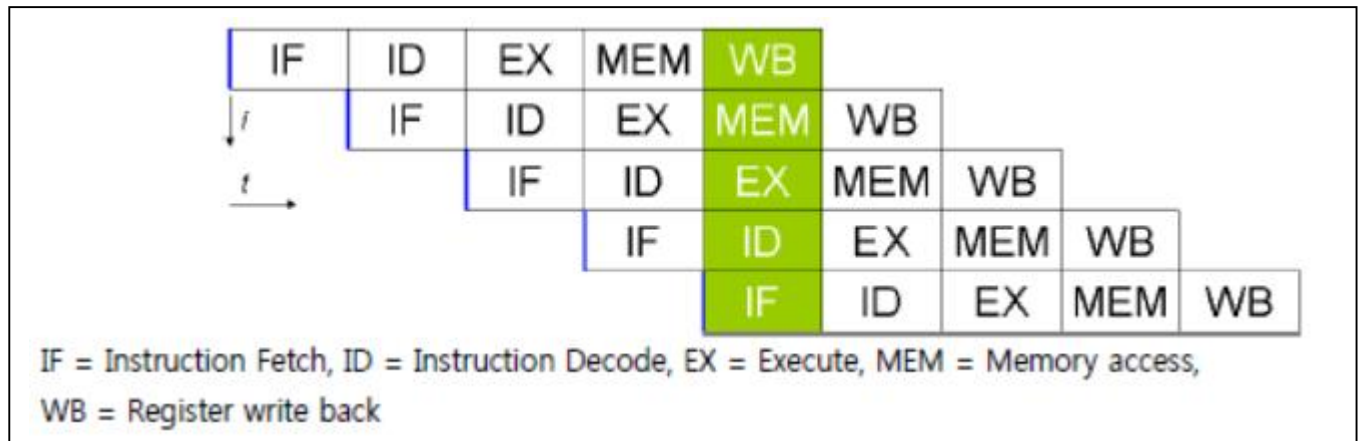
나. 문맥 교환(Context Switching)이 발생하는 조건 설명

발생 조건	발생 조건 설명	상태 전이 과정
scheduler dispatch	- Ready Queue의 첫 번째 프로세스(가장 높은 우선순위)는 CPU를 할당 받고 실행 상태로 전이	- Ready → Running
time slice 만료	- 프로세스가 CPU 사용 가능한 할당 시간 초과 - 시간이 초과되면 PCB저장, CPU 양도 후 준비상태로 전이	- Running → Ready
I/O 작업	- 실행 중인 프로세스가 I/O로 인해 I/O 전용 프로세스에게 CPU 양도 후 대기상태로 전이 - I/O 처리는 CPU가 아니라 I/O 전용 프로세스가 담당하기 때문에 발생	- Running → Waiting
System call	- 실행될 준비가 되어 있지 않은 서비스를 호출할 경우 대기상태로 전이	- Running → Waiting

- 문맥 교환 시, 문맥 교환에 의한 오버헤드 발생
- 오버헤드를 해결하기 위해 다중 프로그래밍의 정도를 낮추어 문맥 교환 발생 최소화 및 Thread 이용한 문맥 교환을 수행하여 데이터 전환 최소화

8	CPU 파이프라인		
문제	CPU 파이프라인과 관련하여 아래 사항에 대하여 설명하시오. 가. CPU 명령어 처리의 파이프라인 방식과 비파이프라인 방식의 CPU 처리 시간 비교 (파이프라인 단계 수를 m, 각 파이프라인 단계에서의 처리시간을 k, 실행할 명령어들의 수를 n이라 함) 나. 명령어 파이프라이닝 해저드 3 가지 유형과 해결방안		
도메인	CA/OS	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제배경	CPU 관련 기본 토픽에 대해 수식화를 통한 비교가 가능한지 점검 필요. 기출된 문제들에 대한 답안 구성 능력 및 시간 관리 능력 검토 필요.		
핵심 키워드	구조, 데이터, 제어 해저드		
참고문헌	KPC 기출문제 및 모의고사 풀이집, 기술사 서브노트		
출제자	표기수 기술사(제 117 회 정보관리기술사 / kisu.pyo@gmail.com)		

1. CPU 명령어 처리의 파이프라인 방식 개념



- 명령어 실행 사이클의 각 단계를 중첩시켜 전체 프로그램 수행 시간을 단축시키고, 명령어 처리율을 향상시키는 기법
- 파이프라인 프로세서에서 명령어 의존성(구조, 데이터, 제어)을 발생시킬 수 있는 문제를 파이프 해저드라 함

2. 파이프라인 방식과 비파이프라인 방식의 CPU 처리 시간 비교

가. 파이프라인 방식과 비파이프라인 방식의 명령어 처리 시간

파이프라인 방식	동작방식	처리시간	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	
		명령어1	IF	ID	EX	WB													
		명령어2		IF	ID	EX	WB												
		명령어3			IF	ID	EX	WB											
		명령어4				IF	ID	EX	WB										
	- 각 명령어의 첫번째 작업을 마친 후 이어서 다음 명령어의 첫번째 단계가 병렬로 처리																		
측정 공식	- 각 명령어의 작업 단계를 4개라 가정 (m=4)																		

4교시

	도출	- 각 명령어의 작업 단계 처리 시간 1ms라 가정 ($k=1$) - 총 명령어 개수가 4개라 가정 ($n=4$) - 위와 같은 경우 CPU 처리 시간이 총 7ms 소요 - 첫번째 명령어 처리 시간 과정 중 몇 개의 명령어가 동시에 처리되는지 표현됨																																																																																					
	측정 공식	- $T = mk + (n-1)k$																																																																																					
비파이프라인 방식	동작방식	<table><tr><td>처리시간</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>13</td><td>14</td><td>15</td><td>16</td></tr><tr><td>명령어1</td><td>IF</td><td>ID</td><td>EX</td><td>WB</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>명령어2</td><td></td><td></td><td></td><td></td><td>IF</td><td>ID</td><td>EX</td><td>WB</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>명령어3</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>IF</td><td>ID</td><td>EX</td><td>WB</td><td></td><td></td><td></td><td></td></tr><tr><td>명령어4</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td>IF</td><td>ID</td><td>EX</td><td>WB</td></tr></table> - 하나의 명령어가 끝난 후 다음 명령어를 실행	처리시간	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	명령어1	IF	ID	EX	WB													명령어2					IF	ID	EX	WB									명령어3									IF	ID	EX	WB					명령어4													IF	ID	EX	WB
	처리시간	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16																																																																						
	명령어1	IF	ID	EX	WB																																																																																		
명령어2					IF	ID	EX	WB																																																																															
명령어3									IF	ID	EX	WB																																																																											
명령어4													IF	ID	EX	WB																																																																							
	측정 공식 도출	- 각 명령어의 작업 단계를 4개라 가정 ($m=4$) - 각 명령어의 작업 단계 처리 시간 1ms라 가정 ($k=1$) - 총 명령어 개수가 4개라 가정 ($n=4$) - 위와 같은 경우 CPU 처리 시간이 총 16ms 소요 - 처리 시간이 각 변수 간의 곱으로 표현됨																																																																																					
	측정 공식	- $T = mkn$																																																																																					

- 파이프라인 방식과 비파이프라인 방식의 처리속도 측정에 따라 성능 향상을 파악 필요

나. 파이프라인과 비파이프라인 처리시간 비교를 통한 성능 향상율

성능 향상율	$\text{속도 향상율} = \frac{mkn}{mk+(n-1)k} \times 100\%$		
	- 파이프라인 사용시 속도 향상도 = $100\% \times (\text{비파이프라인 사용처리 시간}) / (\text{파이프라인 사용 처리 시간})$		
명령어 개수	파이프라인 방식	비파이프라인 방식	성능 향상율
10개	$4 * 1 + (10-1) * 1 = 13$	$10 * 4 * 1 = 40$	308%
100개	$4 * 1 + (100-1) * 1 = 103$	$100 * 4 * 1 = 400$	388%
1000개	$4 * 1 + (1000-1) * 1 = 1003$	$1000 * 4 * 1 = 4000$	399%

- $m=4, k=1$ 인 경우 위와 같은 사례 도출 가능

- m (단계), n (명령어 개수)이 커질수록 성능 향상율이 더욱 증가함

3. 명령어 파이프라이닝 해저드 3가지 유형 및 해결방안

가. 명령어 파이프라이닝 해저드 3가지 유형

유형	발생 원인	설명
구조적 해저드	하드웨어적 제한	- 각각의 다른 명령어가 동일 메모리, 동일 레지스터, ALU 등을 동시에 접근하려 할 때 후속 명령어가 지연되는 현상
데이터 해저드	이전 명령의 값에 종속	- 명령의 값이 현재 파이프라인에서 수행중인 이전 명령의 값에 종속된 경우, 이전 명령의 결과를 기다려야 함 - RAW(Read After Write), WAR(Write After Read), WAW(Write

		After Write) 유형 발생
제어 해저드	분기 명령	- 분기(JUMP, BRANCH 등) 명령어에 의해 발생하는 현상 - 분기가 결정된 시점에 수행되지 않을 명령이 파이프라인에 존재하는 경우 발생

- 각 유형의 파이프라인 해저드에 대한 해결방안 필요

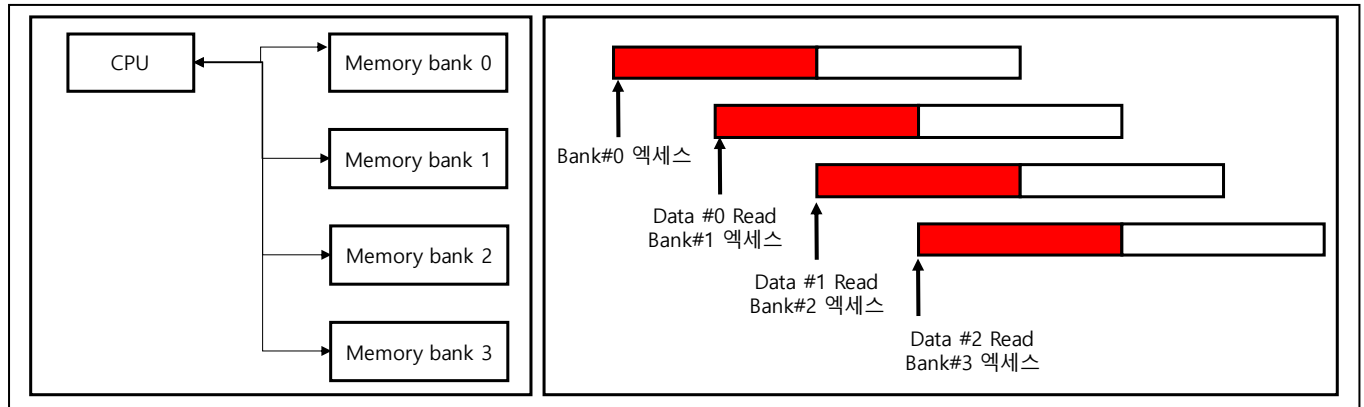
나. 파이프라이닝 해저드 해결방안

유형	해결 방안	설명
구조적 해저드	Adder 추가	- IF(Instruction Fetch) 단계의 PC 값 증가는 ALU 내의 Adder가 아닌 별도의 Adder 추가를 통해 해결
	Cache 메모리	- IF 단계와 MEM(Memory Access) 단계의 메모리 접근 충돌은 별도의 Cache 메모리를 두어 해결
	레지스터 사용	- ID(Instruction Decode)와 WB(Write Back)단계의 레지스터 접근 충돌은 시분할 Read/Write 접근 또는 멀티 포트 레지스터의 사용으로 해결
데이터 해저드	Bypassing (Forwarding)	- ALU의 출력에 MUX를 추가하여 출력 값을 그대로 다음 파이프라인의 ALU 연산 입력으로 사용하도록 Bypassing 시킴
	컴파일러 (명령어 재배치)	- 명령어의 의존성이 존재할 경우 컴파일러에 의해 명령어 순서를 재배치하여 데이터 해저드 해결
	Interlocking / NOP	- 인위적으로 Rollback하거나 하위 파이프라인 실행을 중단, 또는 NOP(No Operation) 명령을 추가하여 해저드 회피
제어 해저드	Predict-Not-Taken	- Branch 명령어 뒤에 Delay slot을 추가하여 실행. Branch가 실행될 경우 Delay slot의 Flush발생으로 충돌 예방
	Branch Target buffer	- 과거 History를 활용하여 분기를 예측하기 위한 2bit버퍼를 활용, 4개의 상태를 유지해서 분기를 예측
	Software Pipelining	- 반복적으로 수행하는 명령어를 순차적으로 수행되도록 재배치하여 수행

- Stall or Bubble Cycle 추가로 해저드 유형의 공통 해결방안이 될 수 있음

2	메모리 인터리빙		
문제	메모리 인터리빙의 유형과 액세스 활용방식에 대해 설명하시오		
도메인	CA / OS	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제의도	컴퓨터 구조의 기본토픽이 정보관리에 자주 출제됨에 따라, 미출제영역에 대한 출제		
핵심 내용 키워드	MAR (memory address register), MDR (memory data register), MBR (memory buffer register) C-Access, S-Access, 순차적 접근, 상위/하위/혼합		
목차예시	1. 메모리의 효율적 사용을 위한, 메모리 인터리빙의 개요 2. 메모리 인터리빙의 유형 3. 메모리 인터리빙의 액세스 활용방식 4. 메모리 인터리빙의 활용사례		
채점 점수 가이드	아래 점수 가이드를 참고로 기술사님의 채점 점수 가이드를 제시하여 주세요. ① 개념, 이해 부족 (1~2점) ② 기본 이해 수준 (3~5점) ③ 정확한 기술 제시 (5~6점) ④ 추가요소 설명(+α)		
참고문헌	KPC 108 회 대비 합숙해설집_3 일차_2 교시_관리 (강희석 PE) https://ko.wikipedia.org/wiki/메모리_인터리빙 http://www.kbench.com/?q=node/8417		
고득점전략 및 답안작성 가이드	상위, 하위, 혼합 인터리빙의 구조 및 메모리 액세스 순서에 대한 표현을 한다면 고득점 가능		
출제자	김유성 기술사(제 113 회 정보관리기술사 / yousungkim1216@gmail.com)		

1. 메모리의 효율적 사용을 위한, 메모리 인터리빙의 개요



(개념) 메모리를 복수개의 뱅크로 나누고, 각 모듈에 연속적인 주소를 부여하여 동시 접근 가능 및 클럭 사이클을 줄이면서 물리적 기능추가 없이 메모리 접근시간을 최소화 하기 위한 기법

2. 메모리 인터리빙의 유형

유형	동작 구성도	설명
상위		- 상위비트로 모듈선택 - 하위비트로 메모리 주소지정 - SPOF 극복가능 - 기억장치 주소를 모듈에 순차적으로 지정 - C-Access
하위		- 상위비트로 메모리 주소지정 - 하위비트로 모듈선택 - SPOF 문제발생 - 기억장치 주소가 뱅크 단위로 인터리빙 - S-Access
혼합		- 뱅크의 선택 bit는 상위 bit - 기억장치 모듈을 뱅크로 묶음 - 모듈 선택은 하위bit로 구성 (각 그룹내에서 하위 인터리빙)

3. 메모리 인터리빙의 액세스 활용방식

가. C-Access (Concurrent-access)

구분	설명
시간 흐름도	The diagram shows four modules (모듈 0 to 3) on the left. Each module has a horizontal bar representing its access time T_a. These bars are staggered in time. Below the modules, a timeline shows data transfer points d_0, d_1, d_2, d_3 with intervals t_b between them. A vertical dashed line marks the '완료 시간' (Completion Time).
엑세스 시간	$T = T_a + (M \times t_b)$ $* T_a = \text{기억장치 액세스 시간}, t_b = \text{버스 클럭의 주기}, M = \text{기억장치 모듈의 수}$
동작방식 특징	- 주소가 순차적으로 도착 - 순차적인 Data Read - 순차적인 CPU 전송
장점	- 별도 장치 없이 구현가능. 단순한 구현
단점	- CPU-메모리간의 버스경합으로 일정간격으로 주소 도착이 어려움

- 순차적으로 처리하며, 구현이 단순한 것이 장점

나. S-Access (Simultaneous Access)

구분	설명
시간 흐름도	The diagram shows modules 0, 1, ..., M-1. All modules have access time bars T_a that start at the same time. Below, the data transfer timeline shows points $d_0, d_1, \dots, d_{M-1}$ with intervals t_b. A vertical dashed line marks the completion time.
엑세스 시간	$T = T_a + (M \times t_b)$ $* T_a = \text{기억장치 액세스 시간}, t_b = \text{버스 클럭의 주기}, M = \text{기억장치 모듈의 수}$
동작방식 특징	- 주소가 모두 도착하면 동시에 Data Read - 읽은 Data는 순차적으로 CPU 전송
장점	- 동시 액세스 보장, 연속적 전송가능(Burst 버스 모드)

단점

- 필요 데이터 수가 동시 액세스 가능한 수 초과시 비효율적

- 주소 버스 경합에 따른 문제점 해결가능

4. 메모리 인터리빙의 활용사례

활용사례	세부특징	상세설명
Pipeline 컴퓨팅	- 연속 Data 흐름	- 다수의 CPU 활용시 메모리 병렬 접근 수행 - S-Access
SPOF 문제극복	- 단일 고장점 극복	- 메모리 순차접근으로 에러의 전파영향 없음 - 상위, 혼합 인터리빙
DDR DRAM	- SDRAM Refresh	- Read의 동시성으로 성능보장

- 캐시 기억장치(Line 단위로 Burst 전송), 고속 DMA 전송(Burst) 등에서 많이 사용

“끝”

5	폰 노이만 아키텍처 / 하버드 아키텍처		
문제	폰 노이만 아키텍처와 하버드 아키텍처에 대해 설명하고 폰 노이만 아키텍처의 문제점 및 해결방안에 대해 설명하시오		
도메인	CA / OS	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제배경	컴퓨터 아키텍처의 기본이 되는 토픽으로 반드시 숙지해야 함.		
핵심 키워드	단일메모리/메모리분리, stored program / Data, Instruction Memory		
참고문헌	101 회 컴퓨터시스템응용 기출풀이		
출제자	이형석 기술사(제 108 회 컴퓨터시스템응용기술사 / ddrseok@gmail.com)		

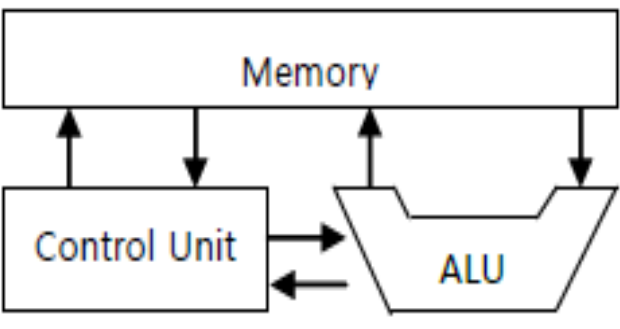
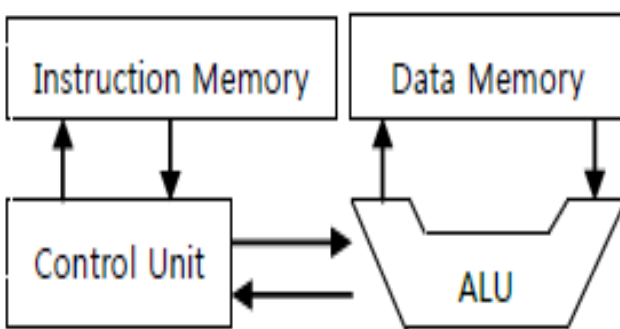
1. 폰 노이만 구조와(Von Neumann Architecture) 하버드 구조(Harvard Architecture)의 개념

폰 노이만 아키텍처	하버드 아키텍처
단일 저장장치(메모리)에 연산의 수행과 관련된 일련의 명령어와 연산에 필요한 또는 연산의 결과로 생성된 데이터를 함께 수용하는 Computing machine 모델	폰 노이만의 메모리 접근 병목현상을 개선하기 위해, 명령어와 데이터 메모리가 분리되어 병렬적으로 작업이 처리되도록 구현한 컴퓨터 아키텍처

- 폰 노이만은 Stored Program concept 컴퓨터라고 부르며 모든 명령어는 반드시 실행 이전에 메모리에 불러 들어야 하며, CPU는 한번에 단 하나의 명령어만 실행 가능한 컴퓨터 구조(Flynn의 분류)

2. Von Neumann 과 Harvard 아키텍처의 비교

가. 구조적 특징 비교

폰 노이만 아키텍처	하버드 아키텍처
	
- 메모리에서 명령어를 Fetch(메모리) -> Decode(cu) -> Execute(ALU) -> Store(메모리) 하는 구조 - 데이터와 명령어가 메모리 및 버스를 공용으로 사용하여 파이프라인 사용시 문제발생	- 파이프라인 구현을 위해 메모리를 분리 - 명령어는 Instruction Memory 에 저장 - 데이터는 Data Memory 에 분리해서 저장 - 동시에 명령어 해석(Execute)와 저장(Store)이 가능한 구조

나. 기능적 특징 비교

구분	폰 노이만 아키텍처	하버드 아키텍처
----	------------	----------

목적	CPU는 한번에 단일 명령어 실행가능	병렬처리를 위해 메모리를 분리
메모리	하나의 메모리 공유	명령어, 데이터 메모리 분리
프로세스	- 메모리 -> FI -> 메모리 -> DI -> 메모리 -> EI -> Store(메모리) - 순차적으로 수행됨	- 명령어 메모리 -> FI -> Store -> 데이터 메모리 - 동시에 명령어와 데이터 처리가능
장점	공용 메모리 사용으로 상대적 구현 비용 저렴	파이프라이닝 기술을 사용을 위한 환경 제공
단점	파이프라이닝 기술 사용시 메모리 공유문제 발생(제어 헤저드 발생)	- 별도 메모리 사용으로 비용 증가 - 회로 구조가 복잡

3. 폰 노이만 아키텍처의 문제점과 해결방안

가. 폰 노이만 아키텍처의 문제점

문제점	설명
CPU의 비효율적 활용	한번에 하나의 명령어 처리로 상대적으로 빠른 속도의 CPU를 효율적으로 사용하지 못함
메모리 버스 병목현상 발생	명령어와 데이터를 모두 하나의 주기억장치에 저장하기 때문

나. 폰 노이만 아키텍처 문제점의 해결방안

해결방안	설명
병렬처리 개념 도입	- 다중 프로세서/다중 데이터 스트림 처리가 가능한 병렬처리 컴퓨터 적용 - 명령어 병렬처리 기술 (Pipeline, Super scalar 등) 적용 - SMP(Shared Memory Processor), MPP(Massively Parallel Processor) 형태의 병렬 처리 - 하나의 Processor에서 여러 개의 Thread를 지원하는 Multi-threading 및 동시에 여러 개의 Thread를 처리 가능한 Hyper threading 기술 출현
주기억장치 병목 해결	- Bus를 Instruction 용과 Data 용으로 분리(Harvard 아키텍처) - CPU에 Memory Controller를 내장하는 방식 도입 - CPU와 주기억장치 사이에 고속의 Memory를 추가한 계층적 구조로 구성(Cache Memory) - 최근에는 Bus를 읽기용과 쓰기용으로 좀 더 세분화 하는 Hyper transport 기술 출현

4. 폰 노이만 아키텍처와 하버드 아키텍처의 적용 사례 및 적용시 특징

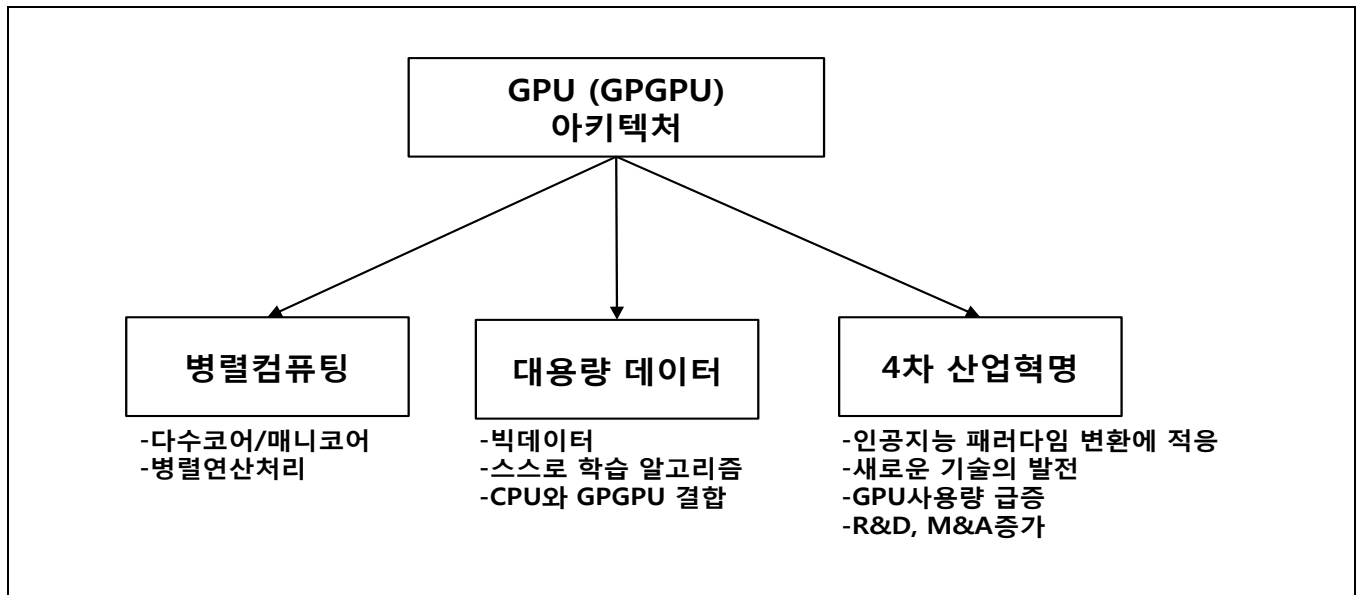
구분	폰 노이만 아키텍처	하버드 아키텍처
적용사례	일반적인 범용 CPU	Microchip Technology 의 PIC Atmel AVR 현재 범용 CPU(Intel 펜티엄 이후)
적용시 특징	Instruction이 전기적인 신호에 의해 변경이 가능한 메모리에 있으므로	- 프로그램 중 데이터와 혼합 된 경우 메모리로부터 데이터를 읽을 수 있는 특수

정보관리(3교시)

	읽기 쓰기가 용이하고 언제든지 프로그램의 수정이 가능함	상황의 명령어가 필요하고, 추가로 명령(프로그램) 메모리와 데이터 메모리의 비트수가 다른 경우 주소를 변환하는 과정 필요 함.
--	-----------------------------------	---

5	GPGPU		
문제	인공지능,빅데이터 등 4 차 산업혁명으로 대변되는 새로운 기술의 발전이 대용량의 데이터를 효율적으로 처리하는 GPU 의 성장을 견인하고 있고 최근 국내에서 금융산업에서도 GPU 기반의 아키텍처 도입되고 있다. 다음 아래의 질문에 대하여 설명하시오. 가. GPU 와 GPGPU 개념 나. GPU 와 CPU 의 차이점		
도메인	CA	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제의도	국내에 알파고 등장 이후 GPU 에 대한 관심이 증대되고 있고, 인공지능 분야 외에 최근에는 금융권등 다양한 분야에서 도입 사례가 발생되고 있어 GPU 아키텍처와 CPU 와의 차이점에 대한 이해 여부 점검		
핵심 내용 키워드	그래픽 처리, 직렬/병렬 연산, 부동소수점 연산, GPU 가속컴퓨팅 , 멀티코어/매니코어, 프로세스/멀티 쓰레드, 임의접근/순차접근 , CUDA, OpenCL, C++ AMP, OpenACC		
목차예시	1.그래픽 처리장치인 GPU 의 성장 배경 2.GPU 와 GPGPU 의 개념 3.CPU 와 GPU 의 차이점 4.GPU 의 미래		
채점 점수 가이드	① GPU,CPU 개념, 이해 부족 (1~8 점) ② GPU,CPU 기본 이해 수준 (9~14 점) ③ GPU,CPU 차이점 정확한 기술 제시 (14~15 점) ④ GPU 시장동향 등 추가요소 설명(+α)		
참고문헌	KB 지식 비타민 : 4차 산업혁명과 GPU의 성장 (2017.06 – 17-49호) 엔비디아 Nvidia : 4차 산업혁명의 BRAIN 머신러닝과 블록체인을 떠받치는 GPU의 모든 기술 https://blog.naver.com/shumin/220558101396		
고득점전략 및 답안작성 가이드	CPU, GPU, GPGPU 의 명확한 개념 제시 및 CPU 와 GPU 의 차이점을 다양한 관점에서 제시 추가적으로 GPU 활용과 시장동향에 대한 제시를 통한 추가 점수 확보 필요		
출제자	박재성 기술사(제 111 회 컴퓨터시스템응용기술사 / kftc@naver.com)		

1. 그래픽 처리장치인 GPU의 성장 배경



- 그래픽 처리의 핵심 프로세서인 GPU는 다수 코어에 의한 병렬 연산의 장점을 바탕으로 일반적인 데이터 처리에도 활용되는 GPGPU(범용 GPU)로 발전
- 방대한 양의 데이터, 스스로 학습하는 알고리즘 등을 특징으로 하는 4차 산업혁명의 등장과 함께 대용량 데이터를 효율적으로 처리하는 GPU의 사용량 급증

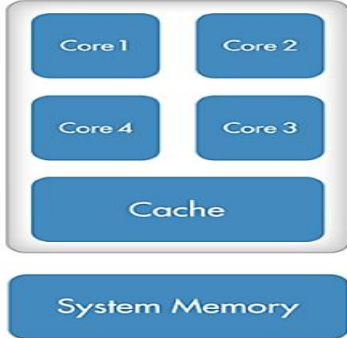
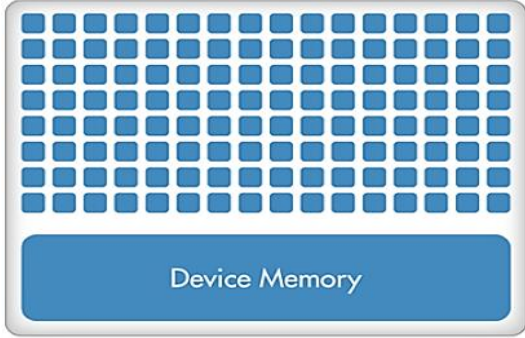
2. GPU와 GPGPU의 개념

구 분	내 용	
GPU	개념	-그래픽 처리장치로써 이미지와 영상을 처리하는 역할을 담당하는 그래픽카드의 핵심반도체 -Floating Point 연산을 하는 작은 ALU 코어를 수천개 반복 구성시켜 빠른 그래픽처리 수행
	목적	-컴퓨터의 그래픽 연산 담당, 3D Graphic -CPU로부터 받은 정보를 신호로 변환하여 화면에 출력
GPGPU (General Purpose GPU)	개념	-GPU를 CPU가 맡고 있는 응용 프로그램들의 계산에 사용할 수 있도록 활용하고자 하는 기법 -부동소수점으로 표현된 계산량이 매우 많은 3D 그래픽 등을 병렬 처리해 오던 GPU를 응용하여 일반 데이터에 병렬 연산을 적용할 수 있도록 구현한 기법
	목적	-컴퓨터 그래픽 연산, 일반 데이터 병렬 연산
	장점	-부동소수점 연산 엄청 빠름 -문맥교환 속도 바름 -엄청난 수의 쓰레드 사용
	단점	-흐름제어 기능 자체가 빈약 -프로그래밍 기법의 제약 (재귀호출불가) -Core 당 클럭이 CPU에 비해 많이 느림 (1/3 ~1/2 수준)

- 기존 CPU와 GPGPU를 결합, 기본적인 컴퓨팅 환경은 CPU가 담당하고 대량의 데이터에 대한 신속한 연산이 필요한 영역은 GPU에게 맡기는 방식이 확대 (이를 'GPU 가속 컴퓨팅'이라 함)

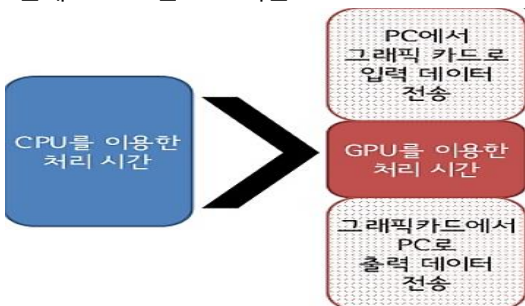
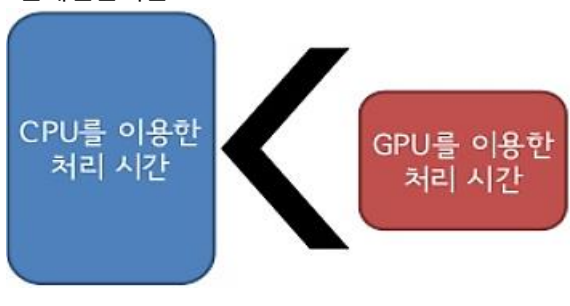
3. CPU와 GPU의 차이점

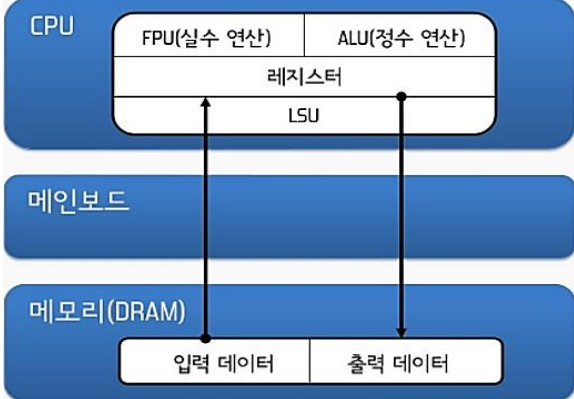
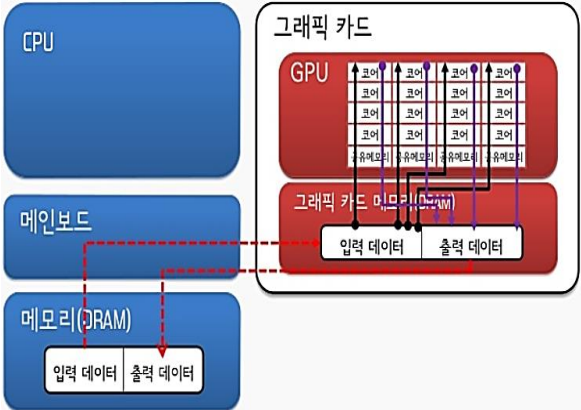
가. CPU와 GPU의 구성측면에서의 차이점

구 분	CPU	GPU
구조		
코어수	-Multiple Cores	-Hundreds of Cores
코어구성	-하나의 CPU 는 직렬 처리에 최적화된 몇 개 코어로 구성	-병렬처리로 설계된 수천개의 소형이고 효율적인 코어로 구성
구성요소	-DRAM, Cache, Control, ALU	-DRAM, 소형 Cache, 소형 Control, 다수의 ALU -SP, SM, TPC
제작방식	-CPU, 메인보드, 메모리 제조사가 다름	-그래픽카드 안에 보드, 메모리, GPU 모두 존재
개발	-제조사에서 개발이 덜 용이	-제조사에서 개발 용이
대역폭	-좁은 대역폭	-넓은 대역폭
전력	-많은 전력 소비	-적은 전력 소비

-CPU는 빠른 작업종료를 위해 복잡한 컨트롤러의 구성이 중점이며, GPU는 고속 병렬처리를 위한 ALU로구성되어 있음

나. CPU와 GPU의 연산측면에서의 차이점

구 분	CPU	GPU
연산처리	-직렬기반 -다양한 명령을 순차적으로 처리	-병렬처리 -특정 연산을 동시에 처리
연산속도	-빠른 명령 처리 -빠른 속도로 여러 번 명령 처리	-고속 병렬 연산 -CPU 보다 속도가 다소 느리지만 동시에 연산하여 병렬 처리
Load Calculation	-전체 프로그램 소요시간 	-실제연산시간 

연산대상	-정수 연산 중심	-부동 소수점 연산 중심
연산수행	-프로세서 중심 -프로세서에서 스레드를 구성하여 동작	-멀티쓰레드가 기본
연결구조	-직렬 파이프 라이닝	-병렬 파이프 라이닝
성능	-작업 종료시간	-전체의 명령 처리 빠르게 처리
접근	-임의의 접근	-순차적 접근
데이터 프로세싱		
문맥교환	-코어의 제한 -문맥교환이 비교적 오래 걸림 -레지스터 8~16 개	-매니코어 -문맥교환의 정보 최소화 -레지스터 SM 마다 16,384 개

-고속 연산, 병렬 연산을 통한 GPU의 데이터 처리 능력으로 인해 머신러닝, 딥러닝 등 인공지능 분야에서 GPU가 보편적으로 사용

4. GPGPU의 미래

가. GPGPU의 시장 동향

구 분	핵심요소	내 용
클라우드 서비스	-GPU 가상화 서비스	-사용자가 필요한 시점에 가상화된 컴퓨팅 환경 제공
자율주행차	-딥러닝 알고리즘	-주변환경 인지, 주행방식 판단, 제어
가상화폐	-병렬/분산 컴퓨팅	-채굴하는 과정에서의 복잡한 연산 수행
가상현실(VR)	-대규모 그래픽 연산	-시각, 촉각, 청각 요소까지 현실감 구현 제공
딥러닝	-병렬연산, GPU 가속기	-수많은 데이터 속에서 특징과 패턴등을 발견하고 분류
컴퓨터비전	-심층학습 기능	-정확한 안면 인식, 고성능 통신 모듈
데이터분석	-병렬 컴퓨팅	-대용량의 데이터를 분석, 시뮬레이션 수행
금융권 서비스	-전력과 성능분할	-현금흐름, 가정관리 시뮬레이션, 결산 데이터 분석
지능형 반도체	-인공신경망 연산	-초지능, 초고성능, 초경량, 초저전력 실현의 핵심원천

-4차산업혁명의 동인 기술을 기반으로 GPU의 활용도와 역할의 증대가 예상

-GPGPU는 병렬 처리를 하기 위해 프로그래밍을 작성해야 하는데, 이를 위한 플랫폼이 출현

나. GPGPU의 플랫폼 동향

플랫폼	특징	내용
CUDA	-병렬컴퓨팅 플랫폼	-GPU 를 이용한 범용적인 프로그램 개발하도록 프로그램모델, 언어, 컴파일러등을 제공하는 병렬 컴퓨팅 플랫폼 -NVIDIA
OpenCL	-개방형 범용 병렬 컴퓨팅	-GPU,CPU 등 이기종 컴퓨터 시스템을 지원하는 개방형 범용 병렬 컴퓨팅 플랫폼 -크로노스 그룹
C++ AMP	-비주얼스튜디오 2012 종속	-GPU,CPU 를 사용한 이기종 컴퓨팅을 위한 개방형 프로그래밍 언어 -마이크로소프트
OpenACC	-크로스 플랫폼	-CUDA 를 추상화시킨 컴파일러 지시문 기반 프로그래밍 모델 -NVIDIA

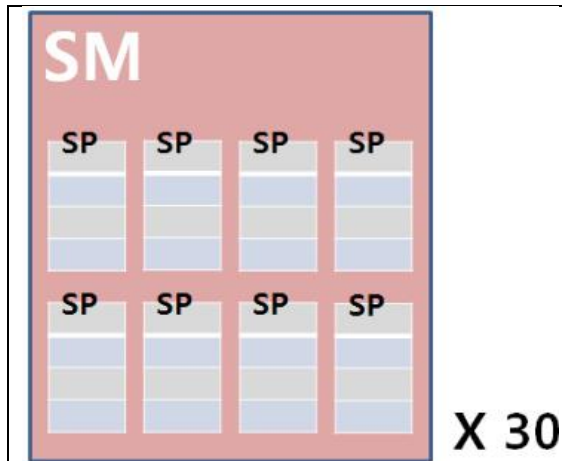
-GPGPU 프로그래밍 작성을 위한 다양한 플랫폼 출현으로 인해 GPU 병렬처리 연산능력이 향상되고 있어, 인공지능, 빅데이터 등 다양한 분야에서 활용도가 증대
끝.

참고자료)

-GPGPU 플랫폼	Controllable Hardware	Platform Dependency	Features
CUDA	GPU	NVIDIA Geforce8†	Runtime API /Device API
OpenCL	Heterogeneous CPU/GPU	Platform-independent	C/Runtime/API
OpenACC			Compiler hints, "directives"
C++ AMP			Lambda /parallel loop statement

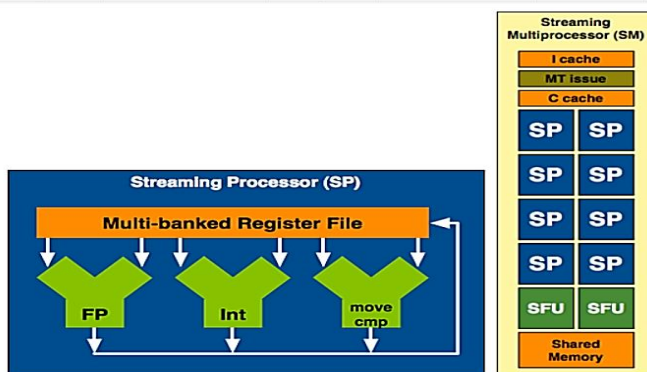
-GPU 구조

	-SM (Scalar Processor) : 4 개의 쓰레드로 구성 -SP (Streaming Multiprocessor) : SM 의 집합 -SM 은 8 개의 SP 로 구성 $SM = 8 SP = 8 SP * 4 Thread = 32 \text{ 개 Thread}$ 32 개 Thread = 워프 (Warp) 단위로 불림 16 개 Thread 로 메모리 참조시 하프워프(HalfWarp)
--	--

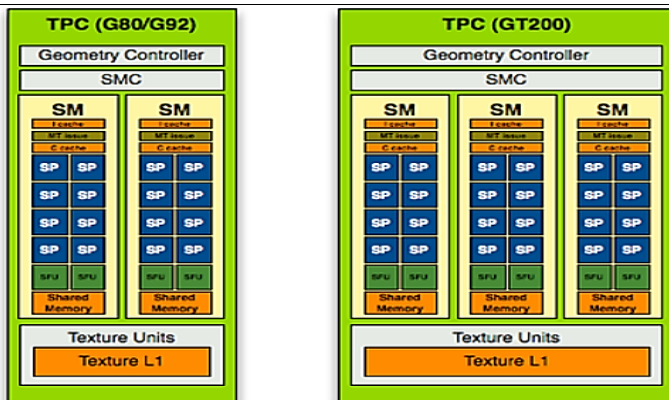


-GPU 는 1SM 단위(32 Threa)로 동작(메모리 참조)

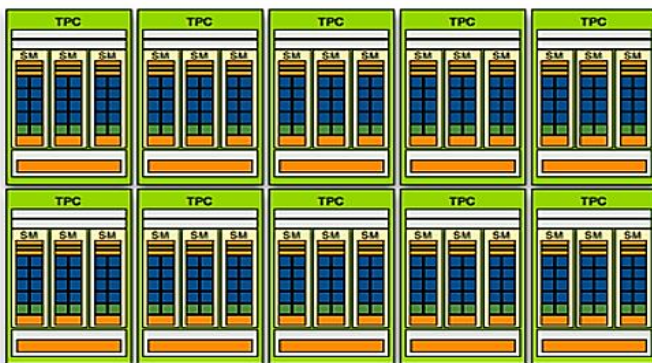
-Overall GPU 구조



-SP (Streaming Processor)로부터 출발
-SFU (Special Function Unit)을 묶어서 SM 을 하나의 빌딩블럭으로 구성

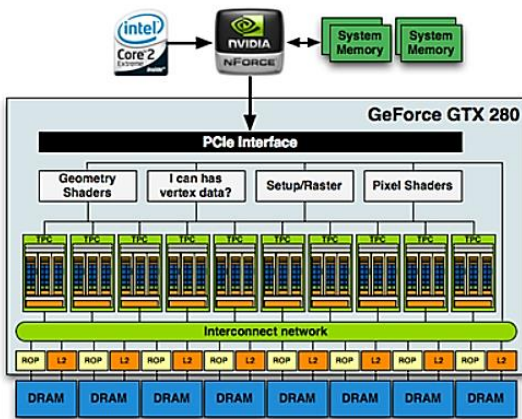


-Texture Processor Cluster 라는 TPC 구성
SM 의 개수에 따른 모델 분류



-TPC 를 10 개 모아서 총 240 개의 Streaming Processor 구성

컴퓨터시스템응용(3교시)



-마지막으로 메모리와 연결하여 GPU 완성

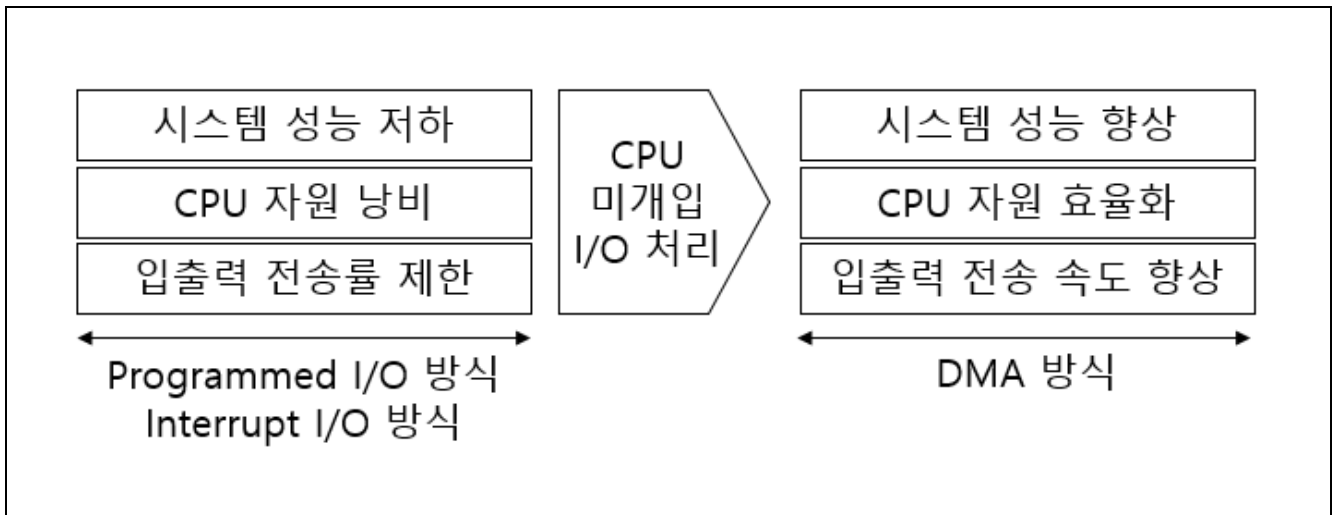
7	DMA (Direct Memory Access)		
문제	DMA(Direct Memory Access) 입출력 제어 방식 중 아래 사항에 대하여 설명하시오. 가. DMA 제어기 구조와 버스 연결 방식 나. DMA 동작 모드와 처리 절차		
도메인	CA	난이도	★ ★ ★ ☆ ☆
출제배경	직접 기억장치 액세스(DMA) 방식을 이용한 입출력 제어 방식의 구조와 연결 방식, 동작 모드와 처리 절차 등 컴퓨터 I/O 원리에 대한 이해도 점검		
핵심 키워드	CPU 미개입, 주소 / 데이터 / 계수 / 제어 레지스터, 제어 회로, 버스 요구 신호(BR), 버스 승인 신호(BG), 인터럽트, 단일 버스 분리형, 단일 버스 통합형, 입출력 버스, Cycle Stealing, Burst Transfer		
참고문헌	컴퓨터 구조와 원리 2.0 (한빛아카데미) 제 67 회 KPC 기술사 IMPACT 실전 모의고사 해설집		
출제자	이시광 기술사(제 119 회 컴퓨터시스템응용기술사 / sk2ckr@naver.com)		

1. 직접 기억장치 액세스 방식, DMA의 개념 및 등장 배경

가. DMA (Direct Memory Access)의 개념

- 효율적 입출력 수행을 위해 기억장치와 입출력 모듈 간 데이터 전송을 DMA 제어기가 처리하고 중앙처리장치는 개입하지 않도록 하는 직접 기억장치 액세스 기술

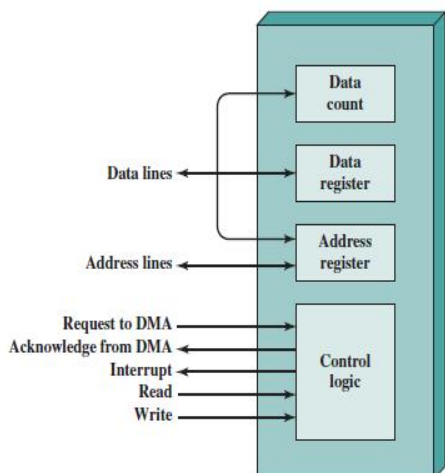
나. DMA의 등장 배경



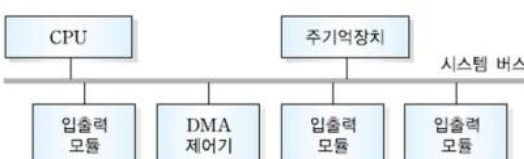
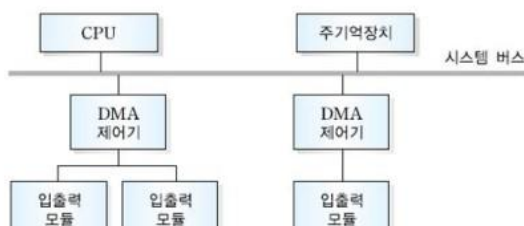
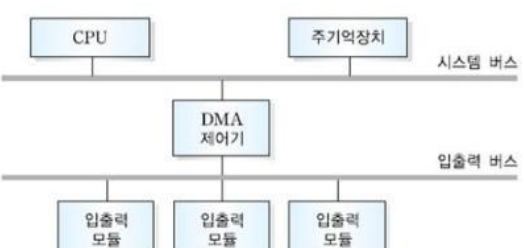
- Programmed I/O 방식 사용 시 CPU가 입출력과 관련된 처리를 수행하기 때문에 빠른 전송속도가 보장되지만 입출력 처리 시 CPU가 다른 작업을 처리하지 못하여 전체적으로 시스템 성능 저하 발생
- Interrupt I/O 방식 사용 시 입출력 모듈이 동작하는 동안 CPU가 다른 작업을 처리할 수 있지만 기억장치와 입출력 모듈 간 데이터 전송이 CPU를 경유하기 때문에 입출력 전송률이 제한
- DMA 방식을 사용하면 CPU가 개입하지 않고 I/O를 처리하므로 전체적인 시스템 성능 향상과 CPU 자원 효율화, 입출력 전송 속도 향상 가능

2. DMA 제어기 구조와 버스 연결 방식

가. DMA 제어기 구조

DMA 제어기 구조	구성 요소	구성 요소 설명
 The diagram shows a DMA controller block with internal components: Data count, Data register, Address register, and Control logic. External connections include Data lines (bidirectional), Address lines (bidirectional), Request to DMA, Acknowledge from DMA, Interrupt, Read, and Write signals.	주소 레지스터	- 주기억장치의 위치를 지정하는 주기억장치의 주소 저장
	데이터 레지스터	- 전송된 데이터 저장
	계수 레지스터	- 전송되는 데이터 단어의 수 저장
	제어 레지스터	- 입출력 시 전송 모드 지정 - Cycle Stealing, Burst Transfer Mode
	제어 회로	입출력 신호 발생 회로 - 버스 요구 신호(BR, BUS REQ) - 버스 승인 신호(BG, BUS GNT) - 인터럽트 (INTR) - 읽기(RD) 및 쓰기(WR)

나. DMA 제어기의 버스 연결 방식

버스 연결 방식	버스 구조	설명 / 장단점
단일 버스 분리형 방식	 CPU and Main Memory are connected to the System Bus. Input/Output modules and the DMA controller are connected to a separate bus.	- DMA 제어기와 입출력 모듈이 시스템 버스에 직접 연결 - 데이터 전송 시 시스템 버스를 두 번 씩 사용하므로 성능 저하 발생
단일 버스 통합형 방식	 CPU and Main Memory are connected to the System Bus. The DMA controller is also connected to the System Bus, and its I/O modules are connected to a local bus.	- DMA 제어기에 입출력 모듈 연결 - 단일 버스 분리형 방식의 단점 극복 - 데이터 전송 과정에서 시스템 버스 한 번만 사용, 성능 저하 방지
입출력 버스를 이용한 방식	 CPU and Main Memory are connected to the System Bus. The DMA controller and its I/O modules are connected to a dedicated I/O bus.	- 시스템 버스와 입출력 버스 분리 - 단일 버스 분리형 방식의 단점 극복 - 데이터 전송 과정에서 시스템 버스 한 번만 사용, 성능 저하 방지

- DMA 제어기와 입출력 모듈이 시스템 버스나 입출력 버스에 연결된 형태에 따라 버스 연결 방식 분류

3. DMA 동작 모드와 처리 절차

가. DMA 동작 모드

동작 모드	개념도	전송 단위 및 동작 방식
Cycle Stealing 모드		- Word 단위 데이터 전송 - DMA 동작 시 CPU Cycle 중지 요청 - DMA 제어기와 CPU 동시 BUS 요청 시 DMA에 우선권 부여
Burst Transfer 모드		- Block 단위 데이터 전송 - 여러 Word로 구성된 Block 전송 - DMA 제어기가 BUS 권한 획득 시 Block 전송 완료될 때까지 BUS 독점

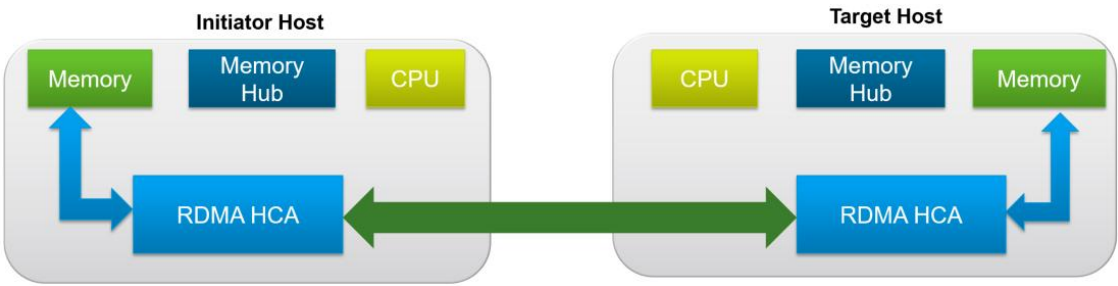
- CPU가 시스템을 사용하지 않을 때 DMA 제어기가 BUS를 사용하는 Interleaved DMA 동작 모드도 존재

나. DMA 처리 절차

구분	상세 설명	
DMA 처리 절차 구성도		
DMA 처리 절차	① 입출력 명령 전송 ② 버스 요구(BR) 신호 전송 ③ 버스 승인(BG) 신호 전송 ④ 데이터 입출력 수행 ⑤ 잔여 전송 데이터 확인 ⑥ 완료 인터럽트 신호 전송	- CPU는 DMA 제어기로 I/O 주소 포함 입출력 명령 전송 - DMA 제어기는 CPU로 버스 요구(BUS REQ) 신호 전송 - CPU는 DMA 제어기로 버스 승인(BUS GNT) 신호 전송 - CPU 개입 없이 DMA 제어기가 버스 독점, 입출력 수행 - 전송할 데이터 존재 시 ② ~ ④ 단계 반복 - I/O 완료 시 DMA 제어기는 CPU로 인터럽트 신호 전송

- CPU는 DMA 제어기로 명령 전송 후 다른 작업을 계속할 수 있으며, DMA 제어기가 모든 입출력 동작을 전담, 모든 입출력 동작이 완료되면 DMA 제어기는 관련 작업이 재개되도록 CPU로 인터럽트 신호를 전송
- 대규모 병렬 컴퓨터 클러스터에서 DMA를 네트워크로 확장한 원격 직접 메모리 액세스(RDMA) 적용 확대 추세

4. DMA 네트워크 확장 기술, 원격 직접 메모리 액세스(RDMA, Remote Direct Memory Access)

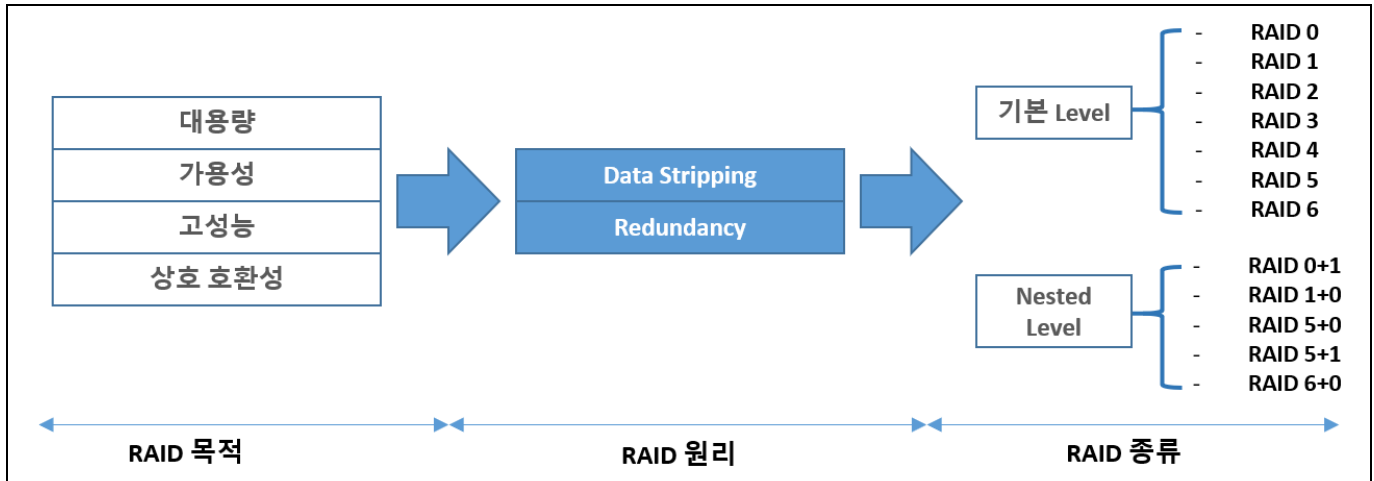
구분	설명	
개념	 - 네트워크 호스트 간 고속 데이터 전송을 위해 CPU 및 운영체제의 관여 없이 두 호스트의 메모리 간 네트워크로 데이터를 직접 전송하는 원격 직접 메모리 액세스 기술	
활용 기술	Infiniband (IB)	- 높은 스루풋과 낮은 레이턴시, 높은 안정성과 확장성 확보 - 컴퓨팅 노드와 스토리지 장비와 같은 고성능 I/O 장비간의 연결에 사용하는 점대점 양방향 직렬 통신 기술
	RDMA Over Converged Ethernet (RoCE)	- 이더넷 네트워크를 통해 RDMA 허용하는 네트워크 프로토콜 - 이더넷을 통해 전송 패킷 캡슐화하며, UDP와 IP를 사용 - RoCE v1은 단일 브로드 캐스트 도메인 제한, RoCE v2 패킷은 라우팅 가능
	Internet Wide Area RDMA Protocol (iWARP)	- TCP와 같은 연결 지향 전송을 통해 RDMA를 수행 - iWARP 패킷은 라우팅과 TCP 흐름 및 안전성 제어 가능 - DDP (Direct Data Placement Protocol) 기반 TCP, SCTP 사용

- Infiniband, Gigabit Ethernet 등 고속 네트워크 상에서 적용하여 대규모 병렬 컴퓨터 클러스터 등에서 초고속 병렬 처리 기술로 활용

“끝”

3	RAID (Redundant Array of Independent/Inexpensive Disks)		
문제	RAID(Redundant Array of Independent/Inexpensive Disks) Level 중에 Raid 0, Raid 5, Raid 6, Raid 1+0, Raid 5+0 에 대해 최소 Disk 필요 수량, 공간 효율, 고장 허용 범위를 포함해 설명하시오.		
도메인	CA	난이도	★ ★ ★ ☆ ☆ (별 5개 기준)
출제의도	Big Data, 4 차 산업 혁명의 부각과 함께 스토리지가 중요 요소의 하나로 자리 잡고 있고, 스토리지의 기본 기술인 RAID 를 정확하게 이해하고 있는지 확인		
핵심 내용 키워드	Striping, Mirroring, Parity, 가용성, 고장 허용		
목차예시	1. RAID(Redundant Array of Independent/Inexpensive Disks)의 개요 2. 기본 Level 의 RAID 0, RAID 5, RAID 6 3. Nested RAID, RAID 0+1, 1+0		
채점 점수 가이드	① RAID 개념, 이해 부족 (5-8점) ② RAID 서브네팅 기본 이해 수준 (9-14점) ③ RAID 서브네팅 정확한 기술 및 변경 내용 제시 (14-15점) ④ 추가요소 설명(+α)		
참고문헌	- 컴퓨터 구조와 원리 2.0 (한빛출판사) - https://en.wikipedia.org/wiki/Standard_RAID_levels - KPC CA 서브노트		
고득점전략 및 답안작성 가이드	- RAID 는 기본적인 토픽이지만 실제 시스템 구성 시 많이 사용하고 있는 기술이므로, RAID 0,1 등의 기본 구성부터 복합 구성에 이르기까지 전반적인 이해 후 기술 필요 - 문제에서는 최소 Disk 수량, 공간 효율, 고장 허용 범위 등을 물어봤지만, 해당 질문이 없더라도 RAID 는 구체적인 수치를 제시해서 가용성/효율성을 기술		
출제자	최범규 기술사(제 107 회 컴퓨터시스템응용기술사 / aahoo@naver.com)		

1. RAID(Redundant Array of Independent/Inexpensive Disks)의 개요



- 여러 드라이브의 집합을 하나의 저장장치처럼 다룰 수 있게 하고, 장애가 발생하였을 때 데이터를 잃어버리지 않게 하며 디스크 각각이 독립적으로 동작할 수 있도록 하는 저장장치 기술

2. 기본 Level의 RAID(Redundant Array of Independent/Inexpensive Disks), RAID 0, RAID 5, RAID 6

가. Data Striping 을 이용한 RAID 0

구성도	항목	설명
	특징	- 디스크 중복을 통한 읽기/쓰기 성능 2 배 향상 - Parity/Mirroring 등이 없어 디스크 안전성 향상 없음 - Bit Level Striping
	최소 수량	- 2 개
	공간 효율	- n (n : Disk 수량)
	읽기 향상	- n 배 향상 (n : Disk 수량)
	쓰기 향상	- n 배 향상 (n : Disk 수량)
	고장 허용	- Zero
	활용	- High end workstation, Data logging, Real-time rendering

나. Parity 분산을 통한 안정성 제공, RAID 5

구성도	항목	설명
	특징	- Block 레벨의 Striping과 Parity 사용(Parity 분산 제공)
	최소 수량	- 3개
	공간 효율	- $1 - 1/n$ (n : disk 수량)
	읽기 향상	- $n-1$ 배 (n : Disk 수량)
	쓰기 향상	- Variable (상황에 따라 다름)
	고장 허용	- 1disk
	활용	- DW, Web Serving, Archiving

다. Double Parity 분산을 통한 안정성 제공, RAID 6

구성도	항목	설명
Disk 0 Disk 1 Disk 2 Disk 3 Disk 4	특징	- Block 레벨의 Striping과 Double Parity 사용(Parity 분산 제공)
	최소 수량	- 4개 (n : disk 수량)
	공간 효율	- $1 - 2/n$ (n : Disk 수량)
	읽기 향상	- 해당 없음
	쓰기 향상	- 해당 없음
	고장 허용	- 2disk
	활용	- Data Archive, Backup, HA Solution, 대용량 서버

3. Nested RAID, RAID 1+0, 5+0

가 RAID 1+0

구성도	항목	설명
Disk 0 Disk 1 Disk 2 Disk 3	특징	- 미러링된 두 개의 어레이에 걸쳐 데이터가 Striping 되었음을 의미하는 "미러링된 세트의 Striping" - 미러링 기능으로 인한 신속한 복구 가능 - Striping 으로 인해 속도는 빠르지만 장치의 사용가능 한 용량은 절반으로 감소
	최소 수량	- 4배수의 디스크 필요
	공간 효율	- 1/2
	읽기 향상	- n/2배 향상 (n : Disk 수량)
	쓰기 향상	- n/2배 향상 (n : Disk 수량)
	고장 허용	- 데이터 손실 없이 미러링된 각각의 쌍에서 한 개의 디스크에 손상 허용
	활용	- DW, Web Serving, Archiving

나. RAID 5+0

항목	설명
구성도	Disk 0 Disk 1 Disk 2 Disk 3 Disk 4 Disk 5

특징	- RAID 5 구성시 드라이브 숫자가 많아지면 패리티 연산 문제로 성능 저하가 발생할수 있는데, 이걸 RAID 0 을 이용해 Striping 하는 방식
최소 수량	- 6 개
공간 효율	- $1 - 2/n$ (n : Disk 수량)
읽기 향상	- 읽기 향상
쓰기 향상	- 쓰기 향상
고장 허용	- Sub array 당 1 개의 Disk 복구 가능
활용	- Large Databases, File Server, Application Server

- 원래 RAID 의 'I'는 'Inexpensive'의 약자로 '저렴한 디스크의 중복 배열'을 의미해 저렴한 저용량 HDD 를 여러 개 중복 연결해 고용량 HDD 처럼 사용하기 위한 대표 기술이었음.

- 고용량 HDD 가격이 대폭 낮아진 지금은 파일 입출력 속도/성능을 높이고, 파일 안정성을 강화하는 주요 수단으로 활용되고, RAID 의 'I'도 'Independent'의 의미로 통용됨

“끝”

문 제	12. 블록스토리지, 파일 스토리지, 오브젝트 스토리지의 데이터 접근방식		
출 제 영 역	데이터베이스	난 이 도	★★★☆☆ (별 5개 기준)
출 제 배 경	데이터 관리 형식인 블록스토리지, 파일스토리지, 오브젝트스토리지의 개념 확인 및 각각의 데이터 접근방식, 특징 등의 이해도 확인		
출 제 빈 도	미출제		
참 고 자 료	클라우드 기술백서(삼성 SDS, www.samsungsds.com/kr/techreport)		
Key word	SAN, 정형 데이터, 파티션, 메타데이터, 비정형 데이터, 성능, NAS, 검색용이성, 블록, 파일, 오브젝트		
풀이 기술사님	허수영(131 회 정보관리기술사 / sktuddl2@naver.com)		

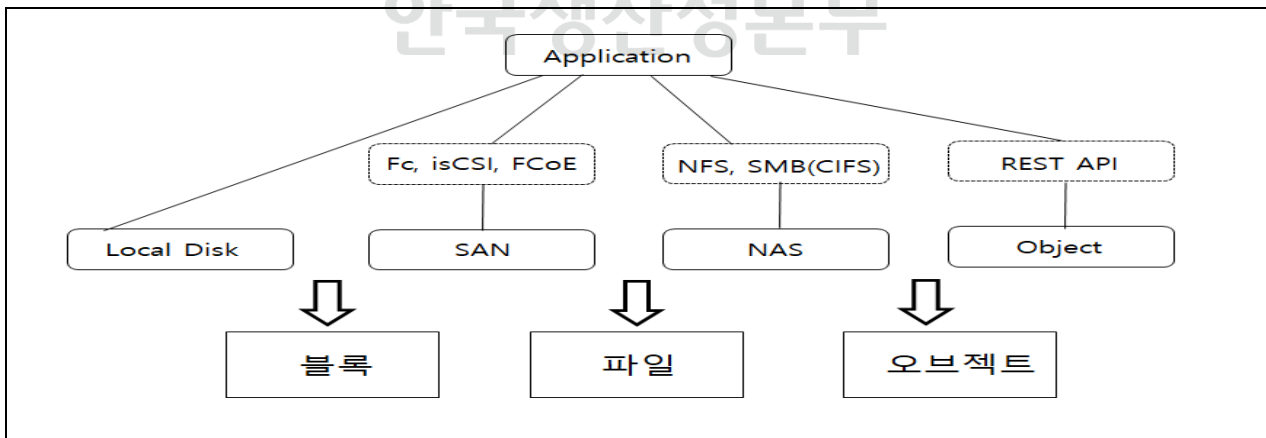
1. 클라우드 데이터 관리 기술, 블록, 파일, 오브젝트 스토리지 정의

구분	블록 스토리지	파일스토리지	오브젝트스토리지
정의	데이터를 블록 단위로 나누어 분리해 관리하는 스토리지	데이터를 파일 형태로 저장, 파일은 폴더 구조로 관리하는 계층형 데이터 스토리지 방법론	대량의 비정형데이터를 처리하기 위한 스토리지 아키텍처, 오브젝트를 플랫폼 구조의 데이터 저장

- 빅데이터, AI 등 사용 증가로 대량 생산 되고 있는 데이터를 효과적으로 관리 처리 하는 기술이 부각되고 있음

2. 블록 스토리지, 파일스토리지, 오브젝트 스토리지 구성도 및 기술요소

가. 블록스토리지, 파일스토리지, 오브젝트 스토리지 데이터 접근 방식 개념도



- 각각의 스토리지 데이터 접근방식은 장단점이 존재하여 이에 대한 활용 영역이 달리 존재함

나. 블록스토리지, 파일스토리지, 오브젝트 스토리지 데이터 접근 방식 기술요소

구분	기술요소	상세설명
블록 스토리지	블록고유주소(블록 ID)	고정크기 블록, 고유주소를 활용하여 블록 재구성, 호출 처리
	전용 프로토콜	FCP(파이버채널 프로토콜), iSCSI, FCoE 프로토콜 사용
파일 스토리지	제한적 메타데이터	이름, 위치 등 제한적 메타데이터 활용
	OS 통신 프로토콜	UNIX(NFS), Windows(SMB, cifs) 파일공유 프로토콜 사용

오브젝트 스토리지	오브젝트 ID	오브젝트 고유(URI) 부여 관리
	확장적 메타데이터	오브젝트 평면(Flat)구조로 저장 접근, 확장성 메타데이터
	오브젝트 통신	REST API (HTTP 프로토콜기반)

- 빠른응답이 필요한경우 블록스토리지, 일반적 파일공유 파일스토리지, 인터넷을 통해 액세스하거나 갱신 빈도가 낮은 데이터는 오브젝트 스토리지 사용

3. 블록 스토리지, 파일스토리지, 오브젝트 스토리지 데이터 접근방식 장단점 비교

구분	블록 스토리지	파일 스토리지	오브젝트 스토리지
특징	오버헤드 적음	파일공유 가능	OS나 파일시스템 의존 없이 데이터 저장, 액세스 가능
	빠른데이터 전송	엑세스 제어 속성관리 쉬움	용량 확장 쉬움 크기에 제한 없음
활용	낮은 레이턴시 요구경우 활용	계층형 파일 관리에 사용	갱신빈도가 적은 대량의 데이터 장기 보존 용도로 활용

"끝"



기출풀이 의견

12. 스토리지의 데이터 접근방식에 대해 풍부하게 작성해주시고, 활용도 측면으로 3단락을 제시해주시면 차별화 된 답안 작성 가능합니다.

14	FTS		
문제	FTS(Fault Tolerant System)에 대해 설명하시오		
도메인	컴퓨터구조	난이도	★ ★ ☆ ☆ ☆ (별 5 개 기준)
출제배경	시스템 안전성, 신뢰도 및 가용성에 대한 이슈로 중요도 증가, 임베디드 시스템 및 IoT 활성화에 따른 장치 FTS 중요도 증가		
핵심 키워드	TMR, Watchdog Timer, Check Point, Shadow Paging, 결함방지(Fault Detection), 결함진단(Fault Diagnosis), 결함통제(Fault Isolation), 결함복구(Fault Recovery)		
참고문헌	https://blog.lgcns.com/744 https://ko.wikipedia.org/wiki/%EC%9E%A5%EC%95%A0_%ED%97%88%EC%9A%A9_%EC%8B%9C%EC%8A%A4%ED%85%9C		
출제자	문광석 기술사(제 117 회 정보관리기술사 / itpe.moon@gmail.com)		

1. 시스템 안전 보장, 결함허용시스템(FTS)의 개념

- 하드웨어 또는 소프트웨어의 결함, 오동작, 오류 등이 발생하더라도 규정된 기능을 지속적으로 수행할 수 있는 시스템

2. 결함허용시스템(FTS)의 주요 단계 및 주요 기법

가. 결함허용시스템(FTS)의 주요 단계

주요 단계	설명	기법
결함감지 (Fault Detection)	- 시스템 내에서 Fault가 발생되면 해당 모듈은 Fault 상태로 들어가게 되고, OS는 각 모든 하드웨어 모듈들의 상태를 분석하여 어느 모듈이 Fault를 유발시켰는가 감지	Ping & Echo, Heartbeat, Exception 처리
결함진단 (Fault Diagnosis)	- Fault가 일시적(Transient)인 것인지 영구(Hard)적인 것인지 판단하여 영구적인 경우 해당 모듈을 시스템 구성에서 제거	모니터링, 비교, 시뮬레이션, 오류 비트
결함통제 (Fault Isolation)	- 결함으로 인한 타 모듈에 오류 파급 차단	중복(Redundancy), 교체, 격리(Isolation)
결함복구 (Fault Recovery)	- Fault를 유발한 모듈을 시스템에서 제거하여 시스템을 재구성	

- 결함 발생시 감지 -> 진단 -> 통제 -> 복구를 통해 정상적 혹은 부분적 기능을 지속적으로 수행 가능

나. 결함허용시스템(FTS)의 주요 기법

구분	주요 기법	설명
HW 측면	Duplication with Comparison	- 하드웨어를 중복하여 그 출력을 비교하여 결함을 감지하는 기법
	TMR(Triple modular	- 3개 이상의 프로세서가 같은 입력에 대하여 동일한 연산수행

구분	주요 기법	설명
	Redundancy	(2개 이상 출력이 같으면 이용)하는 기법
	Watchdog Timer	- 시스템으로 하여금 주기적으로 타이머를 초기화 시키도록 하여 초기화 시키지 않을 때 시스템에 결함이 있음을 감지하는 기법
	Self Purging Redundancy	- N개의 H/W에서 출력 결과가 다른 HW를 계산과정 배제기법
SW 측면	Check pointing	- S/W 수행 중에 검사시점을 설정하여 오류발생이 발견되면 발생이전의 검사시점으로 되돌아가서 재 수행방식
	Recovery Block	- 재 수행(Rollback & Retry)에 근거하여, 검사시점에서 오류가 발견되면 지정된 이전 검사점으로 되돌아가서 같은 기능을 가진 다른 S/W 모듈을 수행방식
	N Self-checking Programming	- 두개 이상의 Self-checking 컴포넌트가 수행되면서 하나는 주어진 기능을 수행하고 다른 컴포넌트는 대기상태로 결함발견 시 대기 부품이 수행하는 기법
	N version Programming	- N개의 독립적인 S/W 모듈의 수행결과를 비교하여 다수의 수행결과를 채택하는 기법
DB 측면	Rollback (Undo)	- 트랜잭션 작업 철회 방식
	Check Point	- 중간 검사시점을 두어 결함 발견시점 이후만 회복 방식
	Shadow Paging	- 트랜잭션 실행동안 수행 페이지 테이블과 그림자 페이지 테이블을 지속 유지 방식
	Log File	- Commit안된 작업을 로그 파일로 회복 방식
DATA 측면	Parity Code	- 2진 정보 내의 "1"의 개수를 홀수 또는 짝수로 규정하여 결함 감지 방식
	Checksum	- 한 블록의 정보내용 합을 추가 정보 결함 감지 방식
	Berger Code	- 한 정보의 "1"의 개수를 2진 정보로 추가 방식
	Hamming Error Correcting Code	- 결함 감지 및 단일 결함의 교정까지 수행하는 코드 기법

- 신속한 결함 복구를 위한 H/W 측면의 FTS와 유연한 방식인 S/W 측면의 FTS를 용도에 맞게 적용 필요

3. HA와 FTS의 비교

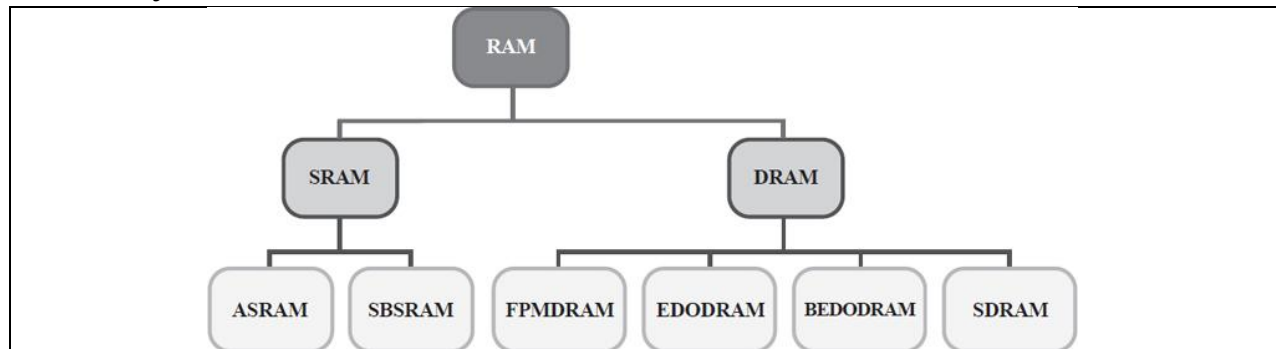
구분	HA(고가용성)	FTS(결함허용)
결함시간	30~300초	0초
Concurrent 유지보수	불필요	필수
동일성능 가격	2배 이상	10~20배
응용 프로그램	대다수 범용제품	제한적
운영체제	범용 OS	전용 OS
사용 하드웨어	범용 하드웨어	전용 하드웨어

- 요건에 따라 HA(High availability) 및 FTS(Fault Tolerant System)를 선택적 이용이 필요

“끝”

문 제	1-4 DRAM(Dynamic RAM)과 SRAM(Static RAM)		
출 제 영 역	CA	난 이 도	★★★★☆
출 제 배 경	반도체 메모리 활용 증가 및 DDR SDRAM 연관 토픽		
출 제 빈 도	93 회 관리, 110 회 응용, 123 응용		
참 고 자 료	https://science.jrank.org/computer-science/RandomAccess_Memory.html		
Key word	- DRAM (1TR / 1C, Dynamic, Refresh) - SRAM (6TR, Static, Flipflop)		
풀 이	김지환(122 회 정보관리기술사)		

1. DRAM(Dynamic RAM)과 SRAM(Static RAM)의 정의

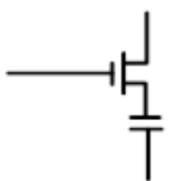


DRAM (Dynamic RAM)	한 셀당 한 개의 트랜지스터와 한 개의 커패시터로 이루어져, 전원이 꺼지면 기억된 내용이 유실되는 휘발성 메모리
SRAM (Static RAM)	6 개의 트랜지스터로 이루어져 있으며 중앙의 4 개 트랜지스터는 인버터 두개가 붙은 플립플롭 구조를 갖는 전원이 공급되는 동안에만 저장된 내용을 기억하는 메모리

- 순서대로 접근하는 것이 아니라, 접근하고 싶은 곳에 바로 접근할 수 있는 기억장치

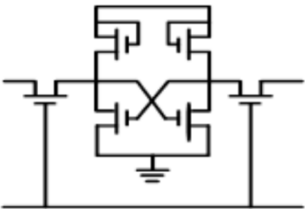
2. DRAM, SRAM 의 상세 설명

가. DRAM (Dynamic RAM)의 상세 설명

아키텍처	요소	설명
 DRAM의 내부구조 (1개의 TR과 C로 구성)	구성	- 트랜지스터 1 개와 커패시터 1 개로 구성
	동작원리	주기적 Refresh 통해 커패시터에 쌓인 전하를 유지
	구조적 특징	- 커패시터
	유형	- FPMDRAM : DRAM 의 속도 향상, 페이지단위 입출력 - EDODRAM : FPMRAM 개선, 입출력의 별도 공간 - BEDODRAM : 4 개 메모리 주소를 한 버스트에서 처리 - SDRAM : 동기식 DRAM
	특징	- 동적 메모리

		- 주기적으로 재충전되어야 기억된 내용을 유지 - 회로가 비교적 간단, 가격이 저렴 - 집적도가 높기 때문에 대용량의 기억장치에 주로 사용
--	--	---

나. SRAM (Static RAM)의 상세 설명

아키텍처	요소	설명
 SRAM의 내부구조 (6개의 TR로 구성)	구성	- 6 개의 트랜지스터로 구성
	동작원리	인버터를 통한 신호 0/1 을 반복하여 데이터 유지
	구조적 특징	- 플립플롭
	유형	- ASRAM : 비동기 SRAM - SBSRAM : synchronous SRAM with a burst feature
	특징	- 정적 메모리 - 전원 공급 되는 동안 기록이 유지되어 재충전 불필요 - 빠른 접근 속도 - 구조가 복잡하고 소비전력이 많이 필요하며 비싼 가격

3. DRAM 과 SRAM 의 상세 비교

구 분	DRAM	SRAM
Used in	Main Memory	Cache Memory
Speed	Slower	Faster
Size	Large	Small
Cost	Cheap	Expensive
Density	Highly dense	Less. Dense
Construction	Simple and uses capacitors and very few transistors	Complex and uses transistors and latches
Single block of memory requires	Only one transistor	6 transistors
Charge leakage property	Present hence require power refresh circuitry	Not present
Power consumption	High	Low

- 가격, 성능 등에 따라 사용하는 목적이 상이하므로 요구 성능에 대한 비교 분석 필요

"끝"

기출풀이 의견

1. 비교문제가 아닌 각각 설명하는 문제이기 때문에 DRAM, SRAM에 대하여 각각 설명을 하고 3단락을 통해 비교를 하시어 작성을 하시면 됩니다. Old 토픽이라도 꼭 다시 보셔야 합니다.

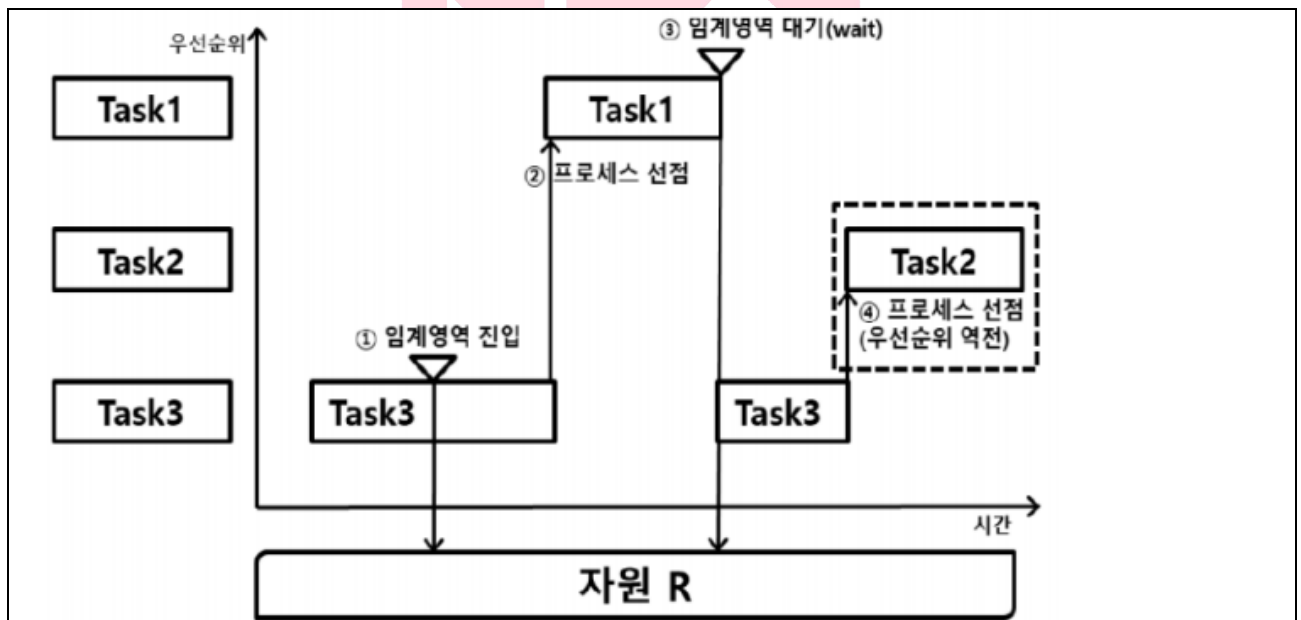
문 제	2. Real TimeScheduling 이 갖는 문제 중 우선순위 역전(Priority Inversion)이 있다. Task1, Task2, Task3 순으로 우선순위가 낮다고 할 때 우선순위 역전을 사례기반으로 설명하고, 우선순위 역전을 해결하기 위한 2 가지 기법에 대하여 설명하시오. (단, P,V 연산을 사용한다)		
출 제 영 역	OS	난 이 도	★★★★☆☆
출 제 배 경			
출 제 빈 도			
참 고 자 료			
K e y w o r d	우선순위 상속, 우선순위 올림, 세마포어		
풀 이	서 OO 기술사(제 124 회 정보관리기술사/ yhsuh2107@naver.com)		

I. Real Time Scheduling 이 갖는 문제 우선순위 역전(Priority Inversion)의 개념

- 낮은 우선순위를 가진 Task 에 의해 높은 우선순위를 가진 Task 가 임계영역 대기 때문에 블록 되어 수행되지않는 현상

II. 우선순위 역전 사례기반 설명

가. Task1, Task2, Task3 순으로 우선순위 낮은 경우, 우선순위 역전 사례개념도



- 우선순위가 가장 높은 Task1 이 결국 가장 나중에 완료.

나. Task1, Task2, Task3 순으로 우선순위 낮은 경우, 우선순위 역전 사례 상세 시나리오

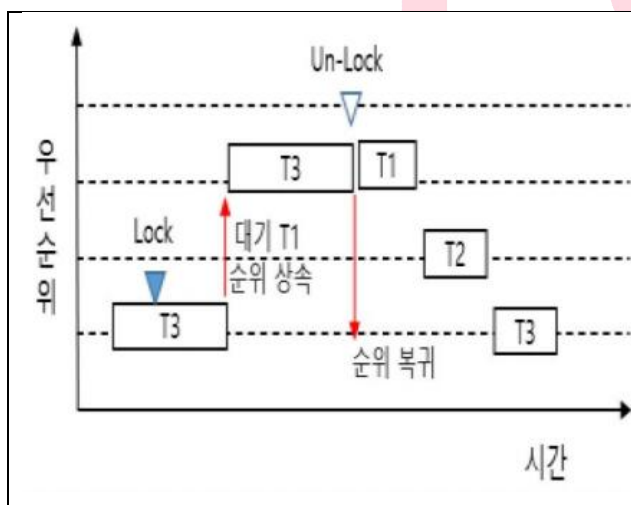
순서	우선순위 역전 발생 시나리오
----	-----------------

1	task3 이 공유자원 접근 위해 바이너리 세마포어 획득
2	스케줄러에 의해 task1 이 수행 (문맥 교환)
3	task1 은 task3 이 획득한 세마포어를 얻으려 하고, task3 이 그 세마포어를 반환할 때까지 waiting 상태
4	스케줄러에 의해 task3 이 수행
5	스케줄러에 의해 task2 수행. 이 때, task1 의 우선순위가 task2 보다 높음에도 task2 가 먼저 수행. 우선순위 역전
6	task2 수행 종료 시 다시 task3 이 수행
7	task3 세마포어 반납
8	task1 세마포어 획득 후 수행 완료

- 우선순위 역전 현상을 해결기법은 "우선순위상속기법"과 "우선순위 올림 기법"이 있음.

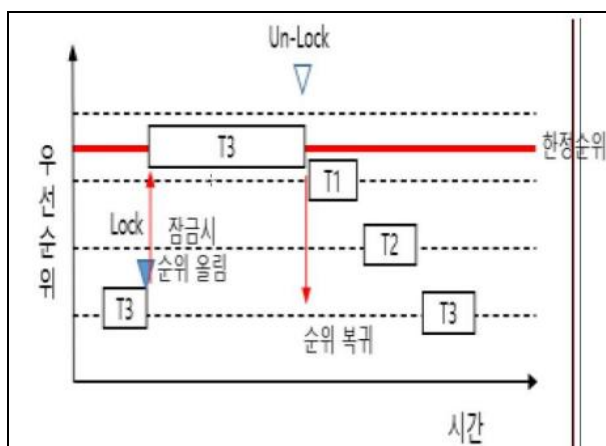
III. 우선순위역전을 해결하기 위한 2 가지 기법, 우선순위 상속기법과 우선순위 올림 기법

가. 우선순위상속(Priority Inheritance)기법



- 우선순위 낮은 Task3 을 임시로 자원요청한 상위우선순위를 부여해서 먼저 실행
- Deadlock 걸릴 확률 있음 (Task3 이 Task1 점유 자원을 요청하는 경우)

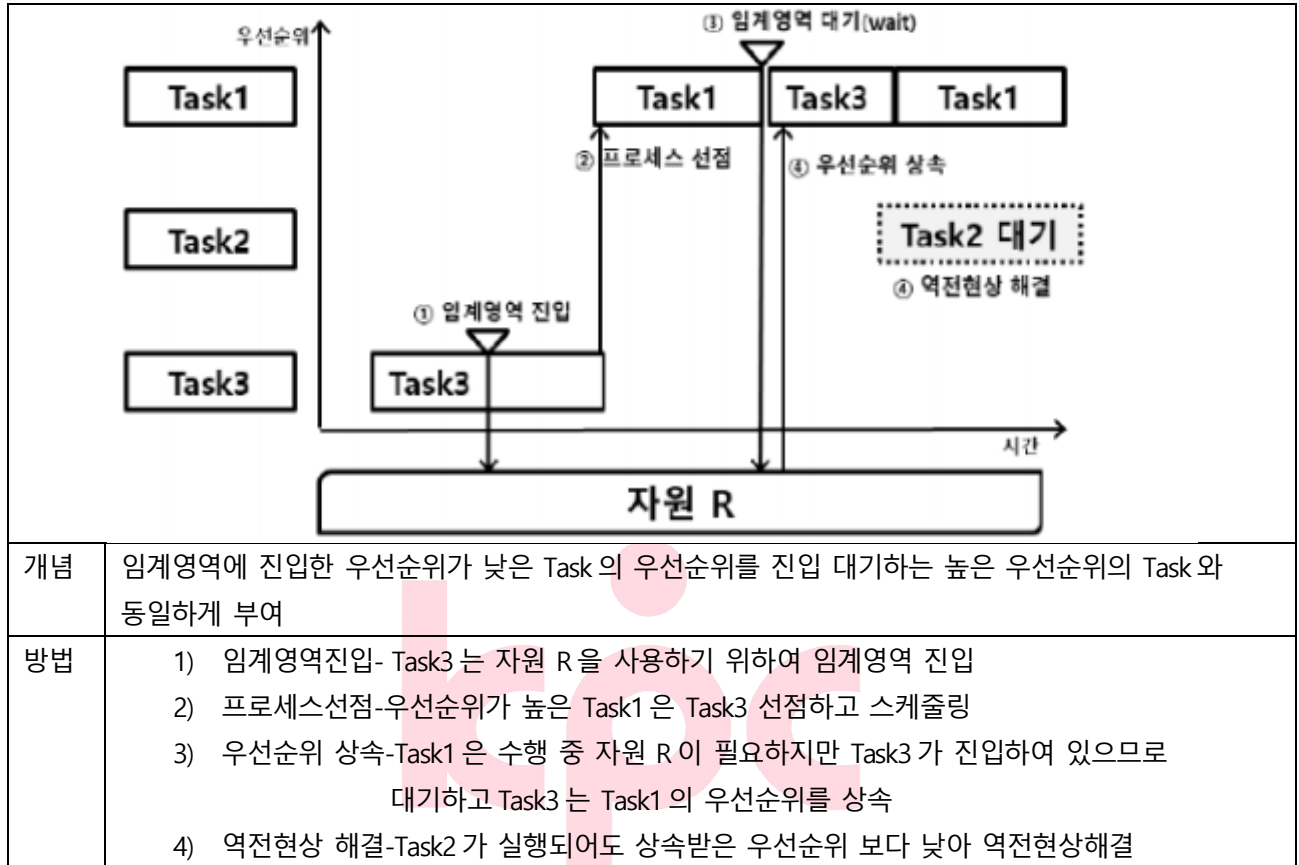
나. 우선순위올림(Priority Celling) 기법



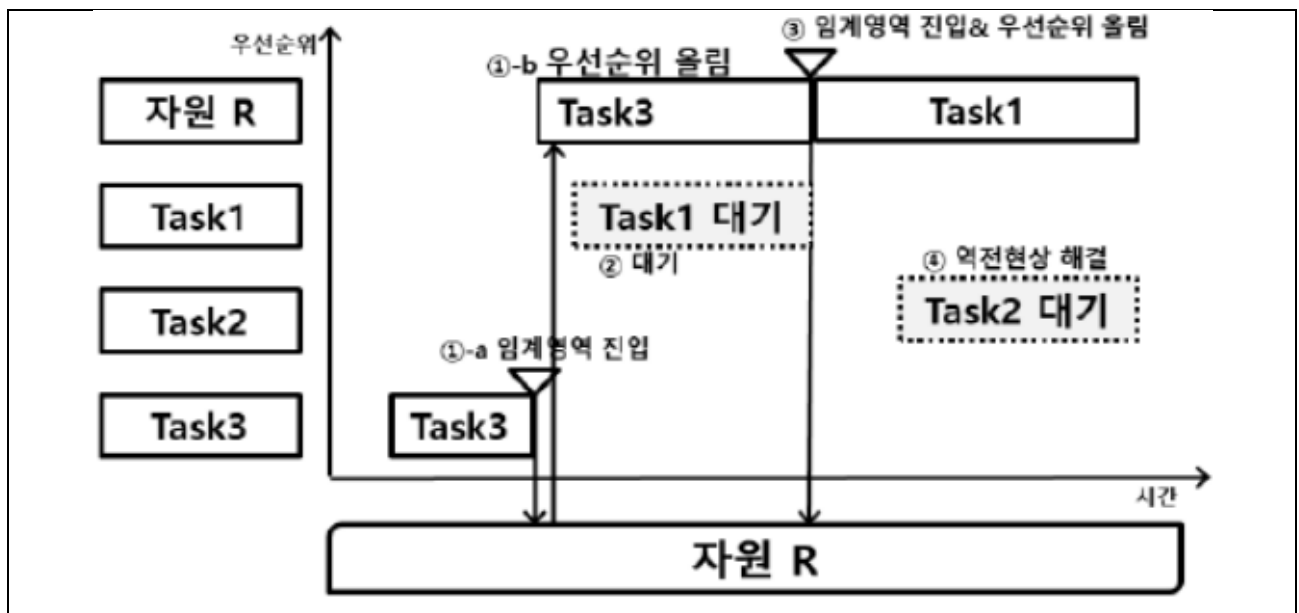
- 어떤 스레드가 세마포어(뮤텍스)를 소유하고 있는 동안은 지정된 우선순위에서 동작
- 단, 스레드의 우선순위가 지정된 우선순위 올림 값보다 작은 경우에만 우선순위 변경
- 세마포어(뮤텍스)를 이미 잠근 스레드는 다른 스레드들에 의해 선점되지 않고 자신의 작업을 종료가능

IV. 우선순위 상속기법과 우선순위 올림기법으로 해결 사례설명

가. 우선순위상속(Priority Inheritance)기법



나. 우선순위올림(Priority Celling) 기법



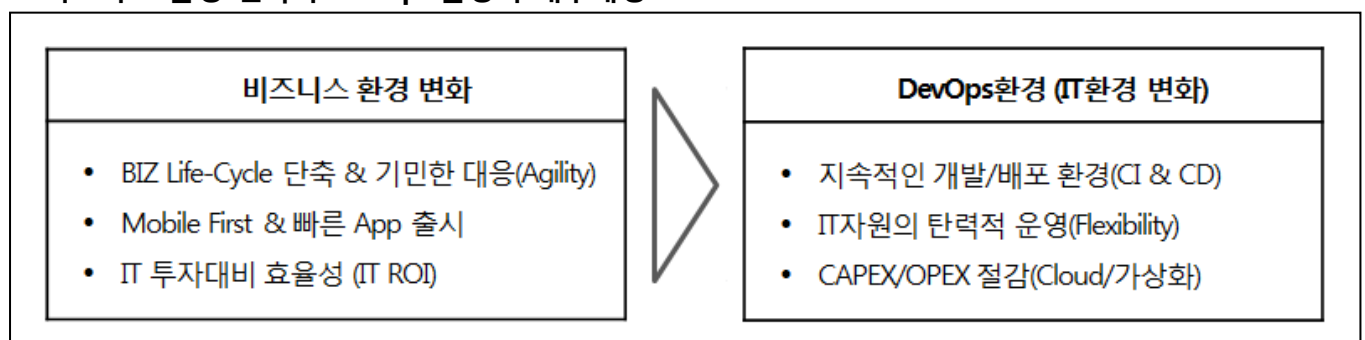
개념	임계영역의 자원 R 에 우선순위를 부여하고 해당 임계영역에 진입하는 Task 의 우선순위를 자원의우선순위로 올림
방법	1) 우선순위 올림-Task3 는 자원 R 을 사용하기 위하여 임계영역 진입하여 자원 R 의 우선순위로 올림 2) 대기-우선순위가 높은 Task1 은 자원 R 의 우선순위보다 낮으므로 대기 3) 임계영역 진입-Task1 은 Task3 가 종료된 이후 임계영역 진입하고 자원 R 의 우선순위로 올림 4) 역전현상 해결-Task2 가 실행되어도 올림 된 우선순위 보다 낮아 역전현상해결

기출풀이 의견

기본토픽으로 우선순위 역전을 알고 있지만, 출제자는 사례기반으로 답안을 요청했습니다. 답안 작성 조건인 Task1, Task2, Task3와 사례를 작성하고, 사례기반으로 우선순위 역전 해결 기법의 개념과 1단락의 우선순위 역전현상을 제시한 기법을 활용하여 해결 사례를 작성하시면 좋을 거 같습니다.

4	도커(Docker)		
문제	최근 비즈니스환경은 고객을 중심으로 빠르게 변화하고 있다. DevOps 환경에 대해 다음을 설명하시오. 가. Hypervisor 의 유형 나. Hypervisor 와 Docker 비교 다. Docker 컨테이너 배포절차		
도메인	디지털신서비스	난이도	★ ★ ★ ☆ ☆ (별 5 개 기준)
출제의도	최근 대두되고 있는 DevOps 가상화기술인 Docker 에 대한 이해 및 기존 서버가상화와 비교		
핵심 내용 키워드	Native/Hosted, Monolithic/Micro, HW/OS 레벨 가상화		
목차예시	1. 비즈니스 환경 변화와 DevOps 환경의 대두배경 2. 하드웨어 레벨의 가상화, Hypervisor 의 유형 3. Hypervisor 와 Docker 비교 4. Docker 컨테이너 배포절차		
채점 점수 가이드	① 개념, 이해 부족 (1~8점) ② 기본 이해 수준 (9~14점) ③ 정확한 기술 제시 (14~15점) ④ 추가요소 설명(+α)		
참고문헌	모의고사 해설지, 인터넷 검색(https://www.docker.com/)		
출제자	임 OO 기술사(컴퓨터시스템응용기술사 / bright2hyun@gmail.com)		

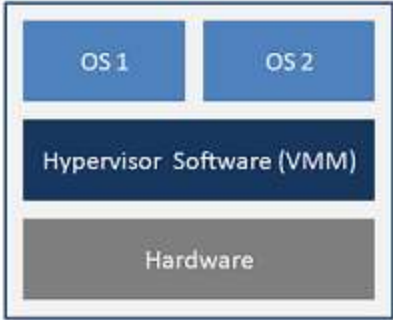
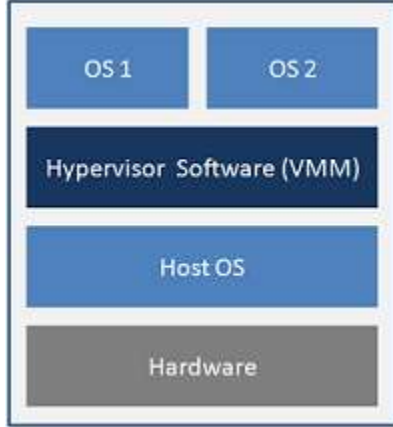
1. 비즈니스 환경 변화와 DevOps 환경의 대두배경



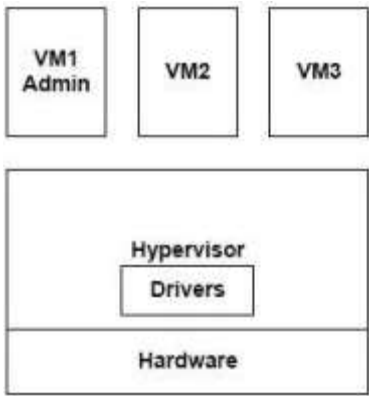
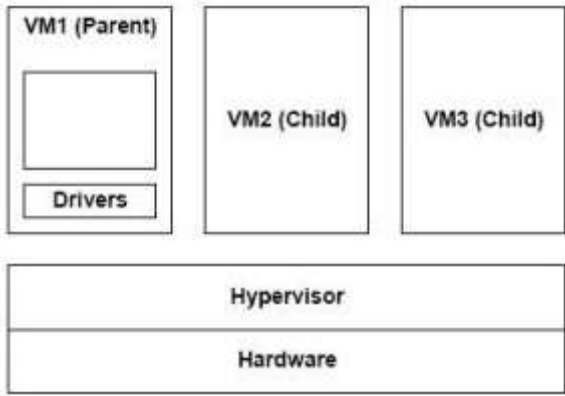
- 비즈니스 Needs 에 보다 기민하게 대응하고 서비스 딜리버리를 최적화하는 IT 환경 변화 요구
- IT 환경은 지속적인 배포(Continuous Delivery) 환경을 제공하는 DevOps 로 변화 중이며, 하이퍼바이저의 한계를 극복하기 위해 경량화된 가상화 기술인 도커(Docker) 각광

2. 하드웨어 레벨의 가상화, Hypervisor의 유형

가. Hypervisor의 방식별 유형

구분	Type1(Native/Bare-Metal)	Type2(Hosted)
개념도	 하드웨어 위에서 2 레벨 수준으로 실행	 하드웨어 위에서 3 레벨 수준으로 실행
특징	- VMM 을 물리 컴퓨터의 H/W 상에 직접 동작시키는 방식 - 기업의 데이터센터분야 이용 - OS 오버헤드 감소로 빠른 속도 제공	- 호스트 OS 위에 VMM 이 인스톨 되어져 이 VMM 위에 게스트 OS 를 동작시키는 방식 - 가상 PC 나 워크스테이션 클라이언트 (USER) 가상화에 주로 이용
장점	- 호스트 OS 불필요 - 호스트형 가상화에 비해 오버헤드 적음 - 물리 컴퓨터 리소스의 관리가 유연	게스트 OS 의 제약이 없어 Windows 에서 FreeBSD 까지 다양한 OS 지원
단점	자체 관리기능이 없어 별도의 관리콘솔 (내지는 관리 컴퓨터)가 필요	물리하드웨어를 에뮬레이팅 하기 때문에 오버헤드 큼

나. H/W 드라이버 레벨에 따른 유형

구분	Monolithic Kernel	Micro Kernel
개념도		
구조	Hypervisor 내부에 드라이버를 포함하는 구조	드라이버를 Hypervisor 에서 분리하여 가상머신 내에 위치시킨 구조
특징	- 운영체제 기능 대부분이 커널이 들어감 - 자체 Hypervisor 가 시스템의 최고 권한 모드에서 실행(most privileged mode)	IPC (Inter-Process Communication), 스케줄링, 기본 입출력, 메모리관리 같은 기본 기능만 제공

	• 자체 드라이버 모델 포함	• 다계층 방어(Defense in depth) 가능 • H/W 를 통한 방어, 공격 취약점의 감소 • Drivers 는 Guest 내에서 실행
장/단점	• 구현 간단, 효율적 자원 관리 가능 • 다양한 환경 적용이 어렵고 볼륨이 상대적으로 커질 수 있음. • 드라이버의 문제가 전체 서비스에 영향을 끼칠 수 있는 단점 존재	• 드라이버와 Hypervisor 가 분리되어 신뢰성과 보완성, 확장성을 강화 • 기능확장, 재사용, 부분 모듈 교체 가능하지만 통신에 대한 오버헤드 증가
사례	• VMWare	• Windows Server 2008 의 가상화 기술

3. Hypervisor 와 Docker 비교

가. Hypervisor 와 Docker 개념 비교

구분	Hypervisor	Docker
개념도	각각은 독립적인 가상머신 (오버헤드가 큼) 가상머신 App App OS 하이퍼 바이저 호스트 OS	도커가 컨테이너를 작성 각각은 단순한 프로세스의 집합 (오버헤드가 적음) 도커 Docker App App App App 컨테이너 OS(리눅스)
개념	호스트 운영체제 공유와 어플리케이션 격리	호스트 컴퓨터의 운영체제 상에 다수의 운영체제를 가상화 (Lightweight virtualization)

- 하이퍼바이저는 H/W 레벨의 가상화이며, 도커는 OS 레벨의 경량화된 가상화 기술임

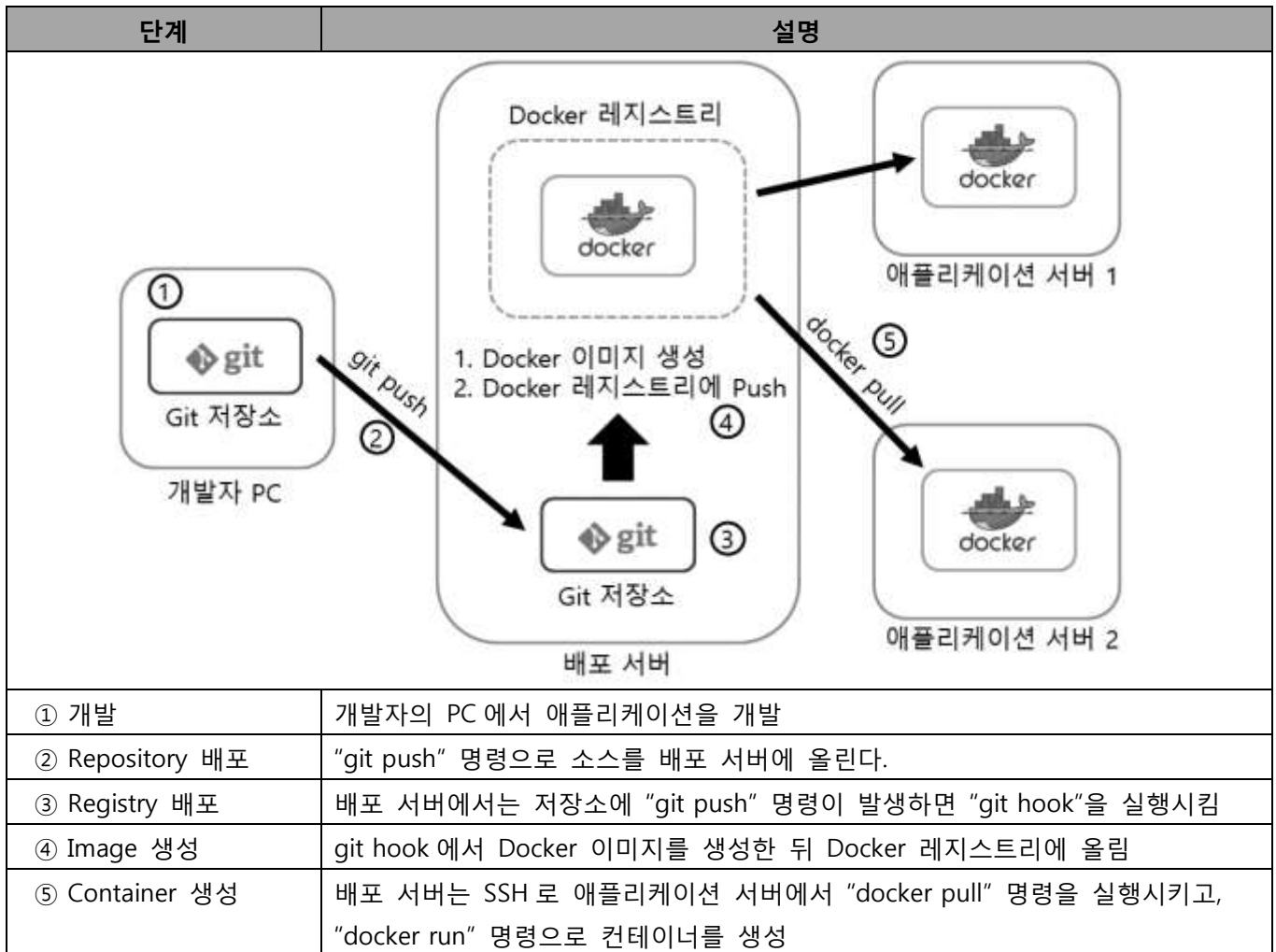
나. Hypervisor 와 Docker 장단점 비교

구분	Hypervisor	Docker
기술	• Hypervisor - H/W 레벨 가상화	• Libcontainer(LXC) - OS 레벨 가상화
장점	• 높은 집적도로 데이터센터의 공간 절약 • 하드웨어의 가동률을 향상 • 다양한 OS 지원	• 빠른 기동시간 (밀리세컨드 단위) • 자원 소비율이 적음 • 이미지의 Copy on write 방식으로 디스크 공간을 거의 쓰지 않음
단점	• 오버헤드가 큼 • 자원 소비율이 높음 (과도한 메모리, 디스크 사용) • 느린 기동시간 (수초~수분 소요)	• Host OS 에 종속적(Linux Kernel) • 자원의 격리와 제한이 어려운 점 • 스토리지 성능이 좋지 않은 점

- 도커는 하이퍼바이저의 자원의 비효율적 분배라는 단점을 극복하고, 애플리케이션 격리라는 장점을 극대화

4. Docker 컨테이너 배포절차

가. Docker 컨테이너 배포절차



- 일반적으로 Dockerfile – build – push 절차로 컨테이너가 배포되며, push – run 명령을 통해 실행

나. 도커의 기대효과 및 구현사례

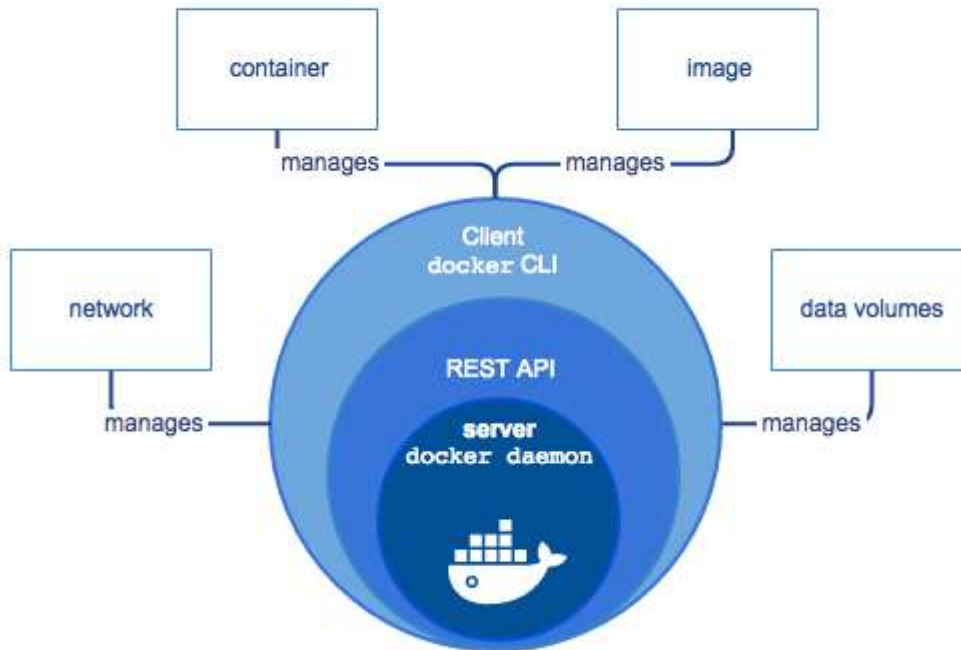
기대 효과	DevOps	개발과 운영의 호환성 증가
	성능 개선	Guest OS 계층이 없기에 더 가볍고, 빠른 성능
	플랫폼 비종속성	컨테이너는 운영체제 커널에 상관없이 실행
구현 사례	Google	지메일부터 자사의 서비스를 관리하기 위해 약 20 억개 컨테이너 운영
	Microsoft	Windows Server 2016 에서 도커 적용
	Amazon	AWS 에서 도커를 활용 및 기능 지원

고득점 전략 및	최근 대두되고 있는 DevOps 환경에 대한 부각배경 및 구현기술로 도커(Docker)에 대한 이해 및 기존 서버가상화와의 차이점 제시
-------------	---

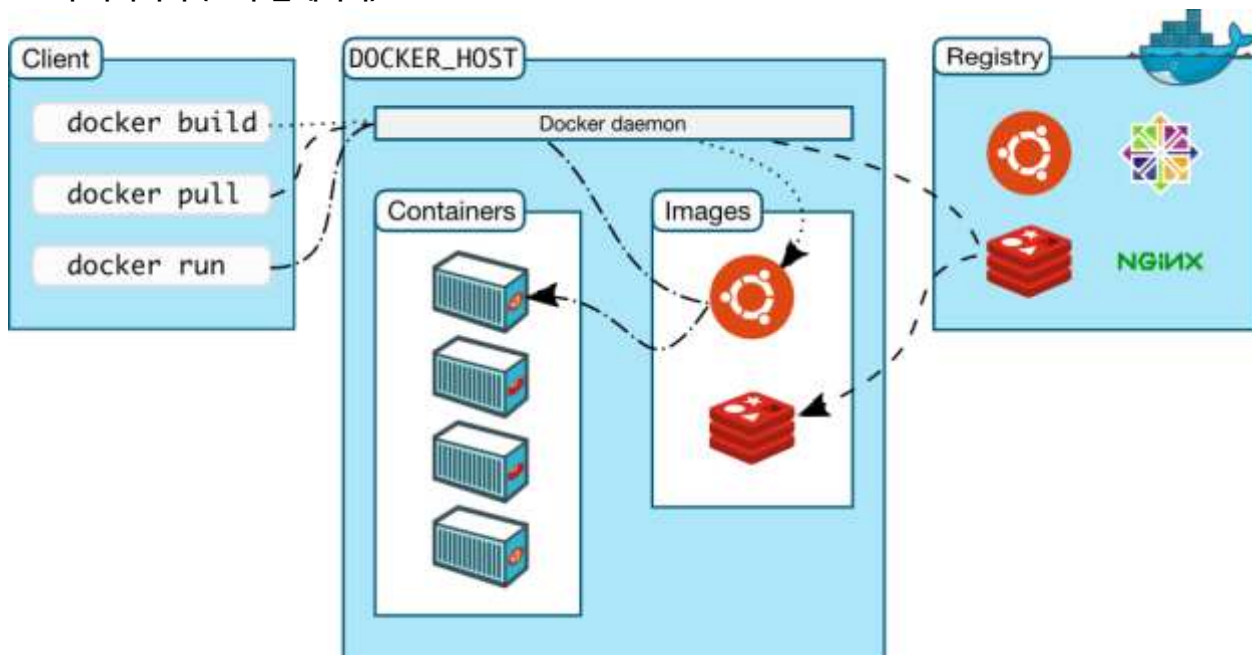
학습가이드

[기타자료]

<도커 엔진 구성요소 (도커 홈페이지)>



<도커 아키텍처 (도커 홈페이지)>



5	Auto Scale
문제	5. 최근 부하 분산을 위하여 클라우드 서비스에서 Auto Scale 기능을 많이 활용한다. 다음에 대하여 설명하시오. 가. Auto Scale Up 과 Auto Scale Out 아키텍처 비교 나. 스케줄 기반의 Auto Scale 과 부하 기반의 Auto Scale 기능 비교
도메인	디지털 서비스
정의	클라우드 환경에서 인스턴스의 부하를 모니터링해서 특정 이벤트에 대해 인스턴스를 추가 하거나 삭제 등의 관리를 위한 기능
키워드	Scale Up, Scale Out, Scale In, Scale Down, On-Demand, Load Balancer, Scheduler
출제의도분석	클라우드 서비스 사용자가 증가함에 따라 부하 분산을 위해 자주 활용 되고 있는 기술에 대한 지식 확인
답안작성 전략	비교 문제이므로 비교 대상에 대한 비교 항목 다양화 필요
참고문헌	AWS EC2 Auto Scaling (https://docs.aws.amazon.com/ko_kr/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html)
풀이 기술사님	서경석 기술사 (제 119 회 정보관리 기술사 / akslemf@naver.com)

1. 클라우드 환경의 능동적 자원 할당, 오토 스케일의 개요

가. 오토 스케일(Auto Scale)의 정의

- 클라우드 환경에서 요청이 늘어나면 이를 수용하기 위해 추가 자원의 할당, 회수를 통해 자원을 효율적으로 활용 하여 On-Demand 서비스를 지원 하는 기술
- 서비스 제공 시 관리 기준에 따라 자동으로 할당 자원을 추가 할당, 회수함으로써 자원을 효율적으로 활용 하기 위한 클라우드 서비스기반의 기술

나. Auto Scale 의 분류

구분	세부	설명
관리 기준	- 스케줄	- 서비스 관점의 스케줄 기준으로 관리
	- 시스템 부하	- 서비스 제공 시 부하 기준으로 관리
구현 방식	- 컨테이너 기반	- 컨테이너형 어플리케이션 플랫폼으로 구축
	- VM 기반	- VM 자체에서 제공 하는 기능 활용 구축
자원 할당	- Scale Up	- 인스턴스 자체의 성능 향상 자원 할당
	- Scale Down	- 인스턴스 성능 향상 자원 회수
	- Scale Out	- 추가 인스턴스 배정 통한 성능 향상
	- Scale In	- 추가 인스턴스 회수 통한 자원 효율화

- 관리 기준, 구현 방식, 자원 할당 등 다양한 방식의 Auto Scale 적용 가능

2. Auto Scale Up 과 Auto Scale Out 아키텍처 비교

가. Auto Scale Up 과 Auto Scale Out 의 개념도 및 개념 비교

구분	세부	설명
Auto Scale Up	개념도	
	개념	- 클라우드 환경에서 요청이 증가 하는 경우 CPU, Memory, Disk 등의 자원을 증설 하는 기술
Auto Scale Out	개념도	
	개념	- 클라우드 환경에서 요청이 증가 하는 경우 추가적인 Instance를 할당 하여 자원을 확보 하는 기술

- Auto Scale Up 과 Auto Scale Out 은 자원의 할당 방식에서 가장 큰 차이 보유

나. Auto Scale Up 과 Auto Scale Out 상세 비교

항목	Auto Scale Up	Auto Scale Out
구현 난이도	- 단순 추가 자원 할당으로 용이	- 상대적 구현 난이도 높음
비용	- 성능 증가에 따른 비용 증가 큼	- 성능에 따른 비용 부담 적음
자원 효율성	- 자원 할당, 회수 제약 있음	- 자원 할당, 회수 용이
장애 대응	- 장애 발생 시 대응 어려움	- 분산 처리 통한 장애 대응 가능
관리	- 관리 대상이 적어 관리 용이	- 다수의 인스턴스 관리 어려움
확장성	- 수직적 확장의 한계 존재	- 수평적 확장 통한 확장 용이
적용 사례	- 전통적 구조의 시스템	- MSA

- 확장성, 비용, 장애 대응 측면에서 유리한 Auto Scale Out 이 주로 사용되는 추세

3. 스케줄 기반의 Auto Scale 과 부하 기반의 Auto Scale 개념 및 기능 비교

가. 스케줄 기반의 Auto Scale 과 부하 기반의 Auto Scale 개념

구분	세부	설명
스케줄 기반	개념도	
	개념	- 이벤트 발생 일정을 미리 알고 있는 경우에 해당 이벤트 발생 시점에 Auto Scale을 Scheduler에 입력 하여 자원 할당 기술
부하 기반	개념도	
	개념	- 미리 스케줄링 하지 않고 부하 발생 시점에 Load Balancer 모니터링을 통해 동적으로 자원을 할당 하는 기술

- 시스템 환경이나 업무 특성에 맞춘 Auto Scale 기술 선택 필요

나. 스케줄 기반의 Auto Scale 과 부하 기반의 Auto Scale 기능 비교

항목	스케줄 기반	부하 기반
이벤트 예약	- 특정 일정 기준 예약 용이	- 추가적인 시스템 구성 필요
부하 관리	- 부하 관리 기능 부족	- Load Balancer 활용 부하 관리
모니터링	- 체계적인 모니터링 진행 가능	- 현재 발생 기준 모니터링 가능
실시간 대응	- 실시간 이벤트 대응 어려움	- 부하 관리 통한 대응 가능
필요 자원 산정	- 이벤트 시점 필요 자원 산정 가능	- 정확한 필요 자원 산정 어려움
장애 발생 예측	- 스케줄 예측이 잘못된 경우 장애 발생 예측 어려움	- 부하 측정 통한 장애 발생 예측 가능
적용 사례	- 세일 기간 사전 자원 확보	- 동시 접속자 증가 시 자원 확보

- 이벤트 추이가 예측 가능한 경우 스케줄 기반, 변동이 큰 경우는 부하 기반 Auto Scale 적용

4. Auto Scale 서비스 적용 방안

단계	작업	설명
사전 검토	- 적용 대상 검토	- Auto Scale 기능 필요 여부 확인
	- 일정, 비용 확인	- 일정을 고려한 투자 비용 분석
구축	- 스케줄, 부하 기반 결정	- 업무에 적합한 방식 결정
	- Auto Scale 환경 구축	- 필요 자원 확보 및 조직 구성
운영	- 성능 모니터링	- 기능 정상 수행 여부 및 오류 확인
	- 자원 최적화	- 성능 모니터링 결과 반영 자원 효율화

- 효율적 Auto Scale 기능 적용 통한 추가적 기업 이윤 발생 기대

”끝”

문 제	6. 엣지(Edge) 컴퓨팅과 클라우드 컴퓨팅의 차이점		
출 제 영 역	디지털서비스	난 이 도	★★★☆☆ (별 5 개 기준)
출 제 배 경	실시간 데이터 처리 요구가 증가함에 따라 컴퓨팅 패러다임이 어떻게 진화하는지에 대한 이해		
출 제 빈 도	135 회 컴시응 3 교시, 125 회 컴시응 1 교시, 114 회 정보관리 1 교시		
참 고 자 료	- ETRI 지식공유플랫폼 컴퓨팅 컨티뉴엄 - https://blogs.nvidia.co.kr/blog/difference-between-cloud-and-edge-computing/ - https://news.samsungdisplay.com/29813/		
Key word	실시간 처리, 분산형 컴퓨팅, 초저지연, 대규모 처리 및 분석, 중앙집중형 컴퓨팅, 통합 처리		
풀이 기술사님	이영학 PPE (138 회 정보관리기술사 / dudgkr159@naver.com)		

1. 컴퓨팅 자원 효율적 제공, 엣지 컴퓨팅과 클라우드 컴퓨팅 개념 비교 설명

엣지 컴퓨팅	클라우드 컴퓨팅
- 데이터가 발생하는 단말 근처에서 데이터를 직접 처리하여 초저지연 및 실시간 서비스를 제공하는 "분산형" 컴퓨팅 기술	- 인터넷 기반의 중앙 데이터센터에서 서버, 스토리지 등의 IT 자원을 온디맨드 형태로 제공하는 "중앙 집중형" 컴퓨팅 기술
- 엣지 컴퓨팅은 초저지연 기반의 분산처리에 강점, 클라우드 컴퓨팅은 중앙 집중형 자원 활용에 강점	

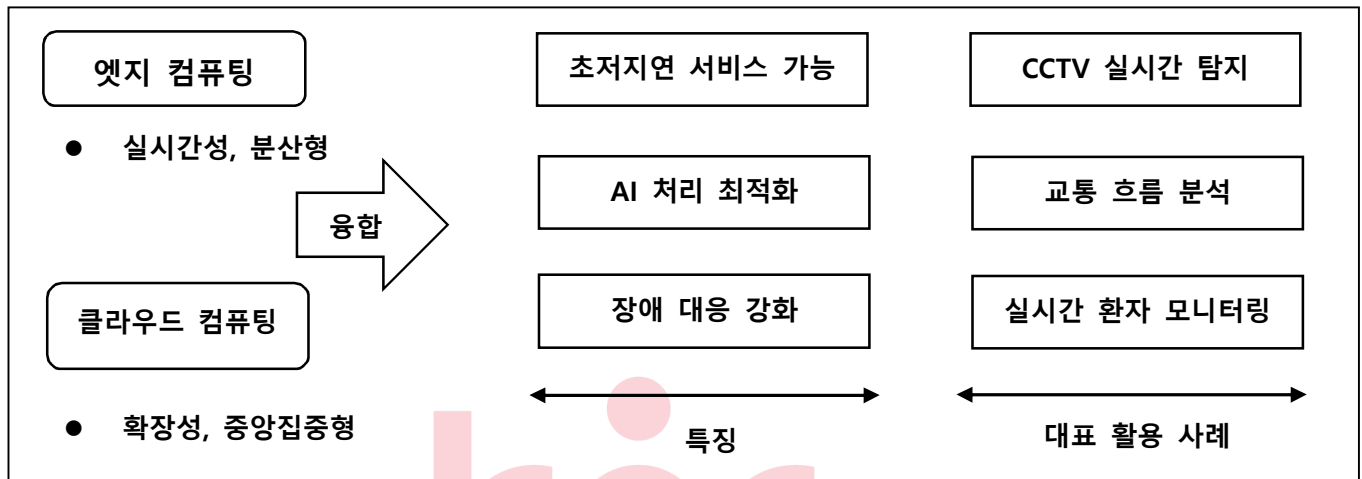
2. 엣지 컴퓨팅과 클라우드 컴퓨팅 상세 비교 설명

구분	엣지 컴퓨팅	클라우드 컴퓨팅
개념도		
목적	- 데이터 실시간 처리	- 대규모 데이터 저장 및 분석
구조	- 데이터 발생 지점 분산형 구조	- 중앙 집중형 통합 처리 구조
처리 위치	- IoT Device, Edge Server, Gateway	- 원격 Cloud Data Center
기술요소	- MEC, Edge AI, Edge Gateway, KubeEdge - Edge Orchestrator	- 가상화, IaaS, PaaS, Data Lake, Kubernetes - Cloud Orchestrator
특징	- 경량화 자원 사용, 초저지연, 실시간성	- 대규모 GPU/HPC 활용, NW 연결 필요

	- 모델 정확도, 데이터 주권	- 유연한 가격 정책, IT 관리 단순화
응답 속도	- 실시간 데이터 처리 가능	- 다소 지연 발생
확장성	- 상대적 확장성 낮음	- 상대적 확장성 높음
사례	- 자율주행, 스마트팩토리, AR/VR	- 빅데이터 분석, 머신러닝 모델 훈련, ERP

- 최근 클라우드 컴퓨팅의 확장성과 엣지 컴퓨팅의 실시간성을 결합하여 “하이브리드 아키텍처” 중심으로 발전하며, AI 및 6G 기반 초연결 서비스의 핵심 인프라로 부상

3. 하이브리드 아키텍처 설명



- 하이브리드 아키텍처는 AI 기반 자동 제어, 실시간 장애 대응 등 AX 시대에서 중요한 지능형 컴퓨팅 구조 기대 "끝"

기출풀이 의견

- 엣지 컴퓨팅의 분산형 처리 구조, 클라우드 컴퓨팅의 중앙 집중형 처리 구조의 차이를 명확화해 주시고 각 컴퓨팅 구조가 가진 핵심 특징에 대해 구분하여 제시하시면 되겠습니다. 추가로 두 컴퓨팅 구조에 대해 하이브리드 구조로 발전하는 동향을 제시하는 것을 추천해 드립니다.